云ERP（零售批发版）源代码

版本号：v1.0.0

著作权人：晋城市掌上乾坤网络科技有限公司

源程序量：约 8700 行

前端：React 18 + TypeScript ｜ 后端：NestJS + TypeScript + Prisma + PostgreSQL

本源代码文档包含系统全部前后端有效代码，用于计算机软件著作权登记。

前端：apps/web/src（44个文件） 后端：apps/api/src（49个文件）

云ERP（零售批发版）
版本号：v1.0.0
著作权人：_____
完成日期：_____年_____月_____日

本源代码文档包含系统全部前后端有效代码，共约 8700 行。
代码由 React 18 + TypeScript（前端）和 NestJS + TypeScript（后端）编写。

前端源代码 (apps/web/src) — 共 44 个文件

src/App.tsx

```
import { Routes, Route, Navigate } from 'react-router-dom';
import { useAuthStore } from './stores/auth';
import MainLayout from './layouts/MainLayout';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';
import WarehouseList from './pages/settings/WarehouseList';
import ProductList from './pages/settings/ProductList';
import CategoryManage from './pages/settings/CategoryManage';
import CustomerList from './pages/settings/CustomerList';
import SupplierList from './pages/settings/SupplierList';
import AccountList from './pages/settings/AccountList';
import PurchaseOrderList from './pages/purchase/PurchaseOrderList';
import PurchaseStockInList from './pages/purchase/PurchaseStockInList';
import PurchaseReturnList from './pages/purchase/PurchaseReturnList';
import SaleOrderList from './pages/sale/SaleOrderList';
import SaleStockOutList from './pages/sale/SaleStockOutList';
import SaleReturnList from './pages/sale/SaleReturnList';
import InventoryList from './pages/inventory/InventoryList';
import StockCheckList from './pages/inventory/StockCheckList';
import StockTransferList from './pages/inventory/StockTransferList';
import InventoryWarning from './pages/inventory/InventoryWarning';
import VoucherList from './pages/finance/VoucherList';
import PaymentList from './pages/finance/PaymentList';

function App() {
  const { token } = useAuthStore();

  if (!token) {
    return <Login />;
  }

  return (
    <MainLayout>
      <Routes>
        <Route path="/" element={<Navigate to="/dashboard" replace />} />
        <Route path="/dashboard" element={<Dashboard />} />

        {/* 采购管理 */}
        <Route path="/purchase/orders" element={<PurchaseOrderList />} />
        <Route path="/purchase/stock-ins" element={<PurchaseStockInList />} />
        <Route path="/purchase/returns" element={<PurchaseReturnList />} />

        {/* 销售管理 */}
        <Route path="/sale/orders" element={<SaleOrderList />} />
        <Route path="/sale/stock-outs" element={<SaleStockOutList />} />
        <Route path="/sale/returns" element={<SaleReturnList />} />

        {/* 库存管理 */}
        <Route path="/inventory/list" element={<InventoryList />} />
        <Route path="/inventory/check" element={<StockCheckList />} />
        <Route path="/inventory/transfer" element={<StockTransferList />} />
        <Route path="/inventory/warning" element={<InventoryWarning />} />

        {/* 财务管理 */}
        <Route path="/finance/vouchers" element={<VoucherList />} />
        <Route path="/finance/payments" element={<PaymentList />} />

        {/* 基础资料 */}
        <Route path="/settings/basic/warehouse" element={<WarehouseList />} />
        <Route path="/settings/basic/product" element={<ProductList />} />
        <Route path="/settings/basic/category" element={<CategoryManage />} />
        <Route path="/settings/basic/customer" element={<CustomerList />} />
        <Route path="/settings/basic/supplier" element={<SupplierList />} />
        <Route path="/settings/basic/account" element={<AccountList />} />
      </Routes>
    </MainLayout>
  );
}

export default App;
```

src/api/account.ts

```
import request from './request';
```

```
export interface Account {
  id: string;
  code: string;
  name: string;
  type?: string;
  bankName?: string;
  bankAccount?: string;
  initialBalance: number;
  currentBalance: number;
  status: string;
  isDefault: boolean;
  sortOrder: number;
}

export interface AccountQuery {
  keyword?: string;
  type?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}

export const accountApi = {
  list: (params: AccountQuery) =>
    request.get<{ list: Account[]; pagination: any }>('/accounts', { params }),

  listSimple: () =>
    request.get<Account[]>('/accounts/simple'),

  get: (id: string) =>
    request.get<Account>(`/accounts/${id}`),

  create: (data: Partial<Account>) =>
    request.post<Account>('/accounts', data),

  update: (id: string, data: Partial<Account>) =>
    request.put<Account>(`/accounts/${id}`, data),

  delete: (id: string) =>
    request.delete<void>(`/accounts/${id}`),

  updateStatus: (id: string, status: string) =>
    request.patch<void>(`/accounts/${id}/status`, { status }),
};
```

src/api/auth.ts

```
import request from './request';

export interface LoginParams {
  username: string;
  password: string;
}

export interface User {
  id: string;
  username: string;
  realName: string;
  roles: string[];
}

export interface LoginResult {
  token: string;
  refreshToken: string;
  expiresIn: number;
  user: User;
  tenant: {
    id: string;
    name: string;
    code: string;
  };
}

export const authApi = {
  login: (params: LoginParams) =>
    request.post<LoginResult>('/auth/login', params),

  logout: () =>
    request.post('/auth/logout'),

  refreshToken: (refreshToken: string) =>
    request.post('/auth/refresh', { refreshToken }),

  getCurrentUser: () =>
    request.get<User>('/auth/current-user'),
};
```

src/api/customer.ts

```
import request from './request';
```

```
export interface Customer {
  id: string;
  code: string;
  name: string;
  contact?: string;
  phone?: string;
  mobile?: string;
  email?: string;
  address?: string;
  customerType?: string;
  creditLimit?: number;
  status: string;
}

export interface CustomerQuery {
  keyword?: string;
  customerType?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}

export const customerApi = {
  list: (params: CustomerQuery) =>
    request.get<{ list: Customer[]; pagination: any }>('/customers', { params }),

  get: (id: string) =>
    request.get<Customer>(`/customers/${id}`),

  create: (data: Partial<Customer>) =>
    request.post<Customer>('/customers', data),

  update: (id: string, data: Partial<Customer>) =>
    request.put<Customer>(`/customers/${id}`, data),

  delete: (id: string) =>
    request.delete<void>(`/customers/${id}`),

  updateStatus: (id: string, status: string) =>
    request.patch<void>(`/customers/${id}/status`, { status }),
};
```

src/api/finance.ts

```
import request from './request';

export interface Voucher {
  id: string;
  voucherNo: string;
  voucherDate: string;
  voucherType: string;
  totalDebit: number;
  totalCredit: number;
  status: string;
  remark?: string;
  items: VoucherItem[];
  createdAt: string;
}

export interface VoucherItem {
  id: string;
  accountId: string;
  account?: { id: string; code: string; name: string };
  direction: string;
  amount: number;
  summary?: string;
}

export interface Payment {
  id: string;
  paymentNo: string;
  paymentDate: string;
  paymentType: string;
  amount: number;
  accountId: string;
  account?: { id: string; name: string };
  customerId?: string;
  customer?: { id: string; name: string };
  supplierId?: string;
  supplier?: { id: string; name: string };
  status: string;
  remark?: string;
}

export interface AccountSubject {
  id: string;
  code: string;
  name: string;
```

```
  type?: string;
  direction?: string;
  parentId?: string;
  children?: AccountSubject[];
}

export const financeApi = {
  // 会计科目
  listSubjects: () => request.get<AccountSubject[]>('/finance/subjects'),
  createSubject: (data: any) => request.post<AccountSubject>('/finance/subjects', data),
  initSubjects: () => request.post('/finance/subjects/init', {}),

  // 凭证
  listVouchers: (params: any) =>
    request.get<{ list: Voucher[]; pagination: any }>('/finance/vouchers', { params }),
  getVoucher: (id: string) => request.get<Voucher>(`/finance/vouchers/${id}`),
  createVoucher: (data: any) => request.post<Voucher>('/finance/vouchers', data),
  auditVoucher: (id: string) => request.patch<Voucher>(`/finance/vouchers/${id}/audit`, {}),
  postVoucher: (id: string) => request.patch<Voucher>(`/finance/vouchers/${id}/post`, {}),
  deleteVoucher: (id: string) => request.delete<void>(`/finance/vouchers/${id}`),

  // 收付款
  listPayments: (params: any) =>
    request.get<{ list: Payment[]; pagination: any }>('/finance/payments', { params }),
  createPayment: (data: any) => request.post<Payment>('/finance/payments', data),
  auditPayment: (id: string) => request.patch<Payment>(`/finance/payments/${id}/audit`, {}),

  // 报表
  getProfitStatement: (startDate: string, endDate: string) =>
    request.get<any>('/finance/reports/profit', { params: { startDate, endDate } }),
};
```

src/api/inventory.ts

```
import request from './request';

export interface Inventory {
  id: string;
  productId: string;
  product?: { id: string; name: string; code: string; spec?: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  quantity: number;
  frozenQuantity: number;
  avgCostPrice: number;
  lastInPrice?: number;
  lastOutPrice?: number;
}

export interface StockCheck {
  id: string;
  orderNo: string;
  checkDate: string;
  warehouseId: string;
  warehouse?: { id: string; name: string };
  totalAmount: number;
  status: string;
  remark?: string;
  items: StockCheckItem[];
}

export interface StockCheckItem {
  id: string;
  productId: string;
  product?: { id: string; name: string };
  bookQuantity: number;
  checkQuantity: number;
  diffQuantity: number;
  costPrice: number;
  diffAmount: number;
  reason?: string;
}

export interface StockTransfer {
  id: string;
  orderNo: string;
  transferDate: string;
  fromWarehouseId: string;
  fromWarehouse?: { id: string; name: string };
  toWarehouseId: string;
  toWarehouse?: { id: string; name: string };
  totalAmount: number;
  status: string;
  remark?: string;
  items: StockTransferItem[];
}
```

```
export interface StockTransferItem {
  id: string;
  productId: string;
  product?: { id: string; name: string };
  quantity: number;
  costPrice: number;
  amount: number;
}

export interface InventoryQuery {
  keyword?: string;
  productId?: string;
  warehouseId?: string;
  stockStatus?: string;
  page?: number;
  pageSize?: number;
}

export interface InventorySummary {
  totalProductTypes: number;
  totalQuantity: number;
  lowStockCount: number;
  overStockCount: number;
}

export const inventoryApi = {
  list: (params: InventoryQuery) =>
    request.get<{ list: Inventory[]; pagination: any }>('/inventory', { params }),

  getSummary: () =>
    request.get<InventorySummary>('/inventory/summary'),

  getWarnings: () =>
    request.get<any[]>('/inventory/warnings'),

  // 盘点
  listChecks: (params: any) =>
    request.get<{ list: StockCheck[]; pagination: any }>('/inventory/checks', { params }),
  getCheck: (id: string) =>
    request.get<StockCheck>(`/inventory/checks/${id}`),
  createCheck: (data: any) =>
    request.post<StockCheck>('/inventory/checks', data),
  auditCheck: (id: string) =>
    request.patch<StockCheck>(`/inventory/checks/${id}/audit`, {}),
  approveCheck: (id: string) =>
    request.patch<StockCheck>(`/inventory/checks/${id}/approve`, {}),

  // 调拨
  listTransfers: (params: any) =>
    request.get<{ list: StockTransfer[]; pagination: any }>('/inventory/transfers', { params }),
  getTransfer: (id: string) =>
    request.get<StockTransfer>(`/inventory/transfers/${id}`),
  createTransfer: (data: any) =>
    request.post<StockTransfer>('/inventory/transfers', data),
  auditTransfer: (id: string) =>
    request.patch<StockTransfer>(`/inventory/transfers/${id}/audit`, {}),
};
```

src/api/product.ts

```
import request from './request';

export interface Product {
  id: string;
  categoryId?: string;
  category?: { id: string; name: string };
  code: string;
  name: string;
  spec?: string;
  unit?: string;
  purchasePrice?: number;
  salePrice?: number;
  costPrice?: number;
  minStock?: number;
  maxStock?: number;
  status: string;
  imageUrl?: string;
  description?: string;
}

export interface ProductCategory {
  id: string;
  parentId?: string;
  name: string;
  sortOrder: number;
  children?: ProductCategory[];
}
```

```
export interface ProductQuery {
  keyword?: string;
  categoryId?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}
export const productCategoryApi = {
  list: () =>
    request.get<ProductCategory[]>('/product-categories'),
  get: (id: string) =>
    request.get<ProductCategory>(`/product-categories/${id}`),
  create: (data: Partial<ProductCategory>) =>
    request.post<ProductCategory>('/product-categories', data),
  update: (id: string, data: Partial<ProductCategory>) =>
    request.put<ProductCategory>(`/product-categories/${id}`, data),
  delete: (id: string) =>
    request.delete<void>(`/product-categories/${id}`),
};
export const productApi = {
  list: (params: ProductQuery) =>
    request.get<{ list: Product[]; pagination: any }>('/products', { params }),
  get: (id: string) =>
    request.get<Product>(`/products/${id}`),
  create: (data: Partial<Product>) =>
    request.post<Product>('/products', data),
  update: (id: string, data: Partial<Product>) =>
    request.put<Product>(`/products/${id}`, data),
  delete: (id: string) =>
    request.delete<void>(`/products/${id}`),
  updateStatus: (id: string, status: string) =>
    request.patch<void>(`/products/${id}/status`, { status }),
};
```

src/api/purchase.ts

```
import request from './request';
export interface PurchaseOrderItem {
  id?: string;
  productId: string;
  product?: { id: string; name: string; code: string; spec?: string };
  warehouseId?: string;
  quantity: number;
  price: number;
  amount: number;
  taxRate?: number;
  taxAmount?: number;
  remark?: string;
}
export interface PurchaseOrder {
  id: string;
  orderNo: string;
  orderDate: string;
  supplierId: string;
  supplier?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  contact?: string;
  phone?: string;
  totalAmount: number;
  paidAmount: number;
  status: string;
  deliveryDate?: string;
  remark?: string;
  items: PurchaseOrderItem[];
  createdAt: string;
}
export interface PurchaseStockIn {
  id: string;
  orderNo: string;
  stockInDate: string;
  supplierId: string;
  supplier?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  totalAmount: number;
```

```
  paidAmount: number;
  status: string;
  remark?: string;
  items: PurchaseOrderItem[];
  createdAt: string;
}

export interface PurchaseReturn {
  id: string;
  orderNo: string;
  returnDate: string;
  supplierId: string;
  supplier?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  totalAmount: number;
  status: string;
  remark?: string;
  items: PurchaseOrderItem[];
  createdAt: string;
}

export interface PurchaseQuery {
  keyword?: string;
  supplierId?: string;
  warehouseId?: string;
  status?: string;
  startDate?: string;
  endDate?: string;
  page?: number;
  pageSize?: number;
}

export const purchaseApi = {
  // 采购订单
  listOrders: (params: PurchaseQuery) =>
    request.get<{ list: PurchaseOrder[]; pagination: any }>('/purchase/orders', { params }),
  getOrder: (id: string) =>
    request.get<PurchaseOrder>(`/purchase/orders/${id}`),
  createOrder: (data: Partial<PurchaseOrder>) =>
    request.post<PurchaseOrder>('/purchase/orders', data),
  updateOrder: (id: string, data: Partial<PurchaseOrder>) =>
    request.put<PurchaseOrder>(`/purchase/orders/${id}`, data),
  deleteOrder: (id: string) =>
    request.delete<void>(`/purchase/orders/${id}`),
  auditOrder: (id: string) =>
    request.patch<PurchaseOrder>(`/purchase/orders/${id}/audit`, {}),
  cancelOrder: (id: string) =>
    request.patch<PurchaseOrder>(`/purchase/orders/${id}/cancel`, {}),

  // 采购入库
  listStockIns: (params: PurchaseQuery) =>
    request.get<{ list: PurchaseStockIn[]; pagination: any }>('/purchase/stock-ins', { params }),
  getStockIn: (id: string) =>
    request.get<PurchaseStockIn>(`/purchase/stock-ins/${id}`),
  createStockIn: (data: Partial<PurchaseStockIn>) =>
    request.post<PurchaseStockIn>('/purchase/stock-ins', data),
  auditStockIn: (id: string) =>
    request.patch<PurchaseStockIn>(`/purchase/stock-ins/${id}/audit`, {}),

  // 采购退货
  listReturns: (params: PurchaseQuery) =>
    request.get<{ list: PurchaseReturn[]; pagination: any }>('/purchase/returns', { params }),
  createReturn: (data: Partial<PurchaseReturn>) =>
    request.post<PurchaseReturn>('/purchase/returns', data),
  auditReturn: (id: string) =>
    request.patch<PurchaseReturn>(`/purchase/returns/${id}/audit`, {}),
};
```

src/api/request.ts

```
import axios, { AxiosError, AxiosResponse } from 'axios';
import { message } from 'antd';
import { useAuthStore } from '@/stores/auth';

const request = axios.create({
  baseURL: '/api/v1',
  timeout: 30000,
});

// 请求拦截器
request.interceptors.request.use(
  (config) => {
    const token = useAuthStore.getState().token;
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
```

```
    }
    const tenantId = useAuthStore.getState().tenant?.id;
    if (tenantId) {
      config.headers['X-Tenant-ID'] = tenantId;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 响应拦截器
request.interceptors.response.use(
  (response: AxiosResponse) => response.data,
  (error: AxiosError<{ code: number; message: string }>) => {
    const { response } = error;

    if (response?.status === 401) {
      useAuthStore.getState().logout();
      message.error('登录已过期，请重新登录');
      window.location.href = '/login';
      return Promise.reject(error);
    }

    const msg = response?.data?.message || error.message || '请求失败';
    message.error(msg);

    return Promise.reject(error);
  }
);

export default request;
```

src/api/sale.ts

```
import request from './request';

export interface SaleOrderItem {
  id?: string;
  productId: string;
  product?: { id: string; name: string; code: string; spec?: string };
  warehouseId?: string;
  quantity: number;
  price: number;
  amount: number;
  taxRate?: number;
  taxAmount?: number;
}

export interface SaleOrder {
  id: string;
  orderNo: string;
  orderDate: string;
  customerId: string;
  customer?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  contact?: string;
  phone?: string;
  totalAmount: number;
  receivedAmount: number;
  status: string;
  deliveryDate?: string;
  remark?: string;
  items: SaleOrderItem[];
  createdAt: string;
}

export interface SaleStockOut {
  id: string;
  orderNo: string;
  stockOutDate: string;
  customerId: string;
  customer?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  totalAmount: number;
  receivedAmount: number;
  status: string;
  remark?: string;
  items: SaleOrderItem[];
  createdAt: string;
}

export interface SaleReturn {
  id: string;
  orderNo: string;
  returnDate: string;
```

```
  customerId: string;
  customer?: { id: string; name: string };
  warehouseId: string;
  warehouse?: { id: string; name: string };
  totalAmount: number;
  status: string;
  remark?: string;
  items: SaleOrderItem[];
  createdAt: string;
}

export interface SaleQuery {
  keyword?: string;
  customerId?: string;
  warehouseId?: string;
  status?: string;
  startDate?: string;
  endDate?: string;
  page?: number;
  pageSize?: number;
}

export const saleApi = {
  // 销售订单
  listOrders: (params: SaleQuery) =>
    request.get<{ list: SaleOrder[]; pagination: any }>('/sale/orders', { params }),
  getOrder: (id: string) =>
    request.get<SaleOrder>(`/sale/orders/${id}`),
  createOrder: (data: Partial<SaleOrder>) =>
    request.post<SaleOrder>('/sale/orders', data),
  updateOrder: (id: string, data: Partial<SaleOrder>) =>
    request.put<SaleOrder>(`/sale/orders/${id}`, data),
  deleteOrder: (id: string) =>
    request.delete<void>(`/sale/orders/${id}`),
  auditOrder: (id: string) =>
    request.patch<SaleOrder>(`/sale/orders/${id}/audit`, {}),
  cancelOrder: (id: string) =>
    request.patch<SaleOrder>(`/sale/orders/${id}/cancel`, {}),

  // 销售出库
  listStockOuts: (params: SaleQuery) =>
    request.get<{ list: SaleStockOut[]; pagination: any }>('/sale/stock-outs', { params }),
  getStockOut: (id: string) =>
    request.get<SaleStockOut>(`/sale/stock-outs/${id}`),
  createStockOut: (data: Partial<SaleStockOut>) =>
    request.post<SaleStockOut>('/sale/stock-outs', data),
  auditStockOut: (id: string) =>
    request.patch<SaleStockOut>(`/sale/stock-outs/${id}/audit`, {}),

  // 销售退货
  listReturns: (params: SaleQuery) =>
    request.get<{ list: SaleReturn[]; pagination: any }>('/sale/returns', { params }),
  createReturn: (data: Partial<SaleReturn>) =>
    request.post<SaleReturn>('/sale/returns', data),
  auditReturn: (id: string) =>
    request.patch<SaleReturn>(`/sale/returns/${id}/audit`, {}),
};
```

src/api/supplier.ts

```
import request from './request';

export interface Supplier {
  id: string;
  code: string;
  name: string;
  contact?: string;
  phone?: string;
  mobile?: string;
  email?: string;
  address?: string;
  bankName?: string;
  bankAccount?: string;
  taxNo?: string;
  status: string;
}

export interface SupplierQuery {
  keyword?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}

export const supplierApi = {
  list: (params: SupplierQuery) =>
```

```
    request.get<{ list: Supplier[]; pagination: any }>('/suppliers', { params }),
  get: (id: string) =>
    request.get<Supplier>(`/suppliers/${id}`),
  create: (data: Partial<Supplier>) =>
    request.post<Supplier>('/suppliers', data),
  update: (id: string, data: Partial<Supplier>) =>
    request.put<Supplier>(`/suppliers/${id}`, data),
  delete: (id: string) =>
    request.delete<void>(`/suppliers/${id}`),
  updateStatus: (id: string, status: string) =>
    request.patch<void>(`/suppliers/${id}/status`, { status }),
};
```

src/api/warehouse.ts

```
import request from './request';

export interface Warehouse {
  id: string;
  code: string;
  name: string;
  address?: string;
  principal?: string;
  sortOrder: number;
  status: string;
  createdAt: string;
}

export interface WarehouseQuery {
  keyword?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}

export const warehouseApi = {
  list: (params: WarehouseQuery) =>
    request.get<{ list: Warehouse[]; pagination: any }>('/warehouses', { params }),
  get: (id: string) =>
    request.get<Warehouse>(`/warehouses/${id}`),
  create: (data: Partial<Warehouse>) =>
    request.post<Warehouse>('/warehouses', data),
  update: (id: string, data: Partial<Warehouse>) =>
    request.put<Warehouse>(`/warehouses/${id}`, data),
  delete: (id: string) =>
    request.delete<void>(`/warehouses/${id}`),
  updateStatus: (id: string, status: string) =>
    request.patch<void>(`/warehouses/${id}/status`, { status }),
};
```

src/components/FormPage.tsx

```
import { ReactNode } from 'react';
import { Form, Card, Button, Space, Spin } from 'antd';
import { useNavigate } from 'react-router-dom';

interface FormItem {
  name: string;
  label: string;
  required?: boolean;
  component: ReactNode;
}

interface FormPageProps {
  title: string;
  items: FormItem[];
  form: any;
  loading?: boolean;
  initialValues?: any;
  onSubmit?: (values: any) => void;
  onCancel?: () => void;
  width?: number;
}

const FormPage = ({
  title,
  items,
  form,
  loading,
  initialValues,
  onSubmit,
```

```
  onCancel,
  width = 600,
}: FormPageProps) => {
  const navigate = useNavigate();

  const handleSubmit = () => {
    form.validateFields().then((values: any) => {
      onSubmit?.(values);
    });
  };

  const handleCancel = () => {
    if (onCancel) {
      onCancel();
    } else {
      navigate(-1);
    }
  };

  return (
    <Card title={title}>
      <Spin spinning={loading}>
        <Form
          form={form}
          layout="vertical"
          initialValues={initialValues}
          style={{ maxWidth: width }}
        >
          {items.map((item) => (
            <Form.Item
              key={item.name}
              name={item.name}
              label={item.label}
              required={item.required}
            >
              {item.component}
            </Form.Item>
          ))}

          <Form.Item style={{ marginTop: 24 }}>
            <Space>
              <Button type="primary" onClick={handleSubmit} loading={loading}>
                保存
              </Button>
              <Button onClick={handleCancel}>取消</Button>
            </Space>
          </Form.Item>
        </Form>
      </Spin>
    </Card>
  );
};

export default FormPage;
```

src/components/ListPage.tsx

```
import { ReactNode } from 'react';
import { Table, Button, Space, Card } from 'antd';
import { PlusOutlined } from '@ant-design/icons';

interface Column {
  title: string;
  dataIndex: string;
  key: string;
  width?: number;
  render?: (value: any, record: any) => ReactNode;
}

interface ListPageProps {
  title: string;
  columns: Column[];
  dataSource: any[];
  loading?: boolean;
  pagination?: any;
  onTableChange?: (pagination: any, filters: any, sorter: any) => void;
  onAdd?: () => void;
  onEdit?: (record: any) => void;
  onDelete?: (record: any) => void;
  onStatusChange?: (record: any, status: string) => void;
  renderActions?: (record: any) => ReactNode;
  showAdd?: boolean;
}

const ListPage = ({
  title,
```

```
  columns,
  dataSource,
  loading,
  pagination,
  onTableChange,
  onAdd,
  onEdit,
  onDelete,
  onStatusChange,
  renderActions,
  showAdd = true,
}: ListPageProps) => {
  const actionColumn = {
    title: '操作',
    key: 'action',
    width: 150,
    render: (_: any, record: any) => (
      <Space size="small">
        <Button type="link" size="small" onClick={() => onEdit?.(record)}>
          编辑
        </Button>
        {onStatusChange && (
          <Button
            type="link"
            size="small"
            onClick={() => onStatusChange(record, record.status === 'active' ? 'inactive' : 'active'
)}
          >
            {record.status === 'active' ? '禁用' : '启用'}
          </Button>
        )}
        {onDelete && (
          <Button type="link" size="small" danger onClick={() => onDelete(record)}>
            删除
          </Button>
        )}
        {renderActions?.(record)}
      </Space>
    ),
  };

  return (
    <Card
      title={title}
      extra={
        showAdd && (
          <Button type="primary" icon={<PlusOutlined />} onClick={onAdd}>
            新增
          </Button>
        )
      }
    >
      <Table
        columns={[...columns, actionColumn]}
        dataSource={dataSource}
        rowKey="id"
        loading={loading}
        pagination={pagination}
        onChange={onTableChange}
      />
    </Card>
  );
};

export default ListPage;
```

src/layouts/MainLayout.tsx

```
import { useState } from 'react';
import { Outlet, useNavigate, useLocation } from 'react-router-dom';
import { Layout, Menu, Avatar, Dropdown, Badge } from 'antd';
import type { MenuProps } from 'antd';
import {
  DashboardOutlined,
  ShoppingCartOutlined,
  ShopOutlined,
  InboxOutlined,
  MoneyCollectOutlined,
  FileTextOutlined,
  BarChartOutlined,
  SettingOutlined,
  LogoutOutlined,
```

```
  UserOutlined,
  BellOutlined,
  BankOutlined,
  TeamOutlined,
  AppstoreOutlined,
} from '@ant-design/icons';

const { Header, Sider, Content } = Layout;

const MainLayout = () => {
  const [collapsed, setCollapsed] = useState(false);
  const navigate = useNavigate();
  const location = useLocation();

  const selectedKey = '/' + location.pathname.split('/').filter(Boolean).join('/');

  const menuItems: MenuProps['items'] = [
    { key: '/dashboard', icon: <DashboardOutlined />, label: '经营看板' },
    {
      key: 'purchase',
      icon: <ShoppingCartOutlined />,
      label: '采购管理',
      children: [
        { key: '/purchase/orders', label: '采购订单' },
        { key: '/purchase/stock-ins', label: '采购入库' },
        { key: '/purchase/returns', label: '采购退货' },
      ],
    },
    {
      key: 'sale',
      icon: <ShopOutlined />,
      label: '销售管理',
      children: [
        { key: '/sale/orders', label: '销售订单' },
        { key: '/sale/stock-outs', label: '销售出库' },
        { key: '/sale/returns', label: '销售退货' },
      ],
    },
    {
      key: 'inventory',
      icon: <InboxOutlined />,
      label: '库存管理',
      children: [
        { key: '/inventory/list', label: '库存查询' },
        { key: '/inventory/check', label: '库存盘点' },
        { key: '/inventory/transfer', label: '库存调拨' },
        { key: '/inventory/warning', label: '库存预警' },
      ],
    },
    {
      key: 'finance',
      icon: <MoneyCollectOutlined />,
      label: '财务管理',
      children: [
        { key: '/finance/vouchers', label: '凭证管理' },
        { key: '/finance/payments', label: '收付款' },
        { key: '/finance/receivable', label: '应收应付' },
      ],
    },
    {
      key: 'reports',
      icon: <BarChartOutlined />,
      label: '报表中心',
      children: [
        { key: '/reports/sale', label: '销售报表' },
        { key: '/reports/purchase', label: '采购报表' },
        { key: '/reports/inventory', label: '库存报表' },
      ],
    },
    {
      key: 'settings-group',
      icon: <SettingOutlined />,
      label: '系统设置',
      children: [
        {
          key: '/settings/basic',
          icon: <AppstoreOutlined />,
          label: '基础资料',
          children: [
            { key: '/settings/basic/warehouse', label: '仓库管理' },
            { key: '/settings/basic/category', label: '商品分类' },
```

```
        { key: '/settings/basic/product', label: '商品管理' },
        { key: '/settings/basic/customer', label: '客户管理' },
        { key: '/settings/basic/supplier', label: '供应商管理' },
        { key: '/settings/basic/account', label: '账户管理' },
      ],
    },
    { key: '/settings/users', icon: <TeamOutlined />, label: '用户管理' },
    { key: '/settings/roles', icon: <BankOutlined />, label: '角色权限' },
  ],
 },
];

const userMenuItems: MenuProps['items'] = [
 { key: 'profile', icon: <UserOutlined />, label: '个人中心' },
 { type: 'divider' },
 { key: 'logout', icon: <LogoutOutlined />, label: '退出登录' },
];

const handleMenuClick: MenuProps['onClick'] = ({ key }) => {
 if (key.startsWith('/')) {
   navigate(key);
 }
};

return (
 <Layout style={{ minHeight: '100vh' }}>
   <Sider
    collapsible
    collapsed={collapsed}
    onCollapse={setCollapsed}
    theme="light"
    style={{ borderRight: '1px solid #f0f0f0' }}
   >
    <div
     style={{
      height: 64,
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center',
      borderBottom: '1px solid #f0f0f0',
     }}
    >
     {collapsed ? (
      <span style={{ fontSize: 18, fontWeight: 600 }}>ERP</span>
     ) : (
      <span style={{ fontSize: 16, fontWeight: 600, whiteSpace: 'nowrap' }}>
       云ERP - 零售批发版
      </span>
     )}
    </div>
    <Menu
     mode="inline"
     selectedKeys={[selectedKey]}
     items={menuItems}
     onClick={handleMenuClick}
     style={{ borderRight: 0, marginTop: 8 }}
    />
   </Sider>

   <Layout>
    <Header
     style={{
      background: '#fff',
      padding: '0 24px',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'flex-end',
      gap: 24,
      borderBottom: '1px solid #f0f0f0',
     }}
    >
     <Badge count={3}>
      <BellOutlined style={{ fontSize: 18, cursor: 'pointer' }} />
     </Badge>

     <Dropdown menu={{ items: userMenuItems }} placement="bottomRight">
      <div style={{ cursor: 'pointer', display: 'flex', alignItems: 'center', gap: 8 }}>
       <Avatar style={{ backgroundColor: '#1677ff' }}>
        <UserOutlined />
       </Avatar>
       <span>管理员</span>
      </div>
     </Dropdown>
    </Header>
```

```
      <Content style={{ margin: 0, padding: 0, background: '#f0f2f5' }}>
        <div style={{ padding: 24 }}>
          <Outlet />
        </div>
      </Content>
    </Layout>
  </Layout>
 );
};

export default MainLayout;
```

src/main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ConfigProvider } from 'antd';
import zhCN from 'antd/locale/zh_CN';
import App from './App';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});

const theme = {
  token: {
    colorPrimary: '#1677ff',
    borderRadius: 4,
    fontFamily: '"PingFang SC", "Microsoft YaHei", sans-serif',
  },
};

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <ConfigProvider locale={zhCN} theme={theme}>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </ConfigProvider>
    </QueryClientProvider>
  </React.StrictMode>
);
```

src/pages/Dashboard.tsx

```
import { Card, Row, Col, Statistic, Table, Tag, Space } from 'antd';
import { ArrowUpOutlined, ArrowDownOutlined } from '@ant-design/icons';

const Dashboard = () => {
  const pendingTasks = [
    { type: 'sale_return_pending', count: 2, desc: '销售退货待审核' },
    { type: 'purchase_in_pending', count: 1, desc: '采购入库待审核' },
    { type: 'voucher_pending', count: 3, desc: '凭证待审核' },
  ];

  const topProducts = [
    { name: '商品A', quantity: 120 },
    { name: '商品B', quantity: 98 },
    { name: '商品C', quantity: 76 },
    { name: '商品D', quantity: 65 },
    { name: '商品E', quantity: 54 },
  ];

  const lowStockItems = [
    { name: '商品X', status: '库存不足' },
    { name: '商品Y', status: '超过保质期' },
    { name: '商品Z', status: '超储' },
  ];

  return (
    <div>
      <Row gutter={16} style={{ marginBottom: 24 }}>
        <Col span={6}>
          <Card>
            <Statistic
              title="今日销售"
              value={12580.00}
              precision={2}
```

```
          prefix="¥"
          valueStyle={{ color: '#3f8600' }}
          suffix={<span style={{ fontSize: 14, color: '#3f8600' }}><ArrowUpOutlined /> 12.5%</sp
an>}
        />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic
          title="本月销售"
          value={186320.00}
          precision={2}
          prefix="¥"
          valueStyle={{ color: '#3f8600' }}
          suffix={<span style={{ fontSize: 14, color: '#3f8600' }}><ArrowUpOutlined /> 8.3%</spa
n>}
        />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic title="库存商品数" value={1256} suffix="种" />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic
          title="应收余额"
          value={45230.00}
          precision={2}
          prefix="¥"
          valueStyle={{ color: '#cf1322' }}
          suffix={<span style={{ fontSize: 14, color: '#cf1322' }}><ArrowDownOutlined /> 3.2%</s
pan>}
        />
      </Card>
    </Col>
  </Row>

  <Row gutter={16}>
    <Col span={12}>
      <Card title="热销商品 TOP 5">
        <Table
          dataSource={topProducts.map((p, i) => ({ ...p, key: i }))}
          columns={[
            { title: '排名', key: 'rank', render: (_: any, __: any, i: number) => i + 1, width: 60
},
            { title: '商品名称', dataIndex: 'name', key: 'name' },
            { title: '销量', dataIndex: 'quantity', key: 'quantity' },
          ]}
          pagination={false}
        />
      </Card>
    </Col>
    <Col span={12}>
      <Card title="待处理事项">
        <Space direction="vertical" style={{ width: '100%' }}>
          {pendingTasks.map((task) => (
            <div key={task.type} style={{ display: 'flex', justifyContent: 'space-between', padd
ing: '8px 0', border...[截断]
              <span>{task.desc}</span>
              <Tag color="orange">{task.count}</Tag>
            </div>
          ))}
        </Space>
      </Card>
    </Col>
  </Row>

  <Row gutter={16} style={{ marginTop: 16 }}>
    <Col span={24}>
      <Card title="库存预警">
        <Space>
          {lowStockItems.map((item, i) => (
            <Tag key={i} color="red">{item.name} - {item.status}</Tag>
          ))}
        </Space>
      </Card>
    </Col>
  </Row>
</div>
```

```
  );
};
export default Dashboard;
```

src/pages/Login.tsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Form, Input, Button, Card, message } from 'antd';
import { UserOutlined, LockOutlined } from '@ant-design/icons';
import { authApi } from '@/api/auth';
import { useAuthStore } from '@/stores/auth';

const Login = () => {
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
  const { setAuth } = useAuthStore();

  const onFinish = async (values: { username: string; password: string }) => {
    setLoading(true);
    try {
      const result = await authApi.login(values);
      setAuth(result.token, result.refreshToken, result.user, result.tenant);
      message.success('登录成功');
      navigate('/');
    } catch {
      // 错误已在拦截器处理
    } finally {
      setLoading(false);
    }
  };

  return (
    <div
      style={{
        minHeight: '100vh',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        background: 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',
      }}
    >
      <Card style={{ width: 400, boxShadow: '0 14px 40px rgba(0,0,0,0.15)' }}>
        <div style={{ textAlign: 'center', marginBottom: 32 }}>
          <h1 style={{ fontSize: 24, fontWeight: 600, marginBottom: 8 }}>
            云ERP系统
          </h1>
          <p style={{ color: '#666' }}>零售批发版</p>
        </div>

        <Form
          name="login"
          onFinish={onFinish}
          autoComplete="off"
          size="large"
        >
          <Form.Item
            name="username"
            rules={[{ required: true, message: '请输入用户名' }]}
          >
            <Input
              prefix={<UserOutlined />}
              placeholder="用户名: admin"
            />
          </Form.Item>

          <Form.Item
            name="password"
            rules={[{ required: true, message: '请输入密码' }]}
          >
            <Input.Password
              prefix={<LockOutlined />}
              placeholder="密码: admin123"
            />
          </Form.Item>

          <Form.Item>
            <Button
              type="primary"
              htmlType="submit"
              block
              loading={loading}
              style={{ marginTop: 8 }}
            >
              登录
```

```
          </Button>
        </Form.Item>
      </Form>
    </Card>
  </div>
 );
};

export default Login;
```

src/pages/finance/PaymentList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { financeApi, Payment } from '@/api/finance';
import { customerApi } from '@/api/customer';
import { supplierApi } from '@/api/supplier';
import { accountApi } from '@/api/account';
import { useAuthStore } from '@/stores/auth';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
};

const typeMap: Record<string, { color: string; label: string }> = {
  receipt: { color: 'green', label: '收款' },
  payment: { color: 'red', label: '付款' },
  other: { color: 'blue', label: '其他' },
};

const PaymentList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Payment[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [customers, setCustomers] = useState<any[]>([]);
  const [suppliers, setSuppliers] = useState<any[]>([]);
  const [accounts, setAccounts] = useState<any[]>([]);
  const [query, setQuery] = useState<any>({});
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchCustomers = async () => {
    const res = await customerApi.list({ status: 'active', pageSize: 100 } as any);
    setCustomers(res.list);
  };

  const fetchSuppliers = async () => {
    const res = await supplierApi.list({ status: 'active', pageSize: 100 } as any);
    setSuppliers(res.list);
  };

  const fetchAccounts = async () => {
    const res = await accountApi.listSimple();
    setAccounts(res);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await financeApi.listPayments({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    fetchCustomers();
    fetchSuppliers();
    fetchAccounts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await financeApi.auditPayment(id);
    message.success('审核成功');
    fetchData();
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
```

```
    setLoading(true);
    try {
      const data = {
        ...values,
        paymentDate: values.paymentDate.format('YYYY-MM-DD'),
      };
      await financeApi.createPayment(data as any);
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const columns = [
    { title: '单据编号', dataIndex: 'paymentNo', key: 'paymentNo', width: 150 },
    { title: '日期', dataIndex: 'paymentDate', key: 'paymentDate', width: 100, render: (d: string) =>
dayjs(d).format('YYY...[截断]
    { title: '类型', dataIndex: 'paymentType', key: 'paymentType', width: 80, render: (t: string) => <
Tag {...typeMap[t]}>...[截断]
    { title: '金额', dataIndex: 'amount', key: 'amount', width: 120, render: (v: number) => `¥${v?.toF
ixed(2)}` },
    { title: '账户', dataIndex: ['account', 'name'], key: 'account', width: 120 },
    { title: '对方', key: 'counterparty', width: 120, render: (_: any, r: Payment) => r.customer?.name
 || r.supplier?.name...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 100,
      render: (_: any, r: Payment) => (
        <Space size="small">
          {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
        </Space>
      ),
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
          <Select placeholder="类型" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, paymentType: ...[截断]
            {Object.entries(typeMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</Sel
ect.Option>)}
          </Select>
          <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
            {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
          </Select>
        </Space>
        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增收付款
</Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>

      <Modal title="新增收付款" open={modalVisible} onCancel={() => setModalVisible(false)}
        onOk={handleSubmit} confirmLoading={loading}>
        <Form form={form} layout="vertical">
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="paymentType" label="类型" rules={[{ required: true }]} style={{ width: 10
0 }}>
              <Select>
                <Select.Option value="receipt">收款</Select.Option>
                <Select.Option value="payment">付款</Select.Option>
                <Select.Option value="other">其他</Select.Option>
              </Select>
            </Form.Item>
            <Form.Item name="paymentDate" label="日期" rules={[{ required: true }]} style={{ width: 15
0 }}>
              <DatePicker />
            </Form.Item>
            <Form.Item name="amount" label="金额" rules={[{ required: true }]} style={{ width: 150 }}>
              <Input type="number" prefix="¥" />
            </Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="accountId" label="收款账户" rules={[{ required: true }]} style={{ width: 20
```

```
0 }}>
          <Select>
            {accounts.map(a => <Select.Option key={a.id} value={a.id}>{a.name}</Select.Option>)}
          </Select>
        </Form.Item>
        <Form.Item name="customerId" label="客户" style={{ width: 200 }}>
          <Select allowClear>
            {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)
}
          </Select>
        </Form.Item>
        <Form.Item name="supplierId" label="供应商" style={{ width: 200 }}>
          <Select allowClear>
            {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)
}
          </Select>
        </Form.Item>
      </Space>
    </Form>
  </Modal>
 </>
 );
};

export default PaymentList;
```

src/pages/finance/VoucherList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, Table as AntTable
} from 'antd';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { financeApi, Voucher, VoucherItem } from '@/api/finance';
import { useAuthStore } from '@/stores/auth';
import { message } from 'antd';

const statusMap: Record<string, { color: string; label: string }> = {
  draft: { color: 'default', label: '草稿' },
  audited: { color: 'blue', label: '已审核' },
  posted: { color: 'green', label: '已过账' },
};

const VoucherList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Voucher[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [subjects, setSubjects] = useState<any[]>([]);
  const [query, setQuery] = useState<any>({});
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchSubjects = async () => {
    await financeApi.initSubjects();
    const res = await financeApi.listSubjects();
    setSubjects(res);
  };

  const flattenSubjects = (subs: any[], prefix = ''): any[] => {
    let result: any[] = [];
    for (const s of subs) {
      result.push({ ...s, label: prefix + s.code + ' ' + s.name });
      if (s.children?.length) {
        result = result.concat(flattenSubjects(s.children, prefix + '  '));
      }
    }
    return result;
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await financeApi.listVouchers({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    setItems([{ accountId: '', direction: 'debit', amount: 0, summary: '' }]);
```

```
    fetchSubjects();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await financeApi.auditVoucher(id);
    message.success('审核成功');
    fetchData();
  };

  const handlePost = async (id: string) => {
    await financeApi.postVoucher(id);
    message.success('过账成功');
    fetchData();
  };

  const handleDelete = async (id: string) => {
    await financeApi.deleteVoucher(id);
    message.success('删除成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { accountId: '', direction: 'debit', amount: 0, summary: '' }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        voucherDate: values.voucherDate.format('YYYY-MM-DD'),
        items,
      };
      await financeApi.createVoucher(data as any);
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const flatSubjects = flattenSubjects(subjects);

  const columns = [
    { title: '凭证号', dataIndex: 'voucherNo', key: 'voucherNo', width: 120 },
    { title: '日期', dataIndex: 'voucherDate', key: 'voucherDate', width: 100, render: (d: string) =>
dayjs(d).format('YYY...[截断]
    { title: '凭证字', dataIndex: 'voucherType', key: 'voucherType', width: 60 },
    { title: '借方金额', dataIndex: 'totalDebit', key: 'totalDebit', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
    { title: '贷方金额', dataIndex: 'totalCredit', key: 'totalCredit', width: 120, render: (v: number) =
> `¥${v?.toFixed(2)}...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 150,
      render: (_: any, r: Voucher) => (
        <Space size="small">
          {r.status === 'draft' && <>
            <Button type="link" size="small" onClick={() => handleAudit(r.id)}>审核</Button>
            <Button type="link" size="small" danger onClick={() => handleDelete(r.id)}>删除</Button>
          </>}
          {r.status === 'audited' && <Button type="link" size="small" onClick={() => handlePost(r.id
)}>过账</Button>}
        </Space>
      ),
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索凭证号/摘要" style={{ width: 200 }} onSearch={(v) => setQuery({ ..
.query, keyword: v }...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
```

```
          {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增凭证<
/Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>

    <Modal title="新增凭证" open={modalVisible} onCancel={() => setModalVisible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={700}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="voucherType" label="凭证字" initialValue="记" style={{ width: 80 }}>
            <Select options={[{ label: '记', value: '记' }, { label: '收', value: '收' }, { label: '付'
, value: '付' }]} />
          </Form.Item>
          <Form.Item name="voucherDate" label="凭证日期" rules={[{ required: true }]} style={{ width:
150 }}>
            <DatePicker />
          </Form.Item>
        </Space>

        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加分录</Button>
        </div>
        <AntTable
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '科目', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.accountId} onChange={(v) => updat
eItem(i, 'accountId'...[截断]
                {flatSubjects.map(s => <Select.Option key={s.id} value={s.id}>{s.label}</Select.Op
tion>)}
              </Select>
            )},
            { title: '方向', width: 80, render: (_: any, __: any, i: number) => (
              <Select value={items[i]?.direction} onChange={(v) => updateItem(i, 'direction', v)}>
                <Select.Option value="debit">借</Select.Option>
                <Select.Option value="credit">贷</Select.Option>
              </Select>
            )},
            { title: '金额', width: 120, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.amount} onChange={(e) => updateItem(i, 'amount
', e.target.value)} ...[截断]
            )},
            { title: '摘要', render: (_: any, __: any, i: number) => (
              <Input value={items[i]?.summary} onChange={(e) => updateItem(i, 'summary', e.target.
value)} />
            )},
            { title: '', width: 50, render: (_: any, __: any, i: number) => (
              <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
            )},
          ]}
          pagination={false}
          size="small"
        />
      </Form>
    </Modal>
  </>
);
};
export default VoucherList;
```

src/pages/finance/index.ts

```
export { default as VoucherList } from './VoucherList';
export { default as PaymentList } from './PaymentList';
```

src/pages/index.ts

```
export { default as Login } from './Login';
export { default as Dashboard } from './Dashboard';
```

src/pages/inventory/InventoryList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Space, Input, Select, Tag } from 'antd';
import { warehouseApi } from '@/api/warehouse';
```

```
import { inventoryApi, Inventory } from '@/api/inventory';
import { useAuthStore } from '@/stores/auth';

const InventoryList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Inventory[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [query, setQuery] = useState<any>({});
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await inventoryApi.list({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchWarehouses(); }, []);
  useEffect(() => { fetchData(); }, [query, tenantId]);

  const getStockStatus = (record: Inventory) => {
    if (record.quantity === 0) return { color: 'red', label: '缺货' };
    if (record.quantity < 100) return { color: 'orange', label: '低库存' };
    if (record.quantity > 1000) return { color: 'blue', label: '超储' };
    return { color: 'green', label: '正常' };
  };

  const columns = [
    { title: '商品编码', dataIndex: ['product', 'code'], key: 'code', width: 120 },
    { title: '商品名称', dataIndex: ['product', 'name'], key: 'name' },
    { title: '规格', dataIndex: ['product', 'spec'], key: 'spec', width: 100 },
    { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
    { title: '库存数量', dataIndex: 'quantity', key: 'quantity', width: 100 },
    { title: '冻结数量', dataIndex: 'frozenQuantity', key: 'frozenQuantity', width: 80 },
    { title: '成本价', dataIndex: 'avgCostPrice', key: 'avgCostPrice', width: 100, render: (v: number)
=> `¥${v?.toFixed(2)...[截断]
    { title: '库存金额', key: 'amount', width: 120, render: (_: any, r: Inventory) => `¥${((r.quantity |
| 0) * (r.avgCostPri...[截断]
    {
      title: '状态',
      key: 'status',
      width: 80,
      render: (_: any, r: Inventory) => {
        const status = getStockStatus(r);
        return <Tag color={status.color}>{status.label}</Tag>;
      },
    },
  ];

  return (
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索商品编码/名称" style={{ width: 200 }} onSearch={(v) => setQuery({ ...
query, keyword: v })...[截断]
        <Select placeholder="仓库" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ...qu
ery, warehouseId: v ...[截断]
          {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>)}
        </Select>
        <Select placeholder="库存状态" allowClear style={{ width: 120 }} onChange={(v) => setQuery({ ...
query, stockStatus: ...[截断]
          <Select.Option value="normal">正常</Select.Option>
          <Select.Option value="low">低库存</Select.Option>
          <Select.Option value="over">超储</Select.Option>
          <Select.Option value="zero">缺货</Select.Option>
        </Select>
      </Space>
      <Table columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: pagination.
total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>
  );
};

export default InventoryList;
```

src/pages/inventory/InventoryWarning.tsx

```
import { useState, useEffect } from 'react';
import { Card, Tag, Space } from 'antd';
import { inventoryApi } from '@/api/inventory';
import { useAuthStore } from '@/stores/auth';

const InventoryWarning = () => {
  const [warnings, setWarnings] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  useEffect(() => {
    if (tenantId) {
      inventoryApi.getWarnings().then(setWarnings);
    }
  }, [tenantId]);

  return (
    <Card title="库存预警">
      <Space direction="vertical" style={{ width: '100%' }}>
        {warnings.length === 0 && <span>暂无预警</span>}
        {warnings.map((w, i) => (
          <div key={i} style={{ display: 'flex', justifyContent: 'space-between', padding: '8px 0',
borderBottom: '1px s...[截断]
            <span>{w.product?.name} - {w.message}</span>
            <Tag color={w.type === 'low' ? 'red' : 'orange'}>{w.type === 'low' ? '库存不足' : '超储'}</Tag
>
          </div>
        ))}
      </Space>
    </Card>
  );
};

export default InventoryWarning;
```

src/pages/inventory/StockCheckList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, Table as AntTable
} from 'antd';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { inventoryApi, StockCheck } from '@/api/inventory';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';
import { message } from 'antd';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'blue', label: '已审核' },
  completed: { color: 'green', label: '已完成' },
};

const StockCheckList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<StockCheck[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const [query, setQuery] = useState<any>({});
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await inventoryApi.listChecks({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
```

```
    form.resetFields();
    setItems([]);
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await inventoryApi.auditCheck(id);
    message.success('审核成功');
    fetchData();
  };

  const handleApprove = async (id: string) => {
    await inventoryApi.approveCheck(id);
    message.success('确认成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', bookQuantity: 0, checkQuantity: 0, diffQuantity: 0, costPri
ce: 0, diffAmount: 0...[截断]
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'bookQuantity' || field === 'checkQuantity') {
      newItems[index].diffQuantity = (newItems[index].checkQuantity || 0) - (newItems[index].bookQua
ntity || 0);
      newItems[index].diffAmount = newItems[index].diffQuantity * (newItems[index].costPrice || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        checkDate: values.checkDate.format('YYYY-MM-DD'),
        items,
      };
      await inventoryApi.createCheck(data as any);
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const columns = [
    { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
    { title: '盘点日期', dataIndex: 'checkDate', key: 'checkDate', width: 100, render: (d: string) => da
yjs(d).format('YYYY-...[截断]
    { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
    { title: '差异金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =
> `¥${v?.toFixed(2)}...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 150,
      render: (_: any, r: StockCheck) => (
        <Space size="small">
          {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
          {r.status === 'approved' && <Button type="link" size="small" onClick={() => handleApprove(
r.id)}>确认</Button>}
        </Space>
      ),
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
            {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
```

```
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增盘点单
</Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>
    <Modal title="新增盘点单" open={modalVisible} onCancel={() => setModalVisible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={900}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 20
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
          </Form.Item>
          <Form.Item name="checkDate" label="盘点日期" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <DatePicker />
          </Form.Item>
        </Space>
        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <AntTable
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
              </Select>
            )},
            { title: '账面数量', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.bookQuantity} onChange={(e) => updateItem(i, '
bookQuantity', e.tar...[截断]
            )},
            { title: '盘点数量', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.checkQuantity} onChange={(e) => updateItem(i,
'checkQuantity', e.t...[截断]
            )},
            { title: '差异数量', render: (_: any, __: any, i: number) => <span style={{ color: (items[
i]?.diffQuantity || ...[截断]
            { title: '差异原因', width: 150, render: (_: any, __: any, i: number) => (
              <Input value={items[i]?.reason} onChange={(e) => updateItem(i, 'reason', e.target.va
lue)} />
            )},
            { title: '', width: 50, render: (_: any, __: any, i: number) => (
              <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
            )},
          ]}
          pagination={false}
          size="small"
        />
      </Form>
    </Modal>
  </>
 );
};
export default StockCheckList;
```

src/pages/inventory/StockTransferList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, Table as AntTable
} from 'antd';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { inventoryApi, StockTransfer } from '@/api/inventory';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';
import { message } from 'antd';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
```

```
  approved: { color: 'green', label: '已审核' },
};

const StockTransferList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<StockTransfer[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const [query, setQuery] = useState<any>({});
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await inventoryApi.listTransfers({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    setItems([]);
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await inventoryApi.auditTransfer(id);
    message.success('审核成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', quantity: 1, costPrice: 0, amount: 0 }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'costPrice') {
      newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].costPrice || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        transferDate: values.transferDate.format('YYYY-MM-DD'),
        items,
      };
      await inventoryApi.createTransfer(data as any);
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const columns = [
    { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
    { title: '调拨日期', dataIndex: 'transferDate', key: 'transferDate', width: 100, render: (d: string)
=> dayjs(d).format(...[截断]
    { title: '调出仓库', dataIndex: ['fromWarehouse', 'name'], key: 'fromWarehouse', width: 100 },
    { title: '调入仓库', dataIndex: ['toWarehouse', 'name'], key: 'toWarehouse', width: 100 },
```

```
    { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 100,
      render: (_: any, r: StockTransfer) => (
        <Space size="small">
          {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
        </Space>
      ),
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
            {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
          </Select>
        </Space>
        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增调拨单
</Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>

      <Modal title="新增调拨单" open={modalVisible} onCancel={() => setModalVisible(false)}
        onOk={handleSubmit} confirmLoading={loading} width={800}>
        <Form form={form} layout="vertical">
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="fromWarehouseId" label="调出仓库" rules={[{ required: true }]} style={{ wid
th: 150 }}>
              <Select>
                {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
              </Select>
            </Form.Item>
            <Form.Item name="toWarehouseId" label="调入仓库" rules={[{ required: true }]} style={{ width
: 150 }}>
              <Select>
                {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
              </Select>
            </Form.Item>
            <Form.Item name="transferDate" label="调拨日期" rules={[{ required: true }]} style={{ width:
 150 }}>
              <DatePicker />
            </Form.Item>
          </Space>

          <div style={{ marginBottom: 8 }}>
            <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
          </div>
          <AntTable
            dataSource={items.map((item, i) => ({ ...item, key: i }))}
            columns={[
              { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
                <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                  {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
                </Select>
              )},
              { title: '数量', width: 100, render: (_: any, __: any, i: number) => (
                <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
              )},
              { title: '成本价', width: 100, render: (_: any, __: any, i: number) => (
                <Input type="number" value={items[i]?.costPrice} onChange={(e) => updateItem(i, 'cos
tPrice', e.target.va...[截断]
              )},
              { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) ||
0).toFixed(2)}</span>...[截断]
              { title: '', width: 50, render: (_: any, __: any, i: number) => (
```

```
            <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
          )},
        ]}
        pagination={false}
        size="small"
      />
    </Form>
  </Modal>
 </>
);
};
export default StockTransferList;
```

src/pages/inventory/index.ts

```
export { default as InventoryList } from './InventoryList';
export { default as StockCheckList } from './StockCheckList';
export { default as StockTransferList } from './StockTransferList';
export { default as InventoryWarning } from './InventoryWarning';
```

src/pages/purchase/PurchaseOrderList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message, Popconfir
m } from 'antd';
import { PlusOutlined, CheckOutlined, CloseOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { purchaseApi, PurchaseOrder } from '@/api/purchase';
import { supplierApi } from '@/api/supplier';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';

const { confirm } = Modal;

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
  completed: { color: 'blue', label: '已完成' },
  cancelled: { color: 'gray', label: '已取消' },
};

const PurchaseOrderList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<PurchaseOrder[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState<PurchaseQuery>({});
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [suppliers, setSuppliers] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchSuppliers = async () => {
    const res = await supplierApi.list({ status: 'active', pageSize: 100 } as any);
    setSuppliers(res.list);
  };

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await purchaseApi.listOrders({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    setEditingId(null);
    form.resetFields();
    setItems([]);
```

```
    fetchSuppliers();
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleEdit = async (record: PurchaseOrder) => {
    setEditingId(record.id);
    fetchSuppliers();
    fetchWarehouses();
    fetchProducts();
    const res = await purchaseApi.getOrder(record.id);
    form.setFieldsValue({
      ...res,
      orderDate: dayjs(res.orderDate),
      deliveryDate: res.deliveryDate ? dayjs(res.deliveryDate) : null,
    });
    setItems(res.items || []);
    setModalVisible(true);
  };

  const handleDelete = async (id: string) => {
    await purchaseApi.deleteOrder(id);
    message.success('删除成功');
    fetchData();
  };

  const handleAudit = async (id: string) => {
    await purchaseApi.auditOrder(id);
    message.success('审核成功');
    fetchData();
  };

  const handleCancel = async (id: string) => {
    await purchaseApi.cancelOrder(id);
    message.success('取消成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'price') {
      newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        orderDate: values.orderDate.format('YYYY-MM-DD'),
        deliveryDate: values.deliveryDate?.format('YYYY-MM-DD'),
        items,
      };
      if (editingId) {
        await purchaseApi.updateOrder(editingId, data);
      } else {
        await purchaseApi.createOrder(data as any);
      }
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

  const columns = [
    { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
    { title: '日期', dataIndex: 'orderDate', key: 'orderDate', width: 100, render: (d: string) => dayj
s(d).format('YYYY-MM...[截断]
    { title: '供应商', dataIndex: ['supplier', 'name'], key: 'supplier', width: 150 },
    { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
    { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
```

```
atusMap[s]}>{statusM...[截断]
  {
    title: '操作', key: 'action', width: 180,
    render: (_: any, r: PurchaseOrder) => (
      <Space size="small">
        {r.status === 'pending' && <>
          <Button type="link" size="small" onClick={() => handleEdit(r)}>编辑</Button>
          <Button type="link" size="small" onClick={() => handleAudit(r.id)}>审核</Button>
          <Button type="link" size="small" danger onClick={() => handleCancel(r.id)}>取消</Button>
        </>}
        {r.status === 'pending' && <Button type="link" size="small" danger onClick={() => handleDe
lete(r.id)}>删除</Butt...[截断]
      </Space>
    ),
  },
];

return (
  <>
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
        <Select placeholder="供应商" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ..
.query, supplierId: ...[截断]
          {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)}
        </Select>
        <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
          {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增采购订
单</Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>

    <Modal title={editingId ? '编辑采购订单' : '新增采购订单'} open={modalVisible} onCancel={() => setModalVis
ible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={900}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="supplierId" label="供应商" rules={[{ required: true }]} style={{ width: 20
0 }}>
            <Select>
              {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)
}
            </Select>
          </Form.Item>
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
          </Form.Item>
          <Form.Item name="orderDate" label="订单日期" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <DatePicker />
          </Form.Item>
          <Form.Item name="deliveryDate" label="交货日期" style={{ width: 150 }}>
            <DatePicker />
          </Form.Item>
        </Space>

        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <Table
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
              </Select>
            )},
            { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
```

```
            <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
            </Select>
          )},
          { title: '数量', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
          )},
          { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
          )},
          { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) || 
0).toFixed(2)}</span>...[截断]
          { title: '', width: 50, render: (_: any, __: any, i: number) => (
            <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
          )},
        ]}
        pagination={false}
        size="small"
      />
      <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
        合计: <span style={{ color: '#1677ff', fontWeight: 600 }}>¥{totalAmount.toFixed(2)}</span>
      </div>
    </Form>
  </Modal>
 </>
);
};

export default PurchaseOrderList;
```

src/pages/purchase/PurchaseReturnList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { purchaseApi, PurchaseReturn } from '@/api/purchase';
import { supplierApi } from '@/api/supplier';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
};

const PurchaseReturnList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<PurchaseReturn[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState<any>({});
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [suppliers, setSuppliers] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchSuppliers = async () => {
    const res = await supplierApi.list({ status: 'active', pageSize: 100 } as any);
    setSuppliers(res.list);
  };

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await purchaseApi.listReturns({ ...query, tenantId } as any);
      setDataSource(res.list);
```

```
    setPagination(res.pagination);
  } finally { setLoading(false); }
};

useEffect(() => { fetchData(); }, [query, tenantId]);

const handleAdd = () => {
  form.resetFields();
  setItems([]);
  fetchSuppliers();
  fetchWarehouses();
  fetchProducts();
  setModalVisible(true);
};

const handleAudit = async (id: string) => {
  await purchaseApi.auditReturn(id);
  message.success('审核成功');
  fetchData();
};

const addItem = () => {
  setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
};

const removeItem = (index: number) => {
  setItems(items.filter((_, i) => i !== index));
};

const updateItem = (index: number, field: string, value: any) => {
  const newItems = [...items];
  newItems[index] = { ...newItems[index], [field]: value };
  if (field === 'quantity' || field === 'price') {
    newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
  }
  setItems(newItems);
};

const handleSubmit = async () => {
  const values = await form.validateFields();
  setLoading(true);
  try {
    const data = {
      ...values,
      returnDate: values.returnDate.format('YYYY-MM-DD'),
      items,
    };
    await purchaseApi.createReturn(data as any);
    setModalVisible(false);
    fetchData();
  } finally { setLoading(false); }
};

const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

const columns = [
  { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
  { title: '退货日期', dataIndex: 'stockInDate', key: 'stockInDate', width: 100, render: (d: string) =
> dayjs(d).format('Y...[截断]
  { title: '供应商', dataIndex: ['supplier', 'name'], key: 'supplier', width: 150 },
  { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
  { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${Math.abs(v)?.toF...[截断]
  { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
  {
    title: '操作', key: 'action', width: 100,
    render: (_: any, r: PurchaseReturn) => (
      <Space size="small">
        {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
      </Space>
    ),
  },
];

return (
  <>
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
        <Select placeholder="供应商" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ..
.query, supplierId: ...[截断]
          {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)}
        </Select>
        <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
```

```
query, status: v })}...[截断]
        {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
      </Select>
    </Space>
    <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增采购退
货</Button>}
      columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
      pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
        onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
  </Card>

  <Modal title="新增采购退货" open={modalVisible} onCancel={() => setModalVisible(false)}
    onOk={handleSubmit} confirmLoading={loading} width={900}>
    <Form form={form} layout="vertical">
      <Space style={{ display: 'flex' }} align="start" size="large">
        <Form.Item name="supplierId" label="供应商" rules={[{ required: true }]} style={{ width: 20
0 }}>
          <Select>
            {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)
}
          </Select>
        </Form.Item>
        <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
          <Select>
            {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
          </Select>
        </Form.Item>
        <Form.Item name="returnDate" label="退货日期" rules={[{ required: true }]} style={{ width: 1
50 }}>
          <DatePicker />
        </Form.Item>
      </Space>

      <div style={{ marginBottom: 8 }}>
        <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
      </div>
      <Table
        dataSource={items.map((item, i) => ({ ...item, key: i }))}
        columns={[
          { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
            <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
              {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
            </Select>
          )},
          { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
            <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
            </Select>
          )},
          { title: '退货数量', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
          )},
          { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
          )},
          { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) || 
0).toFixed(2)}</span...[截断]
          { title: '', width: 50, render: (_: any, __: any, i: number) => (
            <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
          )},
        ]}
        pagination={false}
        size="small"
      />
      <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
        合计: <span style={{ color: '#cf1322', fontWeight: 600 }}>-¥{totalAmount.toFixed(2)}</span
>
      </div>
    </Form>
  </Modal>
 </>
);
```

```
};
export default PurchaseReturnList;
```

src/pages/purchase/PurchaseStockInList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { purchaseApi, PurchaseStockIn } from '@/api/purchase';
import { supplierApi } from '@/api/supplier';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
};

const PurchaseStockInList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<PurchaseStockIn[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState<any>({});
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [suppliers, setSuppliers] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchSuppliers = async () => {
    const res = await supplierApi.list({ status: 'active', pageSize: 100 } as any);
    setSuppliers(res.list);
  };

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await purchaseApi.listStockIns({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    setItems([]);
    fetchSuppliers();
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await purchaseApi.auditStockIn(id);
    message.success('审核成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'price') {
```

```
    newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
  }
  setItems(newItems);
};

const handleSubmit = async () => {
  const values = await form.validateFields();
  setLoading(true);
  try {
    const data = {
      ...values,
      stockInDate: values.stockInDate.format('YYYY-MM-DD'),
      items,
    };
    await purchaseApi.createStockIn(data as any);
    setModalVisible(false);
    fetchData();
  } finally { setLoading(false); }
};

const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

const columns = [
  { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
  { title: '日期', dataIndex: 'stockInDate', key: 'stockInDate', width: 100, render: (d: string) =>
dayjs(d).format('YYY...[截断]
  { title: '供应商', dataIndex: ['supplier', 'name'], key: 'supplier', width: 150 },
  { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
  { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
  { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
  {
    title: '操作', key: 'action', width: 100,
    render: (_: any, r: PurchaseStockIn) => (
      <Space size="small">
        {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
      </Space>
    ),
  },
];

return (
  <>
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
        <Select placeholder="供应商" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ..
.query, supplierId: ...[截断]
          {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)}
        </Select>
        <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
          {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增采购入
库</Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>

    <Modal title="新增采购入库" open={modalVisible} onCancel={() => setModalVisible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={900}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="supplierId" label="供应商" rules={[{ required: true }]} style={{ width: 20
0 }}>
            <Select>
              {suppliers.map(s => <Select.Option key={s.id} value={s.id}>{s.name}</Select.Option>)
}
            </Select>
          </Form.Item>
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
```

```
          </Form.Item>
          <Form.Item name="stockInDate" label="入库日期" rules={[{ required: true }]} style={{ width:
150 }}>
            <DatePicker />
          </Form.Item>
        </Space>
        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <Table
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
              </Select>
            )},
            { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
                {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
              </Select>
            )},
            { title: '数量', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
            )},
            { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
            )},
            { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) || |
0).toFixed(2)}</span>...[截断]
            { title: '', width: 50, render: (_: any, __: any, i: number) => (
              <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
            )},
          ]}
          pagination={false}
          size="small"
        />
        <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
          合计: <span style={{ color: '#1677ff', fontWeight: 600 }}>¥{totalAmount.toFixed(2)}</span>
        </div>
      </Form>
    </Modal>
  </>
  );
};
export default PurchaseStockInList;
```

src/pages/purchase/index.ts

```
export { default as PurchaseOrderList } from './PurchaseOrderList';
export { default as PurchaseStockInList } from './PurchaseStockInList';
export { default as PurchaseReturnList } from './PurchaseReturnList';
```

src/pages/sale/SaleOrderList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { saleApi, SaleOrder } from '@/api/sale';
import { customerApi } from '@/api/customer';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';
const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
  completed: { color: 'blue', label: '已完成' },
  cancelled: { color: 'gray', label: '已取消' },
};
const SaleOrderList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<SaleOrder[]>([]);
```

```
const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
const [query, setQuery] = useState<any>({});
const [modalVisible, setModalVisible] = useState(false);
const [editingId, setEditingId] = useState<string | null>(null);
const [form] = Form.useForm();
const [items, setItems] = useState<any[]>([]);
const [customers, setCustomers] = useState<any[]>([]);
const [warehouses, setWarehouses] = useState<any[]>([]);
const [products, setProducts] = useState<any[]>([]);
const tenantId = useAuthStore((s) => s.tenant?.id || '');

const fetchCustomers = async () => {
  const res = await customerApi.list({ status: 'active', pageSize: 100 } as any);
  setCustomers(res.list);
};

const fetchWarehouses = async () => {
  const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
  setWarehouses(res.list);
};

const fetchProducts = async () => {
  const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
  setProducts(res.list);
};

const fetchData = async () => {
  if (!tenantId) return;
  setLoading(true);
  try {
    const res = await saleApi.listOrders({ ...query, tenantId } as any);
    setDataSource(res.list);
    setPagination(res.pagination);
  } finally { setLoading(false); }
};

useEffect(() => { fetchData(); }, [query, tenantId]);

const handleAdd = () => {
  setEditingId(null);
  form.resetFields();
  setItems([]);
  fetchCustomers();
  fetchWarehouses();
  fetchProducts();
  setModalVisible(true);
};

const handleEdit = async (record: SaleOrder) => {
  setEditingId(record.id);
  fetchCustomers();
  fetchWarehouses();
  fetchProducts();
  const res = await saleApi.getOrder(record.id);
  form.setFieldsValue({
    ...res,
    orderDate: dayjs(res.orderDate),
    deliveryDate: res.deliveryDate ? dayjs(res.deliveryDate) : null,
  });
  setItems(res.items || []);
  setModalVisible(true);
};

const handleDelete = async (id: string) => {
  await saleApi.deleteOrder(id);
  message.success('删除成功');
  fetchData();
};

const handleAudit = async (id: string) => {
  await saleApi.auditOrder(id);
  message.success('审核成功');
  fetchData();
};

const handleCancel = async (id: string) => {
  await saleApi.cancelOrder(id);
  message.success('取消成功');
  fetchData();
};

const addItem = () => {
  setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
};

const removeItem = (index: number) => {
  setItems(items.filter((_, i) => i !== index));
};
```

```
  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'price') {
      newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        orderDate: values.orderDate.format('YYYY-MM-DD'),
        deliveryDate: values.deliveryDate?.format('YYYY-MM-DD'),
        items,
      };
      if (editingId) {
        await saleApi.updateOrder(editingId, data);
      } else {
        await saleApi.createOrder(data as any);
      }
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

  const columns = [
    { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
    { title: '日期', dataIndex: 'orderDate', key: 'orderDate', width: 100, render: (d: string) => dayj
s(d).format('YYYY-MM...[截断]
    { title: '客户', dataIndex: ['customer', 'name'], key: 'customer', width: 150 },
    { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
    { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 180,
      render: (_: any, r: SaleOrder) => (
        <Space size="small">
          {r.status === 'pending' && <>
            <Button type="link" size="small" onClick={() => handleEdit(r)}>编辑</Button>
            <Button type="link" size="small" onClick={() => handleAudit(r.id)}>审核</Button>
            <Button type="link" size="small" danger onClick={() => handleCancel(r.id)}>取消</Button>
          </>}
          {r.status === 'pending' && <Button type="link" size="small" danger onClick={() => handleDe
lete(r.id)}>删除</Butt...[截断]
        </Space>
      ),
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
          <Select placeholder="客户" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ...
query, customerId: v...[截断]
            {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)}
          </Select>
          <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
            {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
          </Select>
        </Space>
        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增销售订
单</Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>

      <Modal title={editingId ? '编辑销售订单' : '新增销售订单'} open={modalVisible} onCancel={() => setModalVis
ible(false)}
        onOk={handleSubmit} confirmLoading={loading} width={900}>
```

```
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="customerId" label="客户" rules={[{ required: true }]} style={{ width: 200
 }}>
            <Select>
              {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)
}
            </Select>
          </Form.Item>
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
          </Form.Item>
          <Form.Item name="orderDate" label="订单日期" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <DatePicker />
          </Form.Item>
          <Form.Item name="deliveryDate" label="交货日期" style={{ width: 150 }}>
            <DatePicker />
          </Form.Item>
        </Space>
        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <Table
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
              </Select>
            )},
            { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
                {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
              </Select>
            )},
            { title: '数量', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
            )},
            { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
            )},
            { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) || 
0).toFixed(2)}</span>...[截断]
            { title: '', width: 50, render: (_: any, __: any, i: number) => (
              <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
            )},
          ]}
          pagination={false}
          size="small"
        />
        <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
          合计: <span style={{ color: '#1677ff', fontWeight: 600 }}>¥{totalAmount.toFixed(2)}</span>
        </div>
      </Form>
    </Modal>
  </>
  );
};
export default SaleOrderList;
```

src/pages/sale/SaleReturnList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { saleApi, SaleReturn } from '@/api/sale';
import { customerApi } from '@/api/customer';
```

```
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
};

const SaleReturnList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<SaleReturn[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState<any>({});
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [customers, setCustomers] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchCustomers = async () => {
    const res = await customerApi.list({ status: 'active', pageSize: 100 } as any);
    setCustomers(res.list);
  };

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await saleApi.listReturns({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    setItems([]);
    fetchCustomers();
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await saleApi.auditReturn(id);
    message.success('审核成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'price') {
      newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        returnDate: values.returnDate.format('YYYY-MM-DD'),
        items,
```

```
    };
    await saleApi.createReturn(data as any);
    setModalVisible(false);
    fetchData();
  } finally { setLoading(false); }
};

const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

const columns = [
  { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
  { title: '退货日期', dataIndex: 'stockOutDate', key: 'stockOutDate', width: 100, render: (d: string)
=> dayjs(d).format(...[截断]
  { title: '客户', dataIndex: ['customer', 'name'], key: 'customer', width: 150 },
  { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
  { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${Math.abs(v)?.toF...[截断]
  { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
  {
    title: '操作', key: 'action', width: 100,
    render: (_: any, r: SaleReturn) => (
      <Space size="small">
        {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
      </Space>
    ),
  },
];

return (
  <>
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
        <Select placeholder="客户" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ...
query, customerId: v...[截断]
          {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)}
        </Select>
        <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
          {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增销售退
货</Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>

    <Modal title="新增销售退货" open={modalVisible} onCancel={() => setModalVisible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={900}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="customerId" label="客户" rules={[{ required: true }]} style={{ width: 200
}}>
            <Select>
              {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)
}
            </Select>
          </Form.Item>
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
          </Form.Item>
          <Form.Item name="returnDate" label="退货日期" rules={[{ required: true }]} style={{ width: 1
50 }}>
            <DatePicker />
          </Form.Item>
        </Space>

        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <Table
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
```

```
          { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
            <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
              {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
            </Select>
          )},
          { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
            <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
            </Select>
          )},
          { title: '退货数量', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
          )},
          { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
            <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
          )},
          { title: '金额', render: (_: any, __: any, i: number) => <span>¥{((items[i]?.amount) || 
0).toFixed(2)}</span...[截断]
          { title: '', width: 50, render: (_: any, __: any, i: number) => (
            <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
          )},
        ]}
        pagination={false}
        size="small"
      />
      <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
        合计: <span style={{ color: '#cf1322', fontWeight: 600 }}>-¥{totalAmount.toFixed(2)}</span
>
      </div>
    </Form>
  </Modal>
 </>
);
};
export default SaleReturnList;
```

src/pages/sale/SaleStockOutList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, DatePicker, message } from 'an
td';
import { PlusOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { saleApi, SaleStockOut } from '@/api/sale';
import { customerApi } from '@/api/customer';
import { warehouseApi } from '@/api/warehouse';
import { productApi } from '@/api/product';
import { useAuthStore } from '@/stores/auth';

const statusMap: Record<string, { color: string; label: string }> = {
  pending: { color: 'orange', label: '待审核' },
  approved: { color: 'green', label: '已审核' },
};

const SaleStockOutList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<SaleStockOut[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState<any>({});
  const [modalVisible, setModalVisible] = useState(false);
  const [form] = Form.useForm();
  const [items, setItems] = useState<any[]>([]);
  const [customers, setCustomers] = useState<any[]>([]);
  const [warehouses, setWarehouses] = useState<any[]>([]);
  const [products, setProducts] = useState<any[]>([]);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchCustomers = async () => {
    const res = await customerApi.list({ status: 'active', pageSize: 100 } as any);
    setCustomers(res.list);
  };

  const fetchWarehouses = async () => {
    const res = await warehouseApi.list({ status: 'active', pageSize: 100 } as any);
    setWarehouses(res.list);
  };

  const fetchProducts = async () => {
```

```
    const res = await productApi.list({ status: 'active', pageSize: 100 } as any);
    setProducts(res.list);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await saleApi.listStockOuts({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    form.resetFields();
    setItems([]);
    fetchCustomers();
    fetchWarehouses();
    fetchProducts();
    setModalVisible(true);
  };

  const handleAudit = async (id: string) => {
    await saleApi.auditStockOut(id);
    message.success('审核成功');
    fetchData();
  };

  const addItem = () => {
    setItems([...items, { productId: '', warehouseId: '', quantity: 1, price: 0, amount: 0 }]);
  };

  const removeItem = (index: number) => {
    setItems(items.filter((_, i) => i !== index));
  };

  const updateItem = (index: number, field: string, value: any) => {
    const newItems = [...items];
    newItems[index] = { ...newItems[index], [field]: value };
    if (field === 'quantity' || field === 'price') {
      newItems[index].amount = (newItems[index].quantity || 0) * (newItems[index].price || 0);
    }
    setItems(newItems);
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      const data = {
        ...values,
        stockOutDate: values.stockOutDate.format('YYYY-MM-DD'),
        items,
      };
      await saleApi.createStockOut(data as any);
      setModalVisible(false);
      fetchData();
    } finally { setLoading(false); }
  };

  const totalAmount = items.reduce((sum, item) => sum + (item.amount || 0), 0);

  const columns = [
    { title: '单据编号', dataIndex: 'orderNo', key: 'orderNo', width: 150 },
    { title: '日期', dataIndex: 'stockOutDate', key: 'stockOutDate', width: 100, render: (d: string) =
> dayjs(d).format('Y...[截断]
    { title: '客户', dataIndex: ['customer', 'name'], key: 'customer', width: 150 },
    { title: '仓库', dataIndex: ['warehouse', 'name'], key: 'warehouse', width: 100 },
    { title: '金额', dataIndex: 'totalAmount', key: 'totalAmount', width: 120, render: (v: number) =>
`¥${v?.toFixed(2)}` ...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag {...st
atusMap[s]}>{statusM...[截断]
    {
      title: '操作', key: 'action', width: 100,
      render: (_: any, r: SaleStockOut) => (
        <Space size="small">
          {r.status === 'pending' && <Button type="link" size="small" onClick={() => handleAudit(r.i
d)}>审核</Button>}
        </Space>
      ),
    },
  ];

  return (
    <>
```

```
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索单据编号" style={{ width: 200 }} onSearch={(v) => setQuery({ ...q
uery, keyword: v })}...[截断]
        <Select placeholder="客户" allowClear style={{ width: 150 }} onChange={(v) => setQuery({ ...
query, customerId: v...[截断]
          {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)}
        </Select>
        <Select placeholder="状态" allowClear style={{ width: 100 }} onChange={(v) => setQuery({ ...
query, status: v })}...[截断]
          {Object.entries(statusMap).map(([k, v]) => <Select.Option key={k} value={k}>{v.label}</S
elect.Option>)}
        </Select>
      </Space>
      <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增销售出
库</Button>}
        columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
        pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
          onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
    </Card>

    <Modal title="新增销售出库" open={modalVisible} onCancel={() => setModalVisible(false)}
      onOk={handleSubmit} confirmLoading={loading} width={900}>
      <Form form={form} layout="vertical">
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="customerId" label="客户" rules={[{ required: true }]} style={{ width: 200
 }}>
            <Select>
              {customers.map(c => <Select.Option key={c.id} value={c.id}>{c.name}</Select.Option>)
}
            </Select>
          </Form.Item>
          <Form.Item name="warehouseId" label="仓库" rules={[{ required: true }]} style={{ width: 15
0 }}>
            <Select>
              {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Option>
)}
            </Select>
          </Form.Item>
          <Form.Item name="stockOutDate" label="出库日期" rules={[{ required: true }]} style={{ width:
 150 }}>
            <DatePicker />
          </Form.Item>
        </Space>

        <div style={{ marginBottom: 8 }}>
          <Button type="dashed" onClick={addItem}>+ 添加商品</Button>
        </div>
        <Table
          dataSource={items.map((item, i) => ({ ...item, key: i }))}
          columns={[
            { title: '商品', width: 200, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.productId} onChange={(v) => updat
eItem(i, 'productId'...[截断]
                {products.map(p => <Select.Option key={p.id} value={p.id}>{p.name}</Select.Option>
)}
              </Select>
            )},
            { title: '仓库', width: 120, render: (_: any, __: any, i: number) => (
              <Select style={{ width: '100%' }} value={items[i]?.warehouseId} onChange={(v) => upd
ateItem(i, 'warehous...[截断]
                {warehouses.map(w => <Select.Option key={w.id} value={w.id}>{w.name}</Select.Optio
n>)}
              </Select>
            )},
            { title: '数量', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.quantity} onChange={(e) => updateItem(i, 'quan
tity', e.target.valu...[截断]
            )},
            { title: '单价', width: 100, render: (_: any, __: any, i: number) => (
              <Input type="number" value={items[i]?.price} onChange={(e) => updateItem(i, 'price',
 e.target.value)} />
            )},
            { title: '金额', render: (_: any, __: any, i: number) => <span>¥{(items[i]?.amount) ||
0).toFixed(2)}</span>...[截断]
            { title: '', width: 50, render: (_: any, __: any, i: number) => (
              <Button type="link" danger size="small" onClick={() => removeItem(i)}>删除</Button>
            )},
          ]}
          pagination={false}
          size="small"
```

```
        />
        <div style={{ textAlign: 'right', marginTop: 8, fontSize: 16 }}>
          合计: <span style={{ color: '#1677ff', fontWeight: 600 }}>¥{totalAmount.toFixed(2)}</span>
        </div>
      </Form>
    </Modal>
  </>
 );
};

export default SaleStockOutList;
```

src/pages/sale/index.ts

```
export { default as SaleOrderList } from './SaleOrderList';
export { default as SaleStockOutList } from './SaleStockOutList';
export { default as SaleReturnList } from './SaleReturnList';
```

src/pages/settings/AccountList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, InputNumber, message } from 'a
ntd';
import { PlusOutlined } from '@ant-design/icons';
import { accountApi, Account } from '@/api/account';
import { useAuthStore } from '@/stores/auth';

const AccountList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Account[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState({ keyword: '', type: '', status: '' });
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await accountApi.list({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => { setEditingId(null); form.resetFields(); setModalVisible(true); };
  const handleEdit = (r: Account) => { setEditingId(r.id); form.setFieldsValue(r); setModalVisible(t
rue); };
  const handleDelete = async (id: string) => {
    Modal.confirm({ title: '确认删除', content: '确定删除该账户吗？', onOk: async () => { await accountApi.delete
(id); fetchData(); }...[截断]
  };
  const handleStatusChange = async (r: Account) => {
    await accountApi.updateStatus(r.id, r.status === 'active' ? 'inactive' : 'active');
    fetchData();
  };
  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      if (editingId) await accountApi.update(editingId, values);
      else await accountApi.create({ ...values, tenantId } as any);
      setModalVisible(false); fetchData();
    } finally { setLoading(false); }
  };

  const typeMap: Record<string, string> = { cash: '现金', bank: '银行', other: '其他' };
  const columns = [
    { title: '账户编码', dataIndex: 'code', key: 'code', width: 120 },
    { title: '账户名称', dataIndex: 'name', key: 'name' },
    { title: '类型', dataIndex: 'type', key: 'type', width: 80, render: (t: string) => typeMap[t] || t
 },
    { title: '银行', dataIndex: 'bankName', key: 'bankName', width: 120 },
    { title: '账号', dataIndex: 'bankAccount', key: 'bankAccount', width: 180 },
    { title: '期初余额', dataIndex: 'initialBalance', key: 'initialBalance', width: 120, render: (v: num
ber) => `¥${v?.toFix...[截断]
    { title: '当前余额', dataIndex: 'currentBalance', key: 'currentBalance', width: 120, render: (v: num
ber) => `¥${v?.toFix...[截断]
    { title: '默认', dataIndex: 'isDefault', key: 'isDefault', width: 60, render: (v: boolean) => v ?
<Tag color="blue">是<...[截断]
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag color=
```

```
{s === 'active' ? 'g...[截断]
    { title: '操作', key: 'action', width: 150, render: (_: any, r: Account) => (
      <Space size="small">
        <Button type="link" size="small" onClick={() => handleEdit(r)}>编辑</Button>
        <Button type="link" size="small" onClick={() => handleStatusChange(r)}>{r.status === 'active
' ? '禁用' : '启用'}</Bu...[截断]
        <Button type="link" size="small" danger onClick={() => handleDelete(r.id)}>删除</Button>
      </Space>
    )},
  ];
  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索编码/名称" style={{ width: 200 }} onSearch={(v) => setQuery({ ...
query, keyword: v })...[截断]
          <Select placeholder="类型" allowClear style={{ width: 100 }} value={query.type || undefined}
            onChange={(v) => setQuery({ ...query, type: v || '' })}
            options={[{ label: '现金', value: 'cash' }, { label: '银行', value: 'bank' }, { label: '其他',
 value: 'other' }]} ...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} value={query.status || undefine
d}
            onChange={(v) => setQuery({ ...query, status: v || '' })}
            options={[{ label: '启用', value: 'active' }, { label: '禁用', value: 'inactive' }]} />
        </Space>
        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增账户<
/Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>
      <Modal title={editingId ? '编辑账户' : '新增账户'} open={modalVisible} onCancel={() => setModalVisible
(false)}
        onOk={handleSubmit} confirmLoading={loading}>
        <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="code" label="账户编码" rules={[{ required: true }]} style={{ width: 150 }}>
<Input /></Form.Item...[截断]
            <Form.Item name="name" label="账户名称" rules={[{ required: true }]} style={{ flex: 1 }}><In
put /></Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="type" label="账户类型" style={{ width: 150 }}>
              <Select options={[{ label: '现金', value: 'cash' }, { label: '银行', value: 'bank' }, { la
bel: '其他', value: 'o...[截断]
            </Form.Item>
            <Form.Item name="bankName" label="开户银行" style={{ width: 200 }}><Input /></Form.Item>
            <Form.Item name="bankAccount" label="银行账号" style={{ width: 200 }}><Input /></Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="initialBalance" label="期初余额" style={{ width: 150 }}>
              <InputNumber min={0} precision={2} prefix="¥" />
            </Form.Item>
            <Form.Item name="isDefault" label="默认账户" valuePropName="checked" style={{ width: 100 }}>
              <input type="checkbox" />
            </Form.Item>
          </Space>
        </Form>
      </Modal>
    </>
  );
};
export default AccountList;
```

src/pages/settings/CategoryManage.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Modal, Form, Input, InputNumber, Tree } from 'antd';
import { PlusOutlined, EditOutlined, DeleteOutlined } from '@ant-design/icons';
import { productCategoryApi, ProductCategory } from '@/api/product';
import { useAuthStore } from '@/stores/auth';
const CategoryManage = () => {
  const [dataSource, setDataSource] = useState<ProductCategory[]>([]);
  const [loading, setLoading] = useState(false);
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const tenantId = useAuthStore((s) => s.tenant?.id || '');
```

```
  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await productCategoryApi.list();
      setDataSource(res);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, [tenantId]);

  const handleAdd = () => {
    setEditingId(null);
    form.resetFields();
    setModalVisible(true);
  };

  const handleEdit = (record: ProductCategory) => {
    setEditingId(record.id);
    form.setFieldsValue(record);
    setModalVisible(true);
  };

  const handleDelete = async (id: string) => {
    Modal.confirm({
      title: '确认删除',
      content: '确定删除该分类吗？',
      onOk: async () => {
        await productCategoryApi.delete(id);
        fetchData();
      },
    });
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      if (editingId) {
        await productCategoryApi.update(editingId, values);
      } else {
        await productCategoryApi.create({ ...values, tenantId } as any);
      }
      setModalVisible(false);
      fetchData();
    } finally {
      setLoading(false);
    }
  };

  const columns = [
    { title: '分类名称', dataIndex: 'name', key: 'name' },
    { title: '排序', dataIndex: 'sortOrder', key: 'sortOrder', width: 100 },
    {
      title: '操作',
      key: 'action',
      width: 150,
      render: (_: any, record: ProductCategory) => (
        <Space size="small">
          <Button type="link" size="small" onClick={() => handleEdit(record)}>编辑</Button>
          <Button type="link" size="small" danger onClick={() => handleDelete(record.id)}>删除</Button
>
        </Space>
      ),
    },
  ];

  return (
    <Card
      title="商品分类管理"
      extra={<Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增分类</Button>}
    >
      <Table
        columns={columns}
        dataSource={dataSource}
        rowKey="id"
        loading={loading}
        pagination={false}
      />

      <Modal
        title={editingId ? '编辑分类' : '新增分类'}
```

```
      open={modalVisible}
      onCancel={() => setModalVisible(false)}
      onOk={handleSubmit}
      confirmLoading={loading}
    >
      <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
        <Form.Item name="parentId" label="上级分类">
          <Tree treeData={dataSource} placeholder="不选则为顶级分类" allowClear />
        </Form.Item>
        <Form.Item name="name" label="分类名称" rules={[{ required: true }]}>
          <Input />
        </Form.Item>
        <Form.Item name="sortOrder" label="排序" initialValue={0}>
          <InputNumber min={0} />
        </Form.Item>
      </Form>
    </Modal>
  </Card>
  );
};

export default CategoryManage;
```

src/pages/settings/CustomerList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form, InputNumber } from 'antd';
import { PlusOutlined } from '@ant-design/icons';
import { customerApi, Customer } from '@/api/customer';
import { useAuthStore } from '@/stores/auth';

const CustomerList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Customer[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState({ keyword: '', status: '' });
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await customerApi.list({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => {
    setEditingId(null);
    form.resetFields();
    setModalVisible(true);
  };

  const handleEdit = (record: Customer) => {
    setEditingId(record.id);
    form.setFieldsValue(record);
    setModalVisible(true);
  };

  const handleDelete = async (id: string) => {
    Modal.confirm({ title: '确认删除', content: '确定删除该客户吗？', onOk: async () => {
      await customerApi.delete(id);
      fetchData();
    }});
  };

  const handleStatusChange = async (record: Customer) => {
    await customerApi.updateStatus(record.id, record.status === 'active' ? 'inactive' : 'active');
    fetchData();
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      if (editingId) await customerApi.update(editingId, values);
      else await customerApi.create({ ...values, tenantId } as any);
      setModalVisible(false);
      fetchData();
```

```
    } finally { setLoading(false); }
  };

  const columns = [
    { title: '客户编码', dataIndex: 'code', key: 'code', width: 120 },
    { title: '客户名称', dataIndex: 'name', key: 'name' },
    { title: '联系人', dataIndex: 'contact', key: 'contact', width: 100 },
    { title: '电话', dataIndex: 'phone', key: 'phone', width: 120 },
    { title: '手机', dataIndex: 'mobile', key: 'mobile', width: 120 },
    { title: '类型', dataIndex: 'customerType', key: 'customerType', width: 80 },
    {
      title: '状态', dataIndex: 'status', key: 'status', width: 80,
      render: (s: string) => <Tag color={s === 'active' ? 'green' : 'red'}>{s === 'active' ? '启用' :
'禁用'}</Tag>
    },
    {
      title: '操作', key: 'action', width: 150,
      render: (_: any, r: Customer) => (
        <Space size="small">
          <Button type="link" size="small" onClick={() => handleEdit(r)}>编辑</Button>
          <Button type="link" size="small" onClick={() => handleStatusChange(r)}>{r.status === 'acti
ve' ? '禁用' : '启用'}</...[截断]
          <Button type="link" size="small" danger onClick={() => handleDelete(r.id)}>删除</Button>
        </Space>
      )
    },
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索编码/名称/联系人" style={{ width: 200 }} onSearch={(v) => setQuery({
 ...query, keyword: ...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} value={query.status || undefine
d}
            onChange={(v) => setQuery({ ...query, status: v || '' })}
            options={[{ label: '启用', value: 'active' }, { label: '禁用', value: 'inactive' }]} />
        </Space>

        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增客户<
/Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>

      <Modal title={editingId ? '编辑客户' : '新增客户'} open={modalVisible} onCancel={() => setModalVisible
(false)}
        onOk={handleSubmit} confirmLoading={loading} width={700}>
        <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="code" label="客户编码" rules={[{ required: true }]} style={{ width: 150 }}>
              <Input />
            </Form.Item>
            <Form.Item name="name" label="客户名称" rules={[{ required: true }]} style={{ flex: 1 }}>
              <Input />
            </Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="contact" label="联系人" style={{ width: 150 }}>
              <Input />
            </Form.Item>
            <Form.Item name="phone" label="电话" style={{ width: 150 }}>
              <Input />
            </Form.Item>
            <Form.Item name="mobile" label="手机" style={{ width: 150 }}>
              <Input />
            </Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="customerType" label="客户类型" style={{ width: 150 }}>
              <Select options={[{ label: '零售', value: 'retail' }, { label: '批发', value: 'wholesale'
}]} />
            </Form.Item>
            <Form.Item name="creditLimit" label="信用额度" style={{ width: 150 }}>
              <InputNumber min={0} precision={2} prefix="¥" />
            </Form.Item>
          </Space>
          <Form.Item name="address" label="地址">
            <Input />
          </Form.Item>
        </Form>
```

```
    </Modal>
   </>
  );
};

export default CustomerList;
```

src/pages/settings/ProductList.tsx

```
import { useState, useEffect } from 'react';
import { Input, Select, Button, Space, Tag, Table, Card, Modal, Form, InputNumber, TreeSelect, messa
ge } from 'antd';
import { PlusOutlined, EditOutlined, DeleteOutlined } from '@ant-design/icons';
import { productApi, productCategoryApi, Product, ProductCategory } from '@/api/product';
import { warehouseApi } from '@/api/warehouse';
import { useAuthStore } from '@/stores/auth';

const ProductList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Product[]>([]);
  const [categories, setCategories] = useState<ProductCategory[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState({ keyword: '', categoryId: '', status: '' });
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchCategories = async () => {
    const res = await productCategoryApi.list();
    setCategories(res);
  };

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
    try {
      const res = await productApi.list({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchCategories();
  }, []);

  useEffect(() => {
    fetchData();
  }, [query, tenantId]);

  const handleAdd = () => {
    setEditingId(null);
    form.resetFields();
    setModalVisible(true);
  };

  const handleEdit = (record: Product) => {
    setEditingId(record.id);
    form.setFieldsValue(record);
    setModalVisible(true);
  };

  const handleDelete = async (id: string) => {
    Modal.confirm({
      title: '确认删除',
      content: '确定删除该商品吗？',
      onOk: async () => {
        await productApi.delete(id);
        message.success('删除成功');
        fetchData();
      },
    });
  };

  const handleStatusChange = async (record: Product) => {
    const newStatus = record.status === 'active' ? 'inactive' : 'active';
    await productApi.updateStatus(record.id, newStatus);
    fetchData();
  };

  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      if (editingId) {
```

```
      await productApi.update(editingId, values);
    } else {
      await productApi.create({ ...values, tenantId } as any);
    }
    setModalVisible(false);
    fetchData();
  } finally {
    setLoading(false);
  }
};

const columns = [
  { title: '商品编码', dataIndex: 'code', key: 'code', width: 120 },
  { title: '商品名称', dataIndex: 'name', key: 'name' },
  { title: '规格', dataIndex: 'spec', key: 'spec', width: 100 },
  { title: '单位', dataIndex: 'unit', key: 'unit', width: 60 },
  { title: '分类', dataIndex: ['category', 'name'], key: 'category', width: 100 },
  { title: '采购价', dataIndex: 'purchasePrice', key: 'purchasePrice', width: 100 },
  { title: '零售价', dataIndex: 'salePrice', key: 'salePrice', width: 100 },
  {
    title: '状态',
    dataIndex: 'status',
    key: 'status',
    width: 80,
    render: (status: string) => (
      <Tag color={status === 'active' ? 'green' : 'red'}>
        {status === 'active' ? '启用' : '停用'}
      </Tag>
    ),
  },
  {
    title: '操作',
    key: 'action',
    width: 150,
    render: (_: any, record: Product) => (
      <Space size="small">
        <Button type="link" size="small" onClick={() => handleEdit(record)}>编辑</Button>
        <Button type="link" size="small" onClick={() => handleStatusChange(record)}>
          {record.status === 'active' ? '禁用' : '启用'}
        </Button>
        <Button type="link" size="small" danger onClick={() => handleDelete(record.id)}>删除</Button
>
      </Space>
    ),
  },
];

return (
  <>
    <Card>
      <Space style={{ marginBottom: 16 }} wrap>
        <Input.Search placeholder="搜索编码/名称/规格" style={{ width: 200 }} onSearch={(v) => setQuery({
...query, keyword: v...[截断]
        <TreeSelect
          placeholder="选择分类"
          allowClear
          treeData={categories}
          onChange={(v) => setQuery({ ...query, categoryId: v })}
          style={{ width: 200 }}
        />
        <Select
          placeholder="状态"
          allowClear
          style={{ width: 100 }}
          value={query.status || undefined}
          onChange={(v) => setQuery({ ...query, status: v || '' })}
          options={[
            { label: '启用', value: 'active' },
            { label: '停用', value: 'inactive' },
          ]}
        />
      </Space>

      <Table
        title={() => (
          <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增商品</Button>
        )}
        columns={columns}
        dataSource={dataSource}
        rowKey="id"
        loading={loading}
        pagination={{
          current: pagination.current,
```

```
        pageSize: pagination.pageSize,
        total: pagination.total,
        onChange: (page, pageSize) => setQuery({ ...query, page, pageSize }),
      }}
    />
  </Card>

  <Modal
    title={editingId ? '编辑商品' : '新增商品'}
    open={modalVisible}
    onCancel={() => setModalVisible(false)}
    onOk={handleSubmit}
    confirmLoading={loading}
    width={700}
  >
    <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
      <Space style={{ display: 'flex' }} align="start" size="large">
        <Form.Item name="code" label="商品编码" rules={[{ required: true }]} style={{ width: 200 }}>
          <Input />
        </Form.Item>
        <Form.Item name="name" label="商品名称" rules={[{ required: true }]} style={{ flex: 1 }}>
          <Input />
        </Form.Item>
      </Space>

      <Space style={{ display: 'flex' }} align="start" size="large">
        <Form.Item name="categoryId" label="商品分类" style={{ width: 200 }}>
          <TreeSelect treeData={categories} placeholder="选择分类" allowClear />
        </Form.Item>
        <Form.Item name="spec" label="规格" style={{ width: 150 }}>
          <Input />
        </Form.Item>
        <Form.Item name="unit" label="单位" style={{ width: 80 }}>
          <Input />
        </Form.Item>
      </Space>

      <Space style={{ display: 'flex' }} align="start" size="large">
        <Form.Item name="purchasePrice" label="采购价" style={{ width: 120 }}>
          <InputNumber min={0} precision={2} prefix="¥" />
        </Form.Item>
        <Form.Item name="salePrice" label="零售价" style={{ width: 120 }}>
          <InputNumber min={0} precision={2} prefix="¥" />
        </Form.Item>
        <Form.Item name="costPrice" label="成本价" style={{ width: 120 }}>
          <InputNumber min={0} precision={2} prefix="¥" />
        </Form.Item>
      </Space>

      <Space style={{ display: 'flex' }} align="start" size="large">
        <Form.Item name="minStock" label="最低库存预警" style={{ width: 120 }}>
          <InputNumber min={0} />
        </Form.Item>
        <Form.Item name="maxStock" label="最高库存预警" style={{ width: 120 }}>
          <InputNumber min={0} />
        </Form.Item>
      </Space>
    </Form>
  </Modal>
 </>
);
};

export default ProductList;
```

src/pages/settings/SupplierList.tsx

```
import { useState, useEffect } from 'react';
import { Table, Card, Button, Space, Input, Select, Tag, Modal, Form } from 'antd';
import { PlusOutlined } from '@ant-design/icons';
import { supplierApi, Supplier } from '@/api/supplier';
import { useAuthStore } from '@/stores/auth';

const SupplierList = () => {
  const [loading, setLoading] = useState(false);
  const [dataSource, setDataSource] = useState<Supplier[]>([]);
  const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
  const [query, setQuery] = useState({ keyword: '', status: '' });
  const [modalVisible, setModalVisible] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form] = Form.useForm();
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  const fetchData = async () => {
    if (!tenantId) return;
    setLoading(true);
```

```
    try {
      const res = await supplierApi.list({ ...query, tenantId } as any);
      setDataSource(res.list);
      setPagination(res.pagination);
    } finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, [query, tenantId]);

  const handleAdd = () => { setEditingId(null); form.resetFields(); setModalVisible(true); };
  const handleEdit = (r: Supplier) => { setEditingId(r.id); form.setFieldsValue(r); setModalVisible(
true); };
  const handleDelete = async (id: string) => {
    Modal.confirm({ title: '确认删除', content: '确定删除该供应商吗？', onOk: async () => { await supplierApi.dele
te(id); fetchData();...[截断]
  };
  const handleStatusChange = async (r: Supplier) => {
    await supplierApi.updateStatus(r.id, r.status === 'active' ? 'inactive' : 'active');
    fetchData();
  };
  const handleSubmit = async () => {
    const values = await form.validateFields();
    setLoading(true);
    try {
      if (editingId) await supplierApi.update(editingId, values);
      else await supplierApi.create({ ...values, tenantId } as any);
      setModalVisible(false); fetchData();
    } finally { setLoading(false); }
  };

  const columns = [
    { title: '供应商编码', dataIndex: 'code', key: 'code', width: 120 },
    { title: '供应商名称', dataIndex: 'name', key: 'name' },
    { title: '联系人', dataIndex: 'contact', key: 'contact', width: 100 },
    { title: '电话', dataIndex: 'phone', key: 'phone', width: 120 },
    { title: '手机', dataIndex: 'mobile', key: 'mobile', width: 120 },
    { title: '银行', dataIndex: 'bankName', key: 'bankName', width: 120 },
    { title: '银行账号', dataIndex: 'bankAccount', key: 'bankAccount', width: 150 },
    { title: '状态', dataIndex: 'status', key: 'status', width: 80, render: (s: string) => <Tag color=
{s === 'active' ? 'g...[截断]
    { title: '操作', key: 'action', width: 150, render: (_: any, r: Supplier) => (
      <Space size="small">
        <Button type="link" size="small" onClick={() => handleEdit(r)}>编辑</Button>
        <Button type="link" size="small" onClick={() => handleStatusChange(r)}>{r.status === 'active
' ? '禁用' : '启用'}</Bu...[截断]
        <Button type="link" size="small" danger onClick={() => handleDelete(r.id)}>删除</Button>
      </Space>
    )},
  ];

  return (
    <>
      <Card>
        <Space style={{ marginBottom: 16 }} wrap>
          <Input.Search placeholder="搜索编码/名称/联系人" style={{ width: 200 }} onSearch={(v) => setQuery({
 ...query, keyword: ...[截断]
          <Select placeholder="状态" allowClear style={{ width: 100 }} value={query.status || undefine
d}
            onChange={(v) => setQuery({ ...query, status: v || '' })}
            options={[{ label: '启用', value: 'active' }, { label: '禁用', value: 'inactive' }]} />
        </Space>
        <Table title={() => <Button type="primary" icon={<PlusOutlined />} onClick={handleAdd}>新增供应商
</Button>}
          columns={columns} dataSource={dataSource} rowKey="id" loading={loading}
          pagination={{ current: pagination.current, pageSize: pagination.pageSize, total: paginatio
n.total,
            onChange: (p, ps) => setQuery({ ...query, page: p, pageSize: ps }) }} />
      </Card>

      <Modal title={editingId ? '编辑供应商' : '新增供应商'} open={modalVisible} onCancel={() => setModalVisib
le(false)}
        onOk={handleSubmit} confirmLoading={loading} width={700}>
        <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="code" label="供应商编码" rules={[{ required: true }]} style={{ width: 150 }}
><Input /></Form.Ite...[截断]
            <Form.Item name="name" label="供应商名称" rules={[{ required: true }]} style={{ flex: 1 }}><I
nput /></Form.Item>
          </Space>
          <Space style={{ display: 'flex' }} align="start" size="large">
            <Form.Item name="contact" label="联系人" style={{ width: 150 }}><Input /></Form.Item>
            <Form.Item name="phone" label="电话" style={{ width: 150 }}><Input /></Form.Item>
            <Form.Item name="mobile" label="手机" style={{ width: 150 }}><Input /></Form.Item>
          </Space>
```

```
        <Space style={{ display: 'flex' }} align="start" size="large">
          <Form.Item name="bankName" label="开户银行" style={{ width: 200 }}><Input /></Form.Item>
          <Form.Item name="bankAccount" label="银行账号" style={{ width: 200 }}><Input /></Form.Item>
          <Form.Item name="taxNo" label="税务登记号" style={{ width: 200 }}><Input /></Form.Item>
        </Space>
        <Form.Item name="address" label="地址"><Input /></Form.Item>
      </Form>
    </Modal>
  </>
 );
};
export default SupplierList;
```

src/pages/settings/WarehouseForm.tsx

```
import { useState, useEffect } from 'react';
import { Form, Input, InputNumber, Modal } from 'antd';
import { warehouseApi } from '@/api/warehouse';
import { useAuthStore } from '@/stores/auth';

interface WarehouseFormProps {
  open: boolean;
  id?: string | null;
  onCancel: () => void;
  onOk: () => void;
}

const WarehouseForm = ({ open, id, onCancel, onOk }: WarehouseFormProps) => {
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(false);
  const [initLoading, setInitLoading] = useState(false);
  const tenantId = useAuthStore((s) => s.tenant?.id || '');

  useEffect(() => {
    if (open) {
      if (id) {
        setInitLoading(true);
        warehouseApi.get(id).then((res) => {
          form.setFieldsValue(res);
          setInitLoading(false);
        });
      } else {
        form.resetFields();
      }
    }
  }, [open, id, form]);

  const handleSubmit = () => {
    form.validateFields().then(async (values) => {
      setLoading(true);
      try {
        if (id) {
          await warehouseApi.update(id, values);
        } else {
          await warehouseApi.create({ ...values, tenantId } as any);
        }
        onOk();
      } finally {
        setLoading(false);
      }
    });
  };

  return (
    <Modal
      title={id ? '编辑仓库' : '新增仓库'}
      open={open}
      onCancel={onCancel}
      onOk={handleSubmit}
      confirmLoading={loading}
    >
      <Form form={form} layout="vertical" style={{ marginTop: 16 }}>
        <Form.Item name="code" label="仓库编码" rules={[{ required: true }]}>
          <Input />
        </Form.Item>
        <Form.Item name="name" label="仓库名称" rules={[{ required: true }]}>
          <Input />
        </Form.Item>
        <Form.Item name="address" label="地址">
          <Input />
        </Form.Item>
        <Form.Item name="principal" label="负责人">
          <Input />
        </Form.Item>
```

```
      <Form.Item name="sortOrder" label="排序" initialValue={0}>
        <InputNumber min={0} />
      </Form.Item>
    </Form>
  </Modal>
 );
};
export default WarehouseForm;
```

src/pages/settings/WarehouseList.tsx

```
import { useState, useEffect } from 'react';
import { Input, Select, Space, Tag } from 'antd';
import { warehouseApi, Warehouse, WarehouseQuery } from '@/api/warehouse';
import ListPage from '@/components/ListPage';
import WarehouseForm from './WarehouseForm';
import { useAuthStore } from '@/stores/auth';
import { modal } from 'antd';

const statusOptions = [
 { label: '全部', value: '' },
 { label: '启用', value: 'active' },
 { label: '禁用', value: 'inactive' },
];

const WarehouseList = () => {
 const [loading, setLoading] = useState(false);
 const [dataSource, setDataSource] = useState<Warehouse[]>([]);
 const [pagination, setPagination] = useState({ current: 1, pageSize: 20, total: 0 });
 const [query, setQuery] = useState<WarehouseQuery>({});
 const [modalVisible, setModalVisible] = useState(false);
 const [editingId, setEditingId] = useState<string | null>(null);
 const tenantId = useAuthStore((s) => s.tenant?.id || '');

 const fetchData = async () => {
  if (!tenantId) return;
  setLoading(true);
  try {
   const res = await warehouseApi.list({ ...query, tenantId } as any);
   setDataSource(res.list);
   setPagination(res.pagination);
  } finally {
   setLoading(false);
  }
 };

 useEffect(() => {
  fetchData();
 }, [query, tenantId]);

 const handleAdd = () => {
  setEditingId(null);
  setModalVisible(true);
 };

 const handleEdit = (record: Warehouse) => {
  setEditingId(record.id);
  setModalVisible(true);
 };

 const handleDelete = async (record: Warehouse) => {
  modal.confirm({
   title: '确认删除',
   content: `确定删除仓库"${record.name}"吗？`,
   onOk: async () => {
    await warehouseApi.delete(record.id);
    fetchData();
   },
  });
 };

 const handleStatusChange = async (record: Warehouse, status: string) => {
  await warehouseApi.updateStatus(record.id, status);
  fetchData();
 };

 const handleModalOk = () => {
  setModalVisible(false);
  fetchData();
 };

 const columns = [
  { title: '仓库编码', dataIndex: 'code', key: 'code', width: 120 },
  { title: '仓库名称', dataIndex: 'name', key: 'name' },
  { title: '地址', dataIndex: 'address', key: 'address' },
  { title: '负责人', dataIndex: 'principal', key: 'principal', width: 100 },
  {
```

```
    title: '状态',
    dataIndex: 'status',
    key: 'status',
    width: 80,
    render: (status: string) => (
      <Tag color={status === 'active' ? 'green' : 'red'}>
        {status === 'active' ? '启用' : '禁用'}
      </Tag>
    ),
  },
];

return (
  <>
    <div style={{ marginBottom: 16, display: 'flex', gap: 8 }}>
      <Input.Search
        placeholder="搜索编码/名称"
        style={{ width: 200 }}
        onSearch={(v) => setQuery({ ...query, keyword: v })}
      />
      <Select
        options={statusOptions}
        value={query.status || ''}
        onChange={(v) => setQuery({ ...query, status: v })}
        style={{ width: 120 }}
      />
    </div>

    <ListPage
      title="仓库管理"
      columns={columns}
      dataSource={dataSource}
      loading={loading}
      pagination={{
        current: pagination.current,
        pageSize: pagination.pageSize,
        total: pagination.total,
      }}
      onTableChange={(p) =>
        setQuery({ ...query, page: p.current, pageSize: p.pageSize })
      }
      onAdd={handleAdd}
      onEdit={handleEdit}
      onDelete={handleDelete}
      onStatusChange={handleStatusChange}
    />

    <WarehouseForm
      open={modalVisible}
      id={editingId}
      onCancel={() => setModalVisible(false)}
      onOk={handleModalOk}
    />
  </>
);
};

export default WarehouseList;
```

src/pages/settings/index.ts

```
export { default as WarehouseList } from './WarehouseList';
export { default as ProductList } from './ProductList';
export { default as CategoryManage } from './CategoryManage';
export { default as CustomerList } from './CustomerList';
export { default as SupplierList } from './SupplierList';
export { default as AccountList } from './AccountList';
```

src/stores/auth.ts

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface Tenant {
  id: string;
  name: string;
  code: string;
}

interface User {
  id: string;
  username: string;
  realName: string;
  roles: string[];
}
```

```
interface AuthState {
  token: string | null;
  refreshToken: string | null;
  user: User | null;
  tenant: Tenant | null;
  setAuth: (token: string, refreshToken: string, user: User, tenant: Tenant) => void;
  logout: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      refreshToken: null,
      user: null,
      tenant: null,

      setAuth: (token, refreshToken, user, tenant) =>
        set({ token, refreshToken, user, tenant }),

      logout: () =>
        set({
          token: null,
          refreshToken: null,
          user: null,
          tenant: null,
        }),
    }),
    {
      name: 'erp-auth',
    }
  )
);

// 前端代码合计：约 4516 行
```

后端源代码 (apps/api/src) — 共 49 个文件

src/app.module.ts

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { AuthModule } from './modules/auth/auth.module';
import { WarehouseModule } from './modules/warehouse/warehouse.module';
import { ProductCategoryModule } from './modules/product-category/product-category.module';
import { ProductModule } from './modules/product/product.module';
import { CustomerModule } from './modules/customer/customer.module';
import { SupplierModule } from './modules/supplier/supplier.module';
import { AccountModule } from './modules/account/account.module';
import { PurchaseModule } from './modules/purchase/purchase.module';
import { SaleModule } from './modules/sale/sale.module';
import { InventoryModule } from './modules/inventory/inventory.module';
import { FinanceModule } from './modules/finance/finance.module';
import { PrismaModule } from './prisma/prisma.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, envFilePath: '.env' }),
    PrismaModule,
    AuthModule,
    WarehouseModule,
    ProductCategoryModule,
    ProductModule,
    CustomerModule,
    SupplierModule,
    AccountModule,
    PurchaseModule,
    SaleModule,
    InventoryModule,
    FinanceModule,
  ],
})
export class AppModule {}
```

src/common/guards/tenant.guard.ts

```
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { Reflector } from '@nestjs/core';

@Injectable()
export class TenantGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
```

```
    const tenantId = request.headers['x-tenant-id'];
    if (!tenantId) {
      return false;
    }
    request.tenant = { id: tenantId };
    return true;
  }
}
```

src/main.ts

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.setGlobalPrefix('api/v1');

  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      transform: true,
      transformOptions: {
        enableImplicitConversion: true,
      },
    }),
  );

  app.enableCors({
    origin: 'http://localhost:3000',
    credentials: true,
  });

  await app.listen(4000);
  console.log(`Application is running on: http://localhost:4000`);
}
bootstrap();
```

src/modules/account/account.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { AccountService } from './account.service';
import { CreateAccountDto, UpdateAccountDto, AccountQueryDto } from './dto/account.dto';

@Controller('accounts')
@UseGuards(AuthGuard('jwt'))
export class AccountController {
  constructor(private accountService: AccountService) {}

  @Post()
  create(@Body() dto: CreateAccountDto & { tenantId: string }) {
    return this.accountService.create(dto.tenantId, dto);
  }

  @Get()
  findAll(@Query() query: AccountQueryDto, @Body('tenantId') tenantId: string) {
    return this.accountService.findAll(tenantId, query);
  }

  @Get('simple')
  findAllSimple(@Body('tenantId') tenantId: string) {
    return this.accountService.findAllSimple(tenantId);
  }

  @Get(':id')
  findOne(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.accountService.findOne(tenantId, id);
  }

  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateAccountDto & { tenantId: string }) {
    return this.accountService.update(dto.tenantId, id, dto);
  }

  @Delete(':id')
  remove(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.accountService.remove(tenantId, id);
  }

  @Patch(':id/status')
  updateStatus(@Param('id') id: string, @Body() body: { tenantId: string; status: string }) {
    return this.accountService.updateStatus(body.tenantId, id, body.status);
  }
}
```

src/modules/account/account.module.ts

```
import { Module } from '@nestjs/common';
import { AccountController } from './account.controller';
import { AccountService } from './account.service';

@Module({
  controllers: [AccountController],
  providers: [AccountService],
  exports: [AccountService],
})
export class AccountModule {}
```

src/modules/account/account.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateAccountDto, UpdateAccountDto, AccountQueryDto } from './dto/account.dto';

@Injectable()
export class AccountService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateAccountDto) {
    const exists = await this.prisma.account.findFirst({
      where: { tenantId, code: dto.code },
    });
    if (exists) throw new BadRequestException('账户编码已存在');

    // 如果是默认账户，先取消其他默认
    if (dto.isDefault) {
      await this.prisma.account.updateMany({
        where: { tenantId, isDefault: true },
        data: { isDefault: false },
      });
    }

    return this.prisma.account.create({ data: { tenantId, ...dto } });
  }

  async findAll(tenantId: string, query: AccountQueryDto) {
    const { keyword, type, status, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [{ code: { contains: keyword } }, { name: { contains: keyword } }];
    }
    if (type) where.type = type;
    if (status) where.status = status;

    const [list, total] = await Promise.all([
      this.prisma.account.findMany({
        where,
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { sortOrder: 'asc' },
      }),
      this.prisma.account.count({ where }),
    ]);

    return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.account.findFirst({ where: { id, tenantId } });
  }

  async findAllSimple(tenantId: string) {
    return this.prisma.account.findMany({
      where: { tenantId, status: 'active' },
      orderBy: { sortOrder: 'asc' },
    });
  }

  async update(tenantId: string, id: string, dto: UpdateAccountDto) {
    const exists = await this.prisma.account.findFirst({ where: { id, tenantId } });
    if (!exists) throw new BadRequestException('账户不存在');

    if (dto.code && dto.code !== exists.code) {
      const codeExists = await this.prisma.account.findFirst({
        where: { tenantId, code: dto.code, id: { not: id } },
      });
      if (codeExists) throw new BadRequestException('账户编码已存在');
    }

    // 如果设置为默认，先取消其他默认
    if (dto.isDefault) {
      await this.prisma.account.updateMany({
        where: { tenantId, isDefault: true, id: { not: id } },
        data: { isDefault: false },
```

```
    });
  }
  return this.prisma.account.update({ where: { id }, data: dto });
 }
 async remove(tenantId: string, id: string) {
  await this.prisma.account.delete({ where: { id, tenantId } });
 }
 async updateStatus(tenantId: string, id: string, status: string) {
  return this.prisma.account.update({ where: { id, tenantId }, data: { status } });
 }
}
```

src/modules/account/dto/account.dto.ts

```
import { IsString, IsOptional, IsNumber, MinLength, MaxLength } from 'class-validator';
import { Type } from 'class-transformer';
export class CreateAccountDto {
 @IsString()
 @MinLength(1)
 @MaxLength(50)
 code: string;
 @IsString()
 @MinLength(1)
 @MaxLength(100)
 name: string;
 @IsString()
 @IsOptional()
 type?: string;
 @IsString()
 @IsOptional()
 bankName?: string;
 @IsString()
 @IsOptional()
 bankAccount?: string;
 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 initialBalance?: number;
 @IsBoolean()
 @IsOptional()
 isDefault?: boolean;
}
export class UpdateAccountDto extends CreateAccountDto {
 @IsString()
 @IsOptional()
 status?: string;
}
class IsBoolean {
 static(value: any): boolean {
  return typeof value === 'boolean';
 }
}
export class AccountQueryDto {
 @IsString()
 @IsOptional()
 keyword?: string;
 @IsString()
 @IsOptional()
 type?: string;
 @IsString()
 @IsOptional()
 status?: string;
 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 page?: number = 1;
 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 pageSize?: number = 20;
}
```

src/modules/auth/auth.controller.ts

```
import { Controller, Post, Body, Get, UseGuards, Request } from '@nestjs/common';
```

```
import { AuthGuard } from '@nestjs/passport';
import { AuthService } from './auth.service';
import { IsString, MinLength } from 'class-validator';

class LoginDto {
  @IsString()
  @MinLength(1)
  username: string;

  @IsString()
  @MinLength(1)
  password: string;
}

class RefreshDto {
  @IsString()
  refreshToken: string;
}

@Controller('auth')
export class AuthController {
  constructor(private authService: AuthService) {}

  @Post('login')
  async login(@Body() dto: LoginDto) {
    return this.authService.login(dto.username, dto.password);
  }

  @Post('refresh')
  async refresh(@Body() dto: RefreshDto) {
    return this.authService.refreshToken(dto.refreshToken);
  }

  @UseGuards(AuthGuard('jwt'))
  @Post('logout')
  async logout() {
    return { message: '登出成功' };
  }

  @UseGuards(AuthGuard('jwt'))
  @Get('current-user')
  async getCurrentUser(@Request() req) {
    return this.authService.getCurrentUser(req.user.userId);
  }
}
```

src/modules/auth/auth.module.ts

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { JwtStrategy } from './jwt.strategy';

@Module({
  imports: [
    PassportModule.register({ defaultStrategy: 'jwt' }),
    JwtModule.registerAsync({
      imports: [ConfigModule],
      useFactory: async (configService: ConfigService) => ({
        secret: configService.get('JWT_SECRET') || 'erp-secret-key',
        signOptions: { expiresIn: '2h' },
      }),
      inject: [ConfigService],
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
  exports: [AuthService],
})
export class AuthModule {}
```

src/modules/auth/auth.service.ts

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import { PrismaService } from '../../prisma/prisma.service';

@Injectable()
export class AuthService {
  constructor(
    private prisma: PrismaService,
    private jwtService: JwtService,
  ) {}
```

```
  async login(username: string, password: string) {
    const user = await this.prisma.user.findFirst({
      where: { username },
      include: { tenant: true },
    });

    if (!user) {
      throw new UnauthorizedException('用户名或密码错误');
    }

    const isPasswordValid = await bcrypt.compare(password, user.passwordHash);
    if (!isPasswordValid) {
      throw new UnauthorizedException('用户名或密码错误');
    }

    if (user.status !== 'active') {
      throw new UnauthorizedException('账号已被禁用');
    }

    // 更新最后登录时间
    await this.prisma.user.update({
      where: { id: user.id },
      data: { lastLoginAt: new Date() },
    });

    const payload = {
      sub: user.id,
      username: user.username,
      tenantId: user.tenantId,
    };

    return {
      token: this.jwtService.sign(payload),
      refreshToken: this.jwtService.sign(payload, { expiresIn: '7d' }),
      expiresIn: 7200,
      user: {
        id: user.id,
        username: user.username,
        realName: user.realName,
        roles: ['admin'],
      },
      tenant: {
        id: user.tenant.id,
        name: user.tenant.name,
        code: user.tenant.code,
      },
    };
  }

  async refreshToken(token: string) {
    try {
      const payload = this.jwtService.verify(token);
      const user = await this.prisma.user.findUnique({
        where: { id: payload.sub },
      });

      if (!user || user.status !== 'active') {
        throw new UnauthorizedException('Token无效');
      }

      const newPayload = {
        sub: user.id,
        username: user.username,
        tenantId: user.tenantId,
      };

      return {
        token: this.jwtService.sign(newPayload),
        refreshToken: this.jwtService.sign(newPayload, { expiresIn: '7d' }),
        expiresIn: 7200,
      };
    } catch {
      throw new UnauthorizedException('Token无效');
    }
  }

  async getCurrentUser(userId: string) {
    const user = await this.prisma.user.findUnique({
      where: { id: userId },
    });

    if (!user) {
      throw new UnauthorizedException('用户不存在');
    }

    return {
      id: user.id,
      username: user.username,
      realName: user.realName,
```

```
    roles: ['admin'],
  };
 }
}
```

src/modules/auth/jwt.strategy.ts

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
 constructor() {
  super({
   jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
   ignoreExpiration: false,
   secretOrKey: process.env.JWT_SECRET || 'erp-secret-key',
  });
 }

 async validate(payload: any) {
  return {
   userId: payload.sub,
   username: payload.username,
   tenantId: payload.tenantId,
  };
 }
}
```

src/modules/customer/customer.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { CustomerService } from './customer.service';
import { CreateCustomerDto, UpdateCustomerDto, CustomerQueryDto } from './dto/customer.dto';

@Controller('customers')
@UseGuards(AuthGuard('jwt'))
export class CustomerController {
 constructor(private customerService: CustomerService) {}

 @Post()
 create(@Body() dto: CreateCustomerDto & { tenantId: string }) {
  return this.customerService.create(dto.tenantId, dto);
 }

 @Get()
 findAll(@Query() query: CustomerQueryDto, @Body('tenantId') tenantId: string) {
  return this.customerService.findAll(tenantId, query);
 }

 @Get(':id')
 findOne(@Param('id') id: string, @Body('tenantId') tenantId: string) {
  return this.customerService.findOne(tenantId, id);
 }

 @Put(':id')
 update(@Param('id') id: string, @Body() dto: UpdateCustomerDto & { tenantId: string }) {
  return this.customerService.update(dto.tenantId, id, dto);
 }

 @Delete(':id')
 remove(@Param('id') id: string, @Body('tenantId') tenantId: string) {
  return this.customerService.remove(tenantId, id);
 }

 @Patch(':id/status')
 updateStatus(@Param('id') id: string, @Body() body: { tenantId: string; status: string }) {
  return this.customerService.updateStatus(body.tenantId, id, body.status);
 }
}
```

src/modules/customer/customer.module.ts

```
import { Module } from '@nestjs/common';
import { CustomerController } from './customer.controller';
import { CustomerService } from './customer.service';

@Module({
 controllers: [CustomerController],
 providers: [CustomerService],
 exports: [CustomerService],
})
export class CustomerModule {}
```

src/modules/customer/customer.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateCustomerDto, UpdateCustomerDto, CustomerQueryDto } from './dto/customer.dto';

@Injectable()
export class CustomerService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateCustomerDto) {
    const exists = await this.prisma.customer.findFirst({
      where: { tenantId, code: dto.code },
    });
    if (exists) {
      throw new BadRequestException('客户编码已存在');
    }

    return this.prisma.customer.create({
      data: { tenantId, ...dto },
    });
  }

  async findAll(tenantId: string, query: CustomerQueryDto) {
    const { keyword, customerType, status, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [
        { code: { contains: keyword } },
        { name: { contains: keyword } },
        { contact: { contains: keyword } },
        { phone: { contains: keyword } },
      ];
    }
    if (customerType) where.customerType = customerType;
    if (status) where.status = status;

    const [list, total] = await Promise.all([
      this.prisma.customer.findMany({
        where,
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.customer.count({ where }),
    ]);

    return {
      list,
      pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) },
    };
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.customer.findFirst({ where: { id, tenantId } });
  }

  async update(tenantId: string, id: string, dto: UpdateCustomerDto) {
    const exists = await this.prisma.customer.findFirst({ where: { id, tenantId } });
    if (!exists) throw new BadRequestException('客户不存在');

    if (dto.code && dto.code !== exists.code) {
      const codeExists = await this.prisma.customer.findFirst({
        where: { tenantId, code: dto.code, id: { not: id } },
      });
      if (codeExists) throw new BadRequestException('客户编码已存在');
    }

    return this.prisma.customer.update({ where: { id }, data: dto });
  }

  async remove(tenantId: string, id: string) {
    await this.prisma.customer.delete({ where: { id, tenantId } });
  }

  async updateStatus(tenantId: string, id: string, status: string) {
    return this.prisma.customer.update({ where: { id, tenantId }, data: { status } });
  }
}
```

src/modules/customer/dto/customer.dto.ts

```
import { IsString, IsOptional, IsNumber, MinLength, MaxLength } from 'class-validator';
import { Type } from 'class-transformer';

export class CreateCustomerDto {
  @IsString()
  @MinLength(1)
  @MaxLength(50)
  code: string;

  @IsString()
```

```
  @MinLength(1)
  @MaxLength(200)
  name: string;

  @IsString()
  @IsOptional()
  contact?: string;

  @IsString()
  @IsOptional()
  phone?: string;

  @IsString()
  @IsOptional()
  mobile?: string;

  @IsString()
  @IsOptional()
  email?: string;

  @IsString()
  @IsOptional()
  address?: string;

  @IsString()
  @IsOptional()
  customerType?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  creditLimit?: number;
}

export class UpdateCustomerDto extends CreateCustomerDto {
  @IsString()
  @IsOptional()
  status?: string;
}

export class CustomerQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsString()
  @IsOptional()
  customerType?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/finance/dto/finance.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, IsArray, ValidateNested, MinLength } from 'class-va
lidator';
import { Type } from 'class-transformer';

export class VoucherItemDto {
  @IsUUID()
  accountId: string;

  @IsString()
  direction: string; // debit/credit

  @Type(() => Number)
  @IsNumber()
  amount: number;

  @IsString()
  @IsOptional()
  summary?: string;
}

export class CreateVoucherDto {
  @IsString()
  @MinLength(1)
  voucherDate: string;

  @IsString()
  @IsOptional()
```

```
  voucherType?: string;
  @IsString()
  @IsOptional()
  remark?: string;
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => VoucherItemDto)
  items: VoucherItemDto[];
}
export class UpdateVoucherDto extends CreateVoucherDto {
  @IsString()
  @IsOptional()
  status?: string;
}
export class VoucherQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
  @IsString()
  @IsOptional()
  status?: string;
  @IsString()
  @IsOptional()
  startDate?: string;
  @IsString()
  @IsOptional()
  endDate?: string;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
// 收付款
export class CreatePaymentDto {
  @IsString()
  @MinLength(1)
  paymentDate: string;
  @IsString()
  @MinLength(1)
  paymentType: string; // receipt/payment/other
  @Type(() => Number)
  @IsNumber()
  amount: number;
  @IsUUID()
  accountId: string;
  @IsUUID()
  @IsOptional()
  customerId?: string;
  @IsUUID()
  @IsOptional()
  supplierId?: string;
  @IsString()
  @IsOptional()
  relateBusinessType?: string;
  @IsUUID()
  @IsOptional()
  relateBusinessId?: string;
  @IsString()
  @IsOptional()
  relateBusinessNo?: string;
  @IsString()
  @IsOptional()
  remark?: string;
}
export class PaymentQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
  @IsString()
```

```
  @IsOptional()
  paymentType?: string;

  @IsUUID()
  @IsOptional()
  accountId?: string;

  @IsUUID()
  @IsOptional()
  customerId?: string;

  @IsUUID()
  @IsOptional()
  supplierId?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}

// 会计科目
export class CreateAccountSubjectDto {
  @IsString()
  @MinLength(1)
  code: string;

  @IsString()
  @MinLength(1)
  name: string;

  @IsString()
  @IsOptional()
  type?: string;

  @IsString()
  @IsOptional()
  direction?: string;

  @IsUUID()
  @IsOptional()
  parentId?: string;
}

export class AccountSubjectQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsString()
  @IsOptional()
  type?: string;
}
```

src/modules/finance/finance.controller.ts

```
import { Controller, Get, Post, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/common'
;
import { AuthGuard } from '@nestjs/passport';
import { FinanceService } from './finance.service';
import { CreateVoucherDto, UpdateVoucherDto, VoucherQueryDto,
  CreatePaymentDto, PaymentQueryDto,
  CreateAccountSubjectDto, AccountSubjectQueryDto } from './dto/finance.dto';

@Controller('finance')
@UseGuards(AuthGuard('jwt'))
export class FinanceController {
  constructor(private financeService: FinanceService) {}

  // ========== 会计科目 ==========
  @Get('subjects')
  findSubjects(@Query() query: AccountSubjectQueryDto, @Body('tenantId') tenantId: string) {
    return this.financeService.findSubjects(tenantId, query);
  }

  @Post('subjects')
  createSubject(@Body() dto: CreateAccountSubjectDto & { tenantId: string }) {
    return this.financeService.createSubject(dto.tenantId, dto);
  }

  @Post('subjects/init')
  initSubjects(@Body('tenantId') tenantId: string) {
    return this.financeService.initDefaultSubjects(tenantId);
```

```
  }
  // ========== 凭证管理 ==========
  @Post('vouchers')
  createVoucher(@Body() dto: CreateVoucherDto & { tenantId: string }) {
    return this.financeService.createVoucher(dto.tenantId, dto);
  }
  @Get('vouchers')
  findVouchers(@Query() query: VoucherQueryDto, @Body('tenantId') tenantId: string) {
    return this.financeService.findVouchers(tenantId, query);
  }
  @Get('vouchers/:id')
  findVoucherById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.financeService.findVoucherById(tenantId, id);
  }
  @Patch('vouchers/:id/audit')
  auditVoucher(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.financeService.auditVoucher(body.tenantId, id, body.userId);
  }
  @Patch('vouchers/:id/post')
  postVoucher(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.financeService.postVoucher(body.tenantId, id, body.userId);
  }
  @Delete('vouchers/:id')
  deleteVoucher(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.financeService.deleteVoucher(tenantId, id);
  }
  // ========== 收付款 ==========
  @Post('payments')
  createPayment(@Body() dto: CreatePaymentDto & { tenantId: string }) {
    return this.financeService.createPayment(dto.tenantId, dto);
  }
  @Get('payments')
  findPayments(@Query() query: PaymentQueryDto, @Body('tenantId') tenantId: string) {
    return this.financeService.findPayments(tenantId, query);
  }
  @Patch('payments/:id/audit')
  auditPayment(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.financeService.auditPayment(body.tenantId, id, body.userId);
  }
  // ========== 财务报表 ==========
  @Get('reports/profit')
  getProfitStatement(
    @Query('tenantId') tenantId: string,
    @Query('startDate') startDate: string,
    @Query('endDate') endDate: string,
  ) {
    return this.financeService.getProfitStatement(tenantId, startDate, endDate);
  }
}
```

src/modules/finance/finance.module.ts

```
import { Module } from '@nestjs/common';
import { FinanceController } from './finance.controller';
import { FinanceService } from './finance.service';

@Module({
  controllers: [FinanceController],
  providers: [FinanceService],
  exports: [FinanceService],
})
export class FinanceModule {}
```

src/modules/finance/finance.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateVoucherDto, UpdateVoucherDto, VoucherQueryDto,
  CreatePaymentDto, PaymentQueryDto,
  CreateAccountSubjectDto, AccountSubjectQueryDto } from './dto/finance.dto';

@Injectable()
export class FinanceService {
  constructor(private prisma: PrismaService) {}

  // ========== 会计科目 ==========
  async findSubjects(tenantId: string, query: AccountSubjectQueryDto) {
    const { keyword, type } = query;

    const where: any = { tenantId };
```

```
    if (keyword) {
      where.OR = [
        { code: { contains: keyword } },
        { name: { contains: keyword } },
      ];
    }
    if (type) where.type = type;

    const subjects = await this.prisma.accountSubject.findMany({
      where,
      orderBy: { code: 'asc' },
    });

    return this.buildTree(subjects);
  }

  private buildTree(subjects: any[]) {
    const map = new Map<string, any>();
    const roots: any[] = [];

    subjects.forEach(s => map.set(s.id, { ...s, children: [] }));
    subjects.forEach(s => {
      const node = map.get(s.id);
      if (s.parentId && map.has(s.parentId)) {
        map.get(s.parentId).children.push(node);
      } else {
        roots.push(node);
      }
    });

    return roots;
  }

  async createSubject(tenantId: string, dto: CreateAccountSubjectDto) {
    return this.prisma.accountSubject.create({
      data: { tenantId, ...dto },
    });
  }

  async initDefaultSubjects(tenantId: string) {
    const existing = await this.prisma.accountSubject.count({ where: { tenantId } });
    if (existing > 0) return;

    const defaults = [
      { code: '1001', name: '库存商品', type: 'asset', direction: '借' },
      { code: '1002', name: '应收账款', type: 'asset', direction: '借' },
      { code: '1003', name: '其他应收款', type: 'asset', direction: '借' },
      { code: '2001', name: '应付账款', type: 'liability', direction: '贷' },
      { code: '2002', name: '其他应付款', type: 'liability', direction: '贷' },
      { code: '4001', name: '主营业务收入', type: 'revenue', direction: '贷' },
      { code: '5001', name: '主营业务成本', type: 'expense', direction: '借' },
      { code: '1004', name: '银行存款', type: 'asset', direction: '借' },
      { code: '1005', name: '库存现金', type: 'asset', direction: '借' },
    ];

    for (const d of defaults) {
      await this.prisma.accountSubject.create({ data: { tenantId, ...d } });
    }
  }

  // ========== 凭证管理 ==========
  async createVoucher(tenantId: string, dto: CreateVoucherDto) {
    const totalDebit = dto.items.filter(i => i.direction === 'debit').reduce((s, i) => s + i.amount,
0);
    const totalCredit = dto.items.filter(i => i.direction === 'credit').reduce((s, i) => s + i.amoun
t, 0);

    if (Math.abs(totalDebit - totalCredit) > 0.01) {
      throw new BadRequestException('借贷不平衡');
    }

    return this.prisma.$transaction(async (tx) => {
      const voucher = await tx.voucher.create({
        data: {
          tenantId,
          voucherDate: new Date(dto.voucherDate),
          voucherType: dto.voucherType || '记',
          totalDebit,
          totalCredit,
          remark: dto.remark,
          items: {
            create: dto.items.map(item => ({
              tenantId,
              accountId: item.accountId,
              direction: item.direction,
              amount: item.amount,
              summary: item.summary,
            })),
          },
```

```
      },
      include: { items: { include: { account: true } } },
    });

    return voucher;
  });
}

async findVouchers(tenantId: string, query: VoucherQueryDto) {
  const { keyword, status, startDate, endDate, page = 1, pageSize = 20 } = query;

  const where: any = { tenantId };
  if (keyword) where.OR = [{ voucherNo: { contains: keyword } }, { remark: { contains: keyword } }
];
  if (status) where.status = status;
  if (startDate || endDate) {
    where.voucherDate = {};
    if (startDate) where.voucherDate.gte = new Date(startDate);
    if (endDate) where.voucherDate.lte = new Date(endDate);
  }

  const [list, total] = await Promise.all([
    this.prisma.voucher.findMany({
      where,
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { voucherDate: 'desc' },
    }),
    this.prisma.voucher.count({ where }),
  ]);

  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}

async findVoucherById(tenantId: string, id: string) {
  return this.prisma.voucher.findFirst({
    where: { id, tenantId },
    include: { items: { include: { account: true } } },
  });
}

async auditVoucher(tenantId: string, id: string, userId: string) {
  const voucher = await this.prisma.voucher.findFirst({ where: { id, tenantId } });
  if (!voucher) throw new BadRequestException('凭证不存在');
  if (voucher.status !== 'draft') throw new BadRequestException('只能审核草稿状态的凭证');

  return this.prisma.voucher.update({
    where: { id },
    data: { status: 'audited', auditedById: userId, auditedAt: new Date() },
  });
}

async postVoucher(tenantId: string, id: string, userId: string) {
  const voucher = await this.prisma.voucher.findFirst({ where: { id, tenantId } });
  if (!voucher) throw new BadRequestException('凭证不存在');
  if (voucher.status !== 'audited') throw new BadRequestException('只能过账已审核的凭证');

  return this.prisma.voucher.update({
    where: { id },
    data: { status: 'posted', postedById: userId, postedAt: new Date() },
  });
}

async deleteVoucher(tenantId: string, id: string) {
  const voucher = await this.prisma.voucher.findFirst({ where: { id, tenantId } });
  if (!voucher) throw new BadRequestException('凭证不存在');
  if (voucher.status !== 'draft') throw new BadRequestException('只能删除草稿状态的凭证');

  await this.prisma.voucherItem.deleteMany({ where: { voucherId: id } });
  await this.prisma.voucher.delete({ where: { id } });
}

// ========== 收付款 ==========
async createPayment(tenantId: string, dto: CreatePaymentDto) {
  return this.prisma.$transaction(async (tx) => {
    const payment = await tx.payment.create({
      data: {
        tenantId,
        paymentDate: new Date(dto.paymentDate),
        paymentType: dto.paymentType,
        amount: dto.amount,
        accountId: dto.accountId,
        customerId: dto.customerId,
        supplierId: dto.supplierId,
        relateBusinessType: dto.relateBusinessType,
        relateBusinessId: dto.relateBusinessId,
        relateBusinessNo: dto.relateBusinessNo,
        remark: dto.remark,
      },
```

```
    });
    // 更新账户余额
    const account = await tx.account.findFirst({ where: { id: dto.accountId } });
    if (account) {
      const balanceChange = dto.paymentType === 'receipt' ? dto.amount : -dto.amount;
      await tx.account.update({
        where: { id: dto.accountId },
        data: { currentBalance: { increment: balanceChange } },
      });
    }
    return payment;
  });
}
 async findPayments(tenantId: string, query: PaymentQueryDto) {
   const { keyword, paymentType, accountId, customerId, supplierId, status, page = 1, pageSize = 20
} = query;
   const where: any = { tenantId };
   if (keyword) where.OR = [{ paymentNo: { contains: keyword } }];
   if (paymentType) where.paymentType = paymentType;
   if (accountId) where.accountId = accountId;
   if (customerId) where.customerId = customerId;
   if (supplierId) where.supplierId = supplierId;
   if (status) where.status = status;
   const [list, total] = await Promise.all([
     this.prisma.payment.findMany({
       where,
       include: { account: true, customer: true, supplier: true },
       skip: (page - 1) * pageSize,
       take: pageSize,
       orderBy: { paymentDate: 'desc' },
     }),
     this.prisma.payment.count({ where }),
   ]);
   return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
 }
 async auditPayment(tenantId: string, id: string, userId: string) {
   const payment = await this.prisma.payment.findFirst({ where: { id, tenantId } });
   if (!payment) throw new BadRequestException('收付款单不存在');
   if (payment.status !== 'pending') throw new BadRequestException('只能审核待审核单据');
   return this.prisma.payment.update({
     where: { id },
     data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
   });
 }
 // ========== 财务报表 ==========
 async getProfitStatement(tenantId: string, startDate: string, endDate: string) {
   // 简化实现
   const revenues = await this.prisma.voucherItem.aggregate({
     where: {
       tenantId,
       direction: 'credit',
       voucher: {
         voucherDate: { gte: new Date(startDate), lte: new Date(endDate) },
         status: 'posted',
       },
     },
     _sum: { amount: true },
   });
   const expenses = await this.prisma.voucherItem.aggregate({
     where: {
       tenantId,
       direction: 'debit',
       voucher: {
         voucherDate: { gte: new Date(startDate), lte: new Date(endDate) },
         status: 'posted',
       },
     },
     _sum: { amount: true },
   });
   return {
     title: '利润表',
     period: `${startDate} ~ ${endDate}`,
     items: [
       { name: '营业收入', current: revenues._sum.amount || 0, previous: 0 },
       { name: '减:营业成本', current: expenses._sum.amount || 0, previous: 0 },
       { name: '营业利润', current: (revenues._sum.amount || 0) - (expenses._sum.amount || 0), previous
: 0 },
```

```
    ],
  };
 }
}
```

src/modules/inventory/dto/inventory.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, IsArray, ValidateNested, MinLength } from 'class-va
lidator';
import { Type } from 'class-transformer';

export class StockCheckItemDto {
 @IsUUID()
 productId: string;

 @Type(() => Number)
 @IsNumber()
 bookQuantity: number;

 @Type(() => Number)
 @IsNumber()
 checkQuantity: number;

 @Type(() => Number)
 @IsNumber()
 diffQuantity: number;

 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 costPrice?: number;

 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 diffAmount?: number;

 @IsString()
 @IsOptional()
 reason?: string;
}

export class CreateStockCheckDto {
 @IsString()
 @MinLength(1)
 checkDate: string;

 @IsUUID()
 warehouseId: string;

 @IsString()
 @IsOptional()
 remark?: string;

 @IsArray()
 @ValidateNested({ each: true })
 @Type(() => StockCheckItemDto)
 items: StockCheckItemDto[];
}

export class StockCheckQueryDto {
 @IsString()
 @IsOptional()
 keyword?: string;

 @IsUUID()
 @IsOptional()
 warehouseId?: string;

 @IsString()
 @IsOptional()
 status?: string;

 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 page?: number = 1;

 @Type(() => Number)
 @IsNumber()
 @IsOptional()
 pageSize?: number = 20;
}

// 库存调拨
export class StockTransferItemDto {
 @IsUUID()
 productId: string;

 @Type(() => Number)
 @IsNumber()
 quantity: number;
```

```
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  costPrice?: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  amount?: number;
}

export class CreateStockTransferDto {
  @IsString()
  @MinLength(1)
  transferDate: string;

  @IsUUID()
  fromWarehouseId: string;

  @IsUUID()
  toWarehouseId: string;

  @IsString()
  @IsOptional()
  remark?: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => StockTransferItemDto)
  items: StockTransferItemDto[];
}

export class StockTransferQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  fromWarehouseId?: string;

  @IsUUID()
  @IsOptional()
  toWarehouseId?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}

// 库存查询
export class InventoryQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  productId?: string;

  @IsUUID()
  @IsOptional()
  warehouseId?: string;

  @IsString()
  @IsOptional()
  stockStatus?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/inventory/inventory.controller.ts

```
import { Controller, Get, Post, Patch, Body, Param, Query, UseGuards } from '@nestjs/common';
```

```
import { AuthGuard } from '@nestjs/passport';
import { InventoryService } from './inventory.service';
import { CreateStockCheckDto, StockCheckQueryDto,
  CreateStockTransferDto, StockTransferQueryDto,
  InventoryQueryDto } from './dto/inventory.dto';

@Controller('inventory')
@UseGuards(AuthGuard('jwt'))
export class InventoryController {
  constructor(private inventoryService: InventoryService) {}

  // ========== 库存查询 ==========
  @Get()
  findInventories(@Query() query: InventoryQueryDto, @Body('tenantId') tenantId: string) {
    return this.inventoryService.findInventories(tenantId, query);
  }

  @Get('summary')
  getSummary(@Body('tenantId') tenantId: string) {
    return this.inventoryService.getInventorySummary(tenantId);
  }

  // ========== 库存预警 ==========
  @Get('warnings')
  getWarnings(@Body('tenantId') tenantId: string) {
    return this.inventoryService.getWarnings(tenantId);
  }

  // ========== 库存盘点 ==========
  @Post('checks')
  createStockCheck(@Body() dto: CreateStockCheckDto & { tenantId: string }) {
    return this.inventoryService.createStockCheck(dto.tenantId, dto);
  }

  @Get('checks')
  findStockChecks(@Query() query: StockCheckQueryDto, @Body('tenantId') tenantId: string) {
    return this.inventoryService.findStockChecks(tenantId, query);
  }

  @Get('checks/:id')
  findStockCheckById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.inventoryService.findStockCheckById(tenantId, id);
  }

  @Patch('checks/:id/audit')
  auditStockCheck(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.inventoryService.auditStockCheck(body.tenantId, id, body.userId);
  }

  @Patch('checks/:id/approve')
  approveStockCheck(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.inventoryService.approveStockCheck(body.tenantId, id, body.userId);
  }

  // ========== 库存调拨 ==========
  @Post('transfers')
  createStockTransfer(@Body() dto: CreateStockTransferDto & { tenantId: string }) {
    return this.inventoryService.createStockTransfer(dto.tenantId, dto);
  }

  @Get('transfers')
  findStockTransfers(@Query() query: StockTransferQueryDto, @Body('tenantId') tenantId: string) {
    return this.inventoryService.findStockTransfers(tenantId, query);
  }

  @Get('transfers/:id')
  findStockTransferById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.inventoryService.findStockTransferById(tenantId, id);
  }

  @Patch('transfers/:id/audit')
  auditStockTransfer(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.inventoryService.auditStockTransfer(body.tenantId, id, body.userId);
  }
}
```

src/modules/inventory/inventory.module.ts

```
import { Module } from '@nestjs/common';
import { InventoryController } from './inventory.controller';
import { InventoryService } from './inventory.service';

@Module({
  controllers: [InventoryController],
  providers: [InventoryService],
  exports: [InventoryService],
})
export class InventoryModule {}
```

src/modules/inventory/inventory.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateStockCheckDto, StockCheckQueryDto,
  CreateStockTransferDto, StockTransferQueryDto,
  InventoryQueryDto } from './dto/inventory.dto';

@Injectable()
export class InventoryService {
  constructor(private prisma: PrismaService) {}

  // ========== 库存查询 ==========
  async findInventories(tenantId: string, query: InventoryQueryDto) {
    const { keyword, productId, warehouseId, stockStatus, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (productId) where.productId = productId;
    if (warehouseId) where.warehouseId = warehouseId;

    // 子查询来筛选库存状态
    if (stockStatus === 'low') {
      where.quantity = { gt: 0, lte: 100 }; // 简化：低于某值
    } else if (stockStatus === 'over') {
      where.quantity = { gt: 1000 }; // 简化：超过某值
    } else if (stockStatus === 'zero') {
      where.quantity = 0;
    }

    if (keyword) {
      where.product = {
        OR: [
          { name: { contains: keyword } },
          { code: { contains: keyword } },
        ],
      };
    }

    const [list, total] = await Promise.all([
      this.prisma.inventory.findMany({
        where,
        include: { product: true, warehouse: true },
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { updatedAt: 'desc' },
      }),
      this.prisma.inventory.count({ where }),
    ]);

    return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
  }

  async getInventorySummary(tenantId: string) {
    const [totalProductTypes, totalQuantity, lowStockCount, overStockCount] = await Promise.all([
      this.prisma.inventory.count({ where: { tenantId, quantity: { gt: 0 } } }),
      this.prisma.inventory.aggregate({ where: { tenantId }, _sum: { quantity: true } }),
      this.prisma.inventory.count({ where: { tenantId, quantity: { gt: 0, lte: 100 } } }),
      this.prisma.inventory.count({ where: { tenantId, quantity: { gt: 1000 } } }),
    ]);

    return {
      totalProductTypes,
      totalQuantity: totalQuantity._sum.quantity || 0,
      lowStockCount,
      overStockCount,
    };
  }

  // ========== 库存预警 ==========
  async getWarnings(tenantId: string) {
    const products = await this.prisma.product.findMany({
      where: { tenantId, status: 'active' },
      include: { category: true },
    });

    const warnings: any[] = [];
    for (const product of products) {
      const inventories = await this.prisma.inventory.findMany({
        where: { tenantId, productId: product.id },
      });

      const totalQty = inventories.reduce((sum, inv) => sum + (inv.quantity || 0), 0);

      if (product.minStock && totalQty < product.minStock) {
        warnings.push({ product, type: 'low', message: '库存不足' });
      }
      if (product.maxStock && totalQty > product.maxStock) {
        warnings.push({ product, type: 'over', message: '超储' });
      }
    }

    return warnings.slice(0, 10);
```

```
}
// ========== 库存盘点 ==========
async createStockCheck(tenantId: string, dto: CreateStockCheckDto) {
  const totalDiffAmount = dto.items.reduce((sum, item) => sum + (item.diffAmount || 0), 0);

  return this.prisma.$transaction(async (tx) => {
    const check = await tx.stockCheck.create({
      data: {
        tenantId,
        checkDate: new Date(dto.checkDate),
        warehouseId: dto.warehouseId,
        remark: dto.remark,
        totalAmount: totalDiffAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            bookQuantity: item.bookQuantity,
            checkQuantity: item.checkQuantity,
            diffQuantity: item.diffQuantity,
            costPrice: item.costPrice || 0,
            diffAmount: item.diffAmount || 0,
            reason: item.reason,
          })),
        },
      },
      include: { items: true },
    });

    return check;
  });
}

async findStockChecks(tenantId: string, query: StockCheckQueryDto) {
  const { keyword, warehouseId, status, page = 1, pageSize = 20 } = query;

  const where: any = { tenantId };
  if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
  if (warehouseId) where.warehouseId = warehouseId;
  if (status) where.status = status;

  const [list, total] = await Promise.all([
    this.prisma.stockCheck.findMany({
      where,
      include: { warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.stockCheck.count({ where }),
  ]);

  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}

async findStockCheckById(tenantId: string, id: string) {
  return this.prisma.stockCheck.findFirst({
    where: { id, tenantId },
    include: { items: { include: { product: true } }, warehouse: true },
  });
}

async auditStockCheck(tenantId: string, id: string, userId: string) {
  const check = await this.prisma.stockCheck.findFirst({ where: { id, tenantId } });
  if (!check) throw new BadRequestException('盘点单不存在');
  if (check.status !== 'pending') throw new BadRequestException('只能审核待审核盘点单');

  return this.prisma.stockCheck.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}

async approveStockCheck(tenantId: string, id: string, userId: string) {
  const check = await this.prisma.stockCheck.findFirst({
    where: { id, tenantId },
    include: { items: true },
  });
  if (!check) throw new BadRequestException('盘点单不存在');
  if (check.status !== 'approved') throw new BadRequestException('只能确认已审核的盘点单');

  return this.prisma.$transaction(async (tx) => {
    // 更新库存
    for (const item of check.items) {
      await tx.inventory.update({
        where: {
          tenantId_productId_warehouseId: {
            tenantId,
```

```
          productId: item.productId,
          warehouseId: check.warehouseId,
        },
      },
      data: { quantity: item.checkQuantity },
    });

    // 记录库存变动
    await tx.inventoryLog.create({
      data: {
        tenantId,
        productId: item.productId,
        warehouseId: check.warehouseId,
        businessType: 'stock_check',
        businessId: check.id,
        businessNo: check.orderNo,
        quantity: item.diffQuantity,
        beforeQuantity: item.bookQuantity,
        afterQuantity: item.checkQuantity,
        price: item.costPrice,
        amount: item.diffAmount,
        remark: item.reason,
      },
    });
  }

  return tx.stockCheck.update({
    where: { id },
    data: { status: 'completed' },
  });
});
}

// ========== 库存调拨 ==========
async createStockTransfer(tenantId: string, dto: CreateStockTransferDto) {
  const totalAmount = dto.items.reduce((sum, item) => sum + (item.amount || 0), 0);

  return this.prisma.$transaction(async (tx) => {
    const transfer = await tx.stockTransfer.create({
      data: {
        tenantId,
        transferDate: new Date(dto.transferDate),
        fromWarehouseId: dto.fromWarehouseId,
        toWarehouseId: dto.toWarehouseId,
        remark: dto.remark,
        totalAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            quantity: item.quantity,
            costPrice: item.costPrice || 0,
            amount: item.amount || 0,
          })),
        },
      },
      include: { items: true },
    });

    // 出库仓库减少
    for (const item of dto.items) {
      const inv = await tx.inventory.findFirst({
        where: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.fromWarehouseId,
        },
      });

      if (!inv || inv.quantity < item.quantity) {
        throw new BadRequestException(`商品库存不足`);
      }

      await tx.inventory.update({
        where: {
          tenantId_productId_warehouseId: {
            tenantId,
            productId: item.productId,
            warehouseId: dto.fromWarehouseId,
          },
        },
        data: { quantity: { decrement: item.quantity } },
      });

      await tx.inventoryLog.create({
        data: {
```

```
        tenantId,
        productId: item.productId,
        warehouseId: dto.fromWarehouseId,
        businessType: 'transfer_out',
        businessId: transfer.id,
        businessNo: transfer.orderNo,
        quantity: -item.quantity,
        beforeQuantity: inv.quantity,
        afterQuantity: inv.quantity - item.quantity,
        price: item.costPrice,
        amount: item.amount,
      },
    });
  }

  // 入库仓库增加
  for (const item of dto.items) {
    await tx.inventory.upsert({
      where: {
        tenantId_productId_warehouseId: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.toWarehouseId,
        },
      },
      create: {
        tenantId,
        productId: item.productId,
        warehouseId: dto.toWarehouseId,
        quantity: item.quantity,
        costPrice: item.costPrice || 0,
      },
      update: {
        quantity: { increment: item.quantity },
      },
    });

    const inv = await tx.inventory.findFirst({
      where: {
        tenantId,
        productId: item.productId,
        warehouseId: dto.toWarehouseId,
      },
    });

    await tx.inventoryLog.create({
      data: {
        tenantId,
        productId: item.productId,
        warehouseId: dto.toWarehouseId,
        businessType: 'transfer_in',
        businessId: transfer.id,
        businessNo: transfer.orderNo,
        quantity: item.quantity,
        beforeQuantity: (inv?.quantity || 0) - item.quantity,
        afterQuantity: inv?.quantity || item.quantity,
        price: item.costPrice,
        amount: item.amount,
      },
    });
  }

  return transfer;
});
}

async findStockTransfers(tenantId: string, query: StockTransferQueryDto) {
  const { keyword, fromWarehouseId, toWarehouseId, status, page = 1, pageSize = 20 } = query;

  const where: any = { tenantId };
  if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
  if (fromWarehouseId) where.fromWarehouseId = fromWarehouseId;
  if (toWarehouseId) where.toWarehouseId = toWarehouseId;
  if (status) where.status = status;

  const [list, total] = await Promise.all([
    this.prisma.stockTransfer.findMany({
      where,
      include: { fromWarehouse: true, toWarehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    },
    this.prisma.stockTransfer.count({ where }),
  ]);
```

```
    return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
  }

  async findStockTransferById(tenantId: string, id: string) {
    return this.prisma.stockTransfer.findFirst({
      where: { id, tenantId },
      include: { items: { include: { product: true } }, fromWarehouse: true, toWarehouse: true },
    });
  }

  async auditStockTransfer(tenantId: string, id: string, userId: string) {
    const transfer = await this.prisma.stockTransfer.findFirst({ where: { id, tenantId } });
    if (!transfer) throw new BadRequestException('调拨单不存在');
    if (transfer.status !== 'pending') throw new BadRequestException('只能审核待审核调拨单');

    return this.prisma.stockTransfer.update({
      where: { id },
      data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
    });
  }
}
```

src/modules/product-category/dto/product-category.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, MinLength, MaxLength } from 'class-validator';

export class CreateProductCategoryDto {
  @IsUUID()
  @IsOptional()
  parentId?: string;

  @IsString()
  @MinLength(1)
  @MaxLength(100)
  name: string;

  @IsNumber()
  @IsOptional()
  sortOrder?: number;
}

export class UpdateProductCategoryDto {
  @IsString()
  @IsOptional()
  name?: string;

  @IsNumber()
  @IsOptional()
  sortOrder?: number;
}

export class ProductCategoryQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
}
```

src/modules/product-category/product-category.controller.ts

```
import { Controller, Get, Post, Put, Delete, Body, Param, Query, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { ProductCategoryService } from './product-category.service';
import { CreateProductCategoryDto, UpdateProductCategoryDto } from './dto/product-category.dto';

@Controller('product-categories')
@UseGuards(AuthGuard('jwt'))
export class ProductCategoryController {
  constructor(private categoryService: ProductCategoryService) {}

  @Post()
  create(@Body() dto: CreateProductCategoryDto & { tenantId: string }) {
    return this.categoryService.create(dto.tenantId, dto);
  }

  @Get()
  findAll(@Query('tenantId') tenantId: string) {
    return this.categoryService.findAll(tenantId);
  }

  @Get(':id')
  findOne(@Param('id') id: string, @Query('tenantId') tenantId: string) {
    return this.categoryService.findOne(tenantId, id);
  }

  @Put(':id')
  update(
    @Param('id') id: string,
    @Body() dto: UpdateProductCategoryDto & { tenantId: string },
  ) {
    return this.categoryService.update(dto.tenantId, id, dto);
```

```
  }
  @Delete(':id')
  remove(@Param('id') id: string, @Query('tenantId') tenantId: string) {
    return this.categoryService.remove(tenantId, id);
  }
}
```

src/modules/product-category/product-category.module.ts

```
import { Module } from '@nestjs/common';
import { ProductCategoryController } from './product-category.controller';
import { ProductCategoryService } from './product-category.service';

@Module({
  controllers: [ProductCategoryController],
  providers: [ProductCategoryService],
  exports: [ProductCategoryService],
})
export class ProductCategoryModule {}
```

src/modules/product-category/product-category.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateProductCategoryDto, UpdateProductCategoryDto } from './dto/product-category.dto';

@Injectable()
export class ProductCategoryService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateProductCategoryDto) {
    if (dto.parentId) {
      // 验证父分类存在
      const parent = await this.prisma.productCategory.findFirst({
        where: { id: dto.parentId, tenantId },
      });
      if (!parent) {
        throw new BadRequestException('父分类不存在');
      }
    }

    return this.prisma.productCategory.create({
      data: {
        tenantId,
        ...dto,
      },
    });
  }

  async findAll(tenantId: string) {
    // 返回树形结构
    const categories = await this.prisma.productCategory.findMany({
      where: { tenantId },
      orderBy: { sortOrder: 'asc' },
    });

    return this.buildTree(categories);
  }

  private buildTree(categories: any[]) {
    const map = new Map<string, any>();
    const roots: any[] = [];

    categories.forEach((cat) => {
      map.set(cat.id, { ...cat, children: [] });
    });

    categories.forEach((cat) => {
      const node = map.get(cat.id);
      if (cat.parentId && map.has(cat.parentId)) {
        map.get(cat.parentId).children.push(node);
      } else {
        roots.push(node);
      }
    });

    return roots;
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.productCategory.findFirst({
      where: { id, tenantId },
    });
  }

  async update(tenantId: string, id: string, dto: UpdateProductCategoryDto) {
    const exists = await this.prisma.productCategory.findFirst({
      where: { id, tenantId },
```

```
    });
    if (!exists) {
      throw new BadRequestException('分类不存在');
    }

    return this.prisma.productCategory.update({
      where: { id },
      data: dto,
    });
  }

  async remove(tenantId: string, id: string) {
    // 检查是否有子分类
    const children = await this.prisma.productCategory.findMany({
      where: { parentId: id, tenantId },
    });
    if (children.length > 0) {
      throw new BadRequestException('请先删除子分类');
    }

    // 检查是否有商品使用此分类
    const products = await this.prisma.product.findMany({
      where: { categoryId: id, tenantId },
    });
    if (products.length > 0) {
      throw new BadRequestException('该分类下有商品，无法删除');
    }

    await this.prisma.productCategory.delete({
      where: { id },
    });
  }
}
```

src/modules/product/dto/product.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, MinLength, MaxLength } from 'class-validator';
import { Type } from 'class-transformer';

export class CreateProductDto {
  @IsUUID()
  @IsOptional()
  categoryId?: string;

  @IsString()
  @MinLength(1)
  @MaxLength(50)
  code: string;

  @IsString()
  @MinLength(1)
  @MaxLength(200)
  name: string;

  @IsString()
  @IsOptional()
  spec?: string;

  @IsString()
  @IsOptional()
  unit?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  purchasePrice?: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  salePrice?: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  costPrice?: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  minStock?: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  maxStock?: number;

  @IsString()
  @IsOptional()
  imageUrl?: string;
```

```
  @IsString()
  @IsOptional()
  description?: string;
}
export class UpdateProductDto extends CreateProductDto {
  @IsString()
  @IsOptional()
  status?: string;
}
export class ProductQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
  @IsUUID()
  @IsOptional()
  categoryId?: string;
  @IsString()
  @IsOptional()
  status?: string;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/product/product.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { ProductService } from './product.service';
import { CreateProductDto, UpdateProductDto, ProductQueryDto } from './dto/product.dto';
@Controller('products')
@UseGuards(AuthGuard('jwt'))
export class ProductController {
  constructor(private productService: ProductService) {}
  @Post()
  create(@Body() dto: CreateProductDto & { tenantId: string }) {
    return this.productService.create(dto.tenantId, dto);
  }
  @Get()
  findAll(@Query() query: ProductQueryDto, @Body('tenantId') tenantId: string) {
    return this.productService.findAll(tenantId, query);
  }
  @Get(':id')
  findOne(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.productService.findOne(tenantId, id);
  }
  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateProductDto & { tenantId: string }) {
    return this.productService.update(dto.tenantId, id, dto);
  }
  @Delete(':id')
  remove(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.productService.remove(tenantId, id);
  }
  @Patch(':id/status')
  updateStatus(
    @Param('id') id: string,
    @Body() body: { tenantId: string; status: string },
  ) {
    return this.productService.updateStatus(body.tenantId, id, body.status);
  }
}
```

src/modules/product/product.module.ts

```
import { Module } from '@nestjs/common';
import { ProductController } from './product.controller';
import { ProductService } from './product.service';
@Module({
  controllers: [ProductController],
```

```
  providers: [ProductService],
  exports: [ProductService],
})
export class ProductModule {}
```

src/modules/product/product.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateProductDto, UpdateProductDto, ProductQueryDto } from './dto/product.dto';

@Injectable()
export class ProductService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateProductDto) {
    const exists = await this.prisma.product.findFirst({
      where: { tenantId, code: dto.code },
    });
    if (exists) {
      throw new BadRequestException('商品编码已存在');
    }

    if (dto.categoryId) {
      const category = await this.prisma.productCategory.findFirst({
        where: { id: dto.categoryId, tenantId },
      });
      if (!category) {
        throw new BadRequestException('商品分类不存在');
      }
    }

    return this.prisma.product.create({
      data: {
        tenantId,
        ...dto,
      },
    });
  }

  async findAll(tenantId: string, query: ProductQueryDto) {
    const { keyword, categoryId, status, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [
        { code: { contains: keyword } },
        { name: { contains: keyword } },
        { spec: { contains: keyword } },
      ];
    }
    if (categoryId) {
      where.categoryId = categoryId;
    }
    if (status) {
      where.status = status;
    }

    const [list, total] = await Promise.all([
      this.prisma.product.findMany({
        where,
        include: { category: true },
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.product.count({ where }),
    ]);

    return {
      list,
      pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) },
    };
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.product.findFirst({
      where: { id, tenantId },
      include: { category: true },
    });
  }

  async update(tenantId: string, id: string, dto: UpdateProductDto) {
    const exists = await this.prisma.product.findFirst({
      where: { id, tenantId },
    });
    if (!exists) {
```

```
      throw new BadRequestException('商品不存在');
    }
    if (dto.code && dto.code !== exists.code) {
      const codeExists = await this.prisma.product.findFirst({
        where: { tenantId, code: dto.code, id: { not: id } },
      });
      if (codeExists) {
        throw new BadRequestException('商品编码已存在');
      }
    }
    return this.prisma.product.update({
      where: { id },
      data: dto,
    });
  }
  async remove(tenantId: string, id: string) {
    // 检查库存
    const inventory = await this.prisma.inventory.findFirst({
      where: { productId: id, tenantId, quantity: { gt: 0 } },
    });
    if (inventory) {
      throw new BadRequestException('商品有库存，无法删除');
    }
    await this.prisma.product.delete({
      where: { id, tenantId },
    });
  }
  async updateStatus(tenantId: string, id: string, status: string) {
    return this.prisma.product.update({
      where: { id, tenantId },
      data: { status },
    });
  }
}
```

src/modules/purchase/dto/purchase.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, IsArray, ValidateNested, MinLength } from 'class-va
lidator';
import { Type } from 'class-transformer';
export class PurchaseOrderItemDto {
  @IsUUID()
  @IsOptional()
  id?: string;
  @IsUUID()
  productId: string;
  @IsUUID()
  @IsOptional()
  warehouseId?: string;
  @Type(() => Number)
  @IsNumber()
  quantity: number;
  @Type(() => Number)
  @IsNumber()
  price: number;
  @Type(() => Number)
  @IsNumber()
  amount: number;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  taxRate?: number;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  taxAmount?: number;
  @IsString()
  @IsOptional()
  remark?: string;
}
export class CreatePurchaseOrderDto {
  @IsString()
  @MinLength(1)
  orderDate: string;
  @IsUUID()
```

```
  supplierId: string;
  @IsUUID()
  warehouseId: string;
  @IsString()
  @IsOptional()
  contact?: string;
  @IsString()
  @IsOptional()
  phone?: string;
  @IsString()
  @IsOptional()
  deliveryDate?: string;
  @IsString()
  @IsOptional()
  remark?: string;
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => PurchaseOrderItemDto)
  items: PurchaseOrderItemDto[];
}
export class UpdatePurchaseOrderDto extends CreatePurchaseOrderDto {
  @IsString()
  @IsOptional()
  status?: string;
}
export class PurchaseOrderQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
  @IsUUID()
  @IsOptional()
  supplierId?: string;
  @IsUUID()
  @IsOptional()
  warehouseId?: string;
  @IsString()
  @IsOptional()
  status?: string;
  @IsString()
  @IsOptional()
  startDate?: string;
  @IsString()
  @IsOptional()
  endDate?: string;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
// 采购入库
export class PurchaseStockInItemDto {
  @IsUUID()
  productId: string;
  @IsUUID()
  @IsOptional()
  purchaseOrderId?: string;
  @IsUUID()
  @IsOptional()
  purchaseOrderItemId?: string;
  @Type(() => Number)
  @IsNumber()
  quantity: number;
  @Type(() => Number)
  @IsNumber()
  price: number;
  @Type(() => Number)
  @IsNumber()
  amount: number;
}
```

```
export class CreatePurchaseStockInDto {
  @IsString()
  @MinLength(1)
  stockInDate: string;

  @IsUUID()
  supplierId: string;

  @IsUUID()
  warehouseId: string;

  @IsString()
  @IsOptional()
  remark?: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => PurchaseStockInItemDto)
  items: PurchaseStockInItemDto[];
}

export class PurchaseStockInQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  supplierId?: string;

  @IsUUID()
  @IsOptional()
  warehouseId?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @IsString()
  @IsOptional()
  startDate?: string;

  @IsString()
  @IsOptional()
  endDate?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}

// 采购退货
export class CreatePurchaseReturnDto {
  @IsString()
  @MinLength(1)
  returnDate: string;

  @IsUUID()
  supplierId: string;

  @IsUUID()
  warehouseId: string;

  @IsUUID()
  @IsOptional()
  relateStockInId?: string;

  @IsString()
  @IsOptional()
  remark?: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => PurchaseStockInItemDto)
  items: PurchaseStockInItemDto[];
}

export class PurchaseReturnQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  supplierId?: string;

  @IsString()
  @IsOptional()
```

```
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/purchase/purchase.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { PurchaseService } from './purchase.service';
import {
  CreatePurchaseOrderDto, UpdatePurchaseOrderDto, PurchaseOrderQueryDto,
  CreatePurchaseStockInDto, PurchaseStockInQueryDto,
  CreatePurchaseReturnDto, PurchaseReturnQueryDto
} from './dto/purchase.dto';

@Controller('purchase')
@UseGuards(AuthGuard('jwt'))
export class PurchaseController {
  constructor(private purchaseService: PurchaseService) {}

  // ========== 采购订单 ==========
  @Post('orders')
  createOrder(@Body() dto: CreatePurchaseOrderDto & { tenantId: string }) {
    return this.purchaseService.createOrder(dto.tenantId, dto);
  }

  @Get('orders')
  findOrders(@Query() query: PurchaseOrderQueryDto, @Body('tenantId') tenantId: string) {
    return this.purchaseService.findOrders(tenantId, query);
  }

  @Get('orders/:id')
  findOrderById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.purchaseService.findOrderById(tenantId, id);
  }

  @Put('orders/:id')
  updateOrder(@Param('id') id: string, @Body() dto: UpdatePurchaseOrderDto & { tenantId: string }) {
    return this.purchaseService.updateOrder(dto.tenantId, id, dto);
  }

  @Patch('orders/:id/audit')
  auditOrder(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.purchaseService.auditOrder(body.tenantId, id, body.userId);
  }

  @Patch('orders/:id/cancel')
  cancelOrder(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.purchaseService.cancelOrder(tenantId, id);
  }

  @Delete('orders/:id')
  deleteOrder(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.purchaseService.deleteOrder(tenantId, id);
  }

  // ========== 采购入库 ==========
  @Post('stock-ins')
  createStockIn(@Body() dto: CreatePurchaseStockInDto & { tenantId: string }) {
    return this.purchaseService.createStockIn(dto.tenantId, dto);
  }

  @Get('stock-ins')
  findStockIns(@Query() query: PurchaseStockInQueryDto, @Body('tenantId') tenantId: string) {
    return this.purchaseService.findStockIns(tenantId, query);
  }

  @Get('stock-ins/:id')
  findStockInById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.purchaseService.findStockInById(tenantId, id);
  }

  @Patch('stock-ins/:id/audit')
  auditStockIn(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.purchaseService.auditStockIn(body.tenantId, id, body.userId);
  }

  // ========== 采购退货 ==========
  @Post('returns')
  createReturn(@Body() dto: CreatePurchaseReturnDto & { tenantId: string }) {
    return this.purchaseService.createReturn(dto.tenantId, dto);
```

```
  }
  @Get('returns')
  findReturns(@Query() query: PurchaseReturnQueryDto, @Body('tenantId') tenantId: string) {
    return this.purchaseService.findReturns(tenantId, query);
  }
  @Patch('returns/:id/audit')
  auditReturn(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.purchaseService.auditReturn(body.tenantId, id, body.userId);
  }
}
```

src/modules/purchase/purchase.module.ts

```
import { Module } from '@nestjs/common';
import { PurchaseController } from './purchase.controller';
import { PurchaseService } from './purchase.service';

@Module({
  controllers: [PurchaseController],
  providers: [PurchaseService],
  exports: [PurchaseService],
})
export class PurchaseModule {}
```

src/modules/purchase/purchase.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreatePurchaseOrderDto, UpdatePurchaseOrderDto, PurchaseOrderQueryDto,
  CreatePurchaseStockInDto, PurchaseStockInQueryDto,
  CreatePurchaseReturnDto, PurchaseReturnQueryDto } from './dto/purchase.dto';

@Injectable()
export class PurchaseService {
  constructor(private prisma: PrismaService) {}

  // ========== 采购订单 ==========
  async createOrder(tenantId: string, dto: CreatePurchaseOrderDto) {
    const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);

    return this.prisma.purchaseOrder.create({
      data: {
        tenantId,
        orderDate: new Date(dto.orderDate),
        supplierId: dto.supplierId,
        warehouseId: dto.warehouseId,
        contact: dto.contact,
        phone: dto.phone,
        deliveryDate: dto.deliveryDate ? new Date(dto.deliveryDate) : null,
        remark: dto.remark,
        totalAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            warehouseId: item.warehouseId || dto.warehouseId,
            quantity: item.quantity,
            price: item.price,
            amount: item.amount,
            taxRate: item.taxRate || 0,
            taxAmount: item.taxAmount || 0,
            remark: item.remark,
          })),
        },
      },
      include: { items: true, supplier: true, warehouse: true },
    });
  }

  async findOrders(tenantId: string, query: PurchaseOrderQueryDto) {
    const { keyword, supplierId, warehouseId, status, startDate, endDate, page = 1, pageSize = 20 }
= query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [{ orderNo: { contains: keyword } }];
    }
    if (supplierId) where.supplierId = supplierId;
    if (warehouseId) where.warehouseId = warehouseId;
    if (status) where.status = status;
    if (startDate || endDate) {
      where.orderDate = {};
      if (startDate) where.orderDate.gte = new Date(startDate);
      if (endDate) where.orderDate.lte = new Date(endDate);
```

```
  }
  const [list, total] = await Promise.all([
    this.prisma.purchaseOrder.findMany({
      where,
      include: { supplier: true, warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.purchaseOrder.count({ where }),
  ]);
  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}
async findOrderById(tenantId: string, id: string) {
  return this.prisma.purchaseOrder.findFirst({
    where: { id, tenantId },
    include: { items: { include: { product: true } }, supplier: true, warehouse: true },
  });
}
async updateOrder(tenantId: string, id: string, dto: UpdatePurchaseOrderDto) {
  const order = await this.prisma.purchaseOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending') throw new BadRequestException('只能编辑待审核订单');
  const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);
  // 删除旧明细，创建新明细
  await this.prisma.purchaseOrderItem.deleteMany({ where: { orderId: id } });
  return this.prisma.purchaseOrder.update({
    where: { id },
    data: {
      orderDate: new Date(dto.orderDate),
      supplierId: dto.supplierId,
      warehouseId: dto.warehouseId,
      contact: dto.contact,
      phone: dto.phone,
      deliveryDate: dto.deliveryDate ? new Date(dto.deliveryDate) : null,
      remark: dto.remark,
      totalAmount,
      items: {
        create: dto.items.map(item => ({
          tenantId,
          productId: item.productId,
          warehouseId: item.warehouseId || dto.warehouseId,
          quantity: item.quantity,
          price: item.price,
          amount: item.amount,
          taxRate: item.taxRate || 0,
          taxAmount: item.taxAmount || 0,
        })),
      },
    },
    include: { items: true },
  });
}
async auditOrder(tenantId: string, id: string, userId: string) {
  const order = await this.prisma.purchaseOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending') throw new BadRequestException('只能审核待审核订单');
  return this.prisma.purchaseOrder.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}
async cancelOrder(tenantId: string, id: string) {
  const order = await this.prisma.purchaseOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status === 'completed') throw new BadRequestException('已完成订单无法取消');
  return this.prisma.purchaseOrder.update({
    where: { id },
    data: { status: 'cancelled' },
  });
}
async deleteOrder(tenantId: string, id: string) {
  const order = await this.prisma.purchaseOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending' && order.status !== 'cancelled') {
    throw new BadRequestException('只能删除待审核或已取消订单');
  }
```

```
  await this.prisma.purchaseOrderItem.deleteMany({ where: { orderId: id } });
  await this.prisma.purchaseOrder.delete({ where: { id } });
}

// ========== 采购入库 ==========
async createStockIn(tenantId: string, dto: CreatePurchaseStockInDto) {
  const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);

  const result = await this.prisma.$transaction(async (tx) => {
    const stockIn = await tx.purchaseStockIn.create({
      data: {
        tenantId,
        stockInDate: new Date(dto.stockInDate),
        supplierId: dto.supplierId,
        warehouseId: dto.warehouseId,
        remark: dto.remark,
        totalAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            purchaseOrderId: item.purchaseOrderId,
            purchaseOrderItemId: item.purchaseOrderItemId,
            quantity: item.quantity,
            price: item.price,
            amount: item.amount,
          })),
        },
      },
      include: { items: true },
    });

    // 更新库存
    for (const item of dto.items) {
      await tx.inventory.upsert({
        where: {
          tenantId_productId_warehouseId: {
            tenantId,
            productId: item.productId,
            warehouseId: dto.warehouseId,
          },
        },
        create: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.warehouseId,
          quantity: item.quantity,
          avgCostPrice: item.price,
          lastInPrice: item.price,
        },
        update: {
          quantity: { increment: item.quantity },
          avgCostPrice: item.price, // 简化处理
          lastInPrice: item.price,
        },
      });

      // 库存变动日志
      await tx.inventoryLog.create({
        data: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.warehouseId,
          businessType: 'purchase_in',
          businessId: stockIn.id,
          businessNo: stockIn.orderNo,
          quantity: item.quantity,
          beforeQuantity: 0, // 简化
          afterQuantity: item.quantity,
          price: item.price,
          amount: item.amount,
        },
      });
    }

    return stockIn;
  });

  return result;
}

async findStockIns(tenantId: string, query: PurchaseStockInQueryDto) {
  const { keyword, supplierId, warehouseId, status, startDate, endDate, page = 1, pageSize = 20 }
= query;

  const where: any = { tenantId };
```

```
  if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
  if (supplierId) where.supplierId = supplierId;
  if (warehouseId) where.warehouseId = warehouseId;
  if (status) where.status = status;
  if (startDate || endDate) {
    where.stockInDate = {};
    if (startDate) where.stockInDate.gte = new Date(startDate);
    if (endDate) where.stockInDate.lte = new Date(endDate);
  }

  const [list, total] = await Promise.all([
    this.prisma.purchaseStockIn.findMany({
      where,
      include: { supplier: true, warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.purchaseStockIn.count({ where }),
  ]);

  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}

async findStockInById(tenantId: string, id: string) {
  return this.prisma.purchaseStockIn.findFirst({
    where: { id, tenantId },
    include: { items: { include: { product: true } }, supplier: true, warehouse: true },
  });
}

async auditStockIn(tenantId: string, id: string, userId: string) {
  const stockIn = await this.prisma.purchaseStockIn.findFirst({ where: { id, tenantId } });
  if (!stockIn) throw new BadRequestException('入库单不存在');
  if (stockIn.status !== 'pending') throw new BadRequestException('只能审核待审核入库单');

  return this.prisma.purchaseStockIn.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}

// ========== 采购退货 ==========
async createReturn(tenantId: string, dto: CreatePurchaseReturnDto) {
  const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);

  const result = await this.prisma.$transaction(async (tx) => {
    const returnOrder = await tx.purchaseStockIn.create({
      data: {
        tenantId,
        orderNo: `PR${Date.now()}`, // 后续优化为编号规则
        stockInDate: new Date(dto.returnDate),
        supplierId: dto.supplierId,
        warehouseId: dto.warehouseId,
        remark: dto.remark,
        totalAmount: -totalAmount,
        status: 'pending',
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            purchaseOrderId: item.purchaseOrderId,
            quantity: -item.quantity, // 负数表示退货
            price: item.price,
            amount: -item.amount,
          })),
        },
      },
      include: { items: true },
    });

    // 减少库存
    for (const item of dto.items) {
      await tx.inventory.update({
        where: {
          tenantId_productId_warehouseId: {
            tenantId,
            productId: item.productId,
            warehouseId: dto.warehouseId,
          },
        },
        data: { quantity: { decrement: item.quantity } },
      });

      await tx.inventoryLog.create({
        data: {
          tenantId,
```

```
          productId: item.productId,
          warehouseId: dto.warehouseId,
          businessType: 'purchase_return',
          businessId: returnOrder.id,
          businessNo: returnOrder.orderNo,
          quantity: -item.quantity,
          price: item.price,
          amount: -item.amount,
        },
      });
    }

    return returnOrder;
  });

  return result;
}

async findReturns(tenantId: string, query: PurchaseReturnQueryDto) {
  const { keyword, supplierId, status, page = 1, pageSize = 20 } = query;

  const where: any = {
    tenantId,
    totalAmount: { lt: 0 }, // 退货单金额为负
  };
  if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
  if (supplierId) where.supplierId = supplierId;
  if (status) where.status = status;

  const [list, total] = await Promise.all([
    this.prisma.purchaseStockIn.findMany({
      where,
      include: { supplier: true, warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.purchaseStockIn.count({ where }),
  ]);

  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}

async auditReturn(tenantId: string, id: string, userId: string) {
  const returnOrder = await this.prisma.purchaseStockIn.findFirst({ where: { id, tenantId } });
  if (!returnOrder) throw new BadRequestException('退货单不存在');
  if (returnOrder.status !== 'pending') throw new BadRequestException('只能审核待审核退货单');

  return this.prisma.purchaseStockIn.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}
}
```

src/modules/sale/dto/sale.dto.ts

```
import { IsString, IsOptional, IsNumber, IsUUID, IsArray, ValidateNested, MinLength } from 'class-va
lidator';
import { Type } from 'class-transformer';

export class SaleOrderItemDto {
  @IsUUID()
  @IsOptional()
  id?: string;

  @IsUUID()
  productId: string;

  @IsUUID()
  @IsOptional()
  warehouseId?: string;

  @Type(() => Number)
  @IsNumber()
  quantity: number;

  @Type(() => Number)
  @IsNumber()
  price: number;

  @Type(() => Number)
  @IsNumber()
  amount: number;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  taxRate?: number;

  @Type(() => Number)
```

```
  @IsNumber()
  @IsOptional()
  taxAmount?: number;
}
export class CreateSaleOrderDto {
  @IsString()
  @MinLength(1)
  orderDate: string;
  @IsUUID()
  customerId: string;
  @IsUUID()
  warehouseId: string;
  @IsString()
  @IsOptional()
  contact?: string;
  @IsString()
  @IsOptional()
  phone?: string;
  @IsString()
  @IsOptional()
  deliveryDate?: string;
  @IsString()
  @IsOptional()
  remark?: string;
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => SaleOrderItemDto)
  items: SaleOrderItemDto[];
}
export class UpdateSaleOrderDto extends CreateSaleOrderDto {
  @IsString()
  @IsOptional()
  status?: string;
}
export class SaleOrderQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;
  @IsUUID()
  @IsOptional()
  customerId?: string;
  @IsUUID()
  @IsOptional()
  warehouseId?: string;
  @IsString()
  @IsOptional()
  status?: string;
  @IsString()
  @IsOptional()
  startDate?: string;
  @IsString()
  @IsOptional()
  endDate?: string;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;
  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
// 销售出库
export class SaleStockOutItemDto {
  @IsUUID()
  productId: string;
  @IsUUID()
  @IsOptional()
  saleOrderId?: string;
  @IsUUID()
  @IsOptional()
  saleOrderItemId?: string;
  @Type(() => Number)
```

```
  @IsNumber()
  quantity: number;

  @Type(() => Number)
  @IsNumber()
  price: number;

  @Type(() => Number)
  @IsNumber()
  amount: number;
}

export class CreateSaleStockOutDto {
  @IsString()
  @MinLength(1)
  stockOutDate: string;

  @IsUUID()
  customerId: string;

  @IsUUID()
  warehouseId: string;

  @IsString()
  @IsOptional()
  remark?: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => SaleStockOutItemDto)
  items: SaleStockOutItemDto[];
}

export class SaleStockOutQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  customerId?: string;

  @IsUUID()
  @IsOptional()
  warehouseId?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @IsString()
  @IsOptional()
  startDate?: string;

  @IsString()
  @IsOptional()
  endDate?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}

// 销售退货
export class CreateSaleReturnDto {
  @IsString()
  @MinLength(1)
  returnDate: string;

  @IsUUID()
  customerId: string;

  @IsUUID()
  warehouseId: string;

  @IsUUID()
  @IsOptional()
  relateStockOutId?: string;

  @IsString()
  @IsOptional()
  remark?: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => SaleStockOutItemDto)
  items: SaleStockOutItemDto[];
}
```

```
export class SaleReturnQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsUUID()
  @IsOptional()
  customerId?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/sale/sale.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { SaleService } from './sale.service';
import {
  CreateSaleOrderDto, UpdateSaleOrderDto, SaleOrderQueryDto,
  CreateSaleStockOutDto, SaleStockOutQueryDto,
  CreateSaleReturnDto, SaleReturnQueryDto
} from './dto/sale.dto';

@Controller('sale')
@UseGuards(AuthGuard('jwt'))
export class SaleController {
  constructor(private saleService: SaleService) {}

  // ========== 销售订单 ==========
  @Post('orders')
  createOrder(@Body() dto: CreateSaleOrderDto & { tenantId: string }) {
    return this.saleService.createOrder(dto.tenantId, dto);
  }

  @Get('orders')
  findOrders(@Query() query: SaleOrderQueryDto, @Body('tenantId') tenantId: string) {
    return this.saleService.findOrders(tenantId, query);
  }

  @Get('orders/:id')
  findOrderById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.saleService.findOrderById(tenantId, id);
  }

  @Put('orders/:id')
  updateOrder(@Param('id') id: string, @Body() dto: UpdateSaleOrderDto & { tenantId: string }) {
    return this.saleService.updateOrder(dto.tenantId, id, dto);
  }

  @Patch('orders/:id/audit')
  auditOrder(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.saleService.auditOrder(body.tenantId, id, body.userId);
  }

  @Patch('orders/:id/cancel')
  cancelOrder(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.saleService.cancelOrder(tenantId, id);
  }

  @Delete('orders/:id')
  deleteOrder(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.saleService.deleteOrder(tenantId, id);
  }

  // ========== 销售出库 ==========
  @Post('stock-outs')
  createStockOut(@Body() dto: CreateSaleStockOutDto & { tenantId: string }) {
    return this.saleService.createStockOut(dto.tenantId, dto);
  }

  @Get('stock-outs')
  findStockOuts(@Query() query: SaleStockOutQueryDto, @Body('tenantId') tenantId: string) {
    return this.saleService.findStockOuts(tenantId, query);
  }

  @Get('stock-outs/:id')
  findStockOutById(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.saleService.findStockOutById(tenantId, id);
```

```
  }
  @Patch('stock-outs/:id/audit')
  auditStockOut(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.saleService.auditStockOut(body.tenantId, id, body.userId);
  }
  // ========== 销售退货 ==========
  @Post('returns')
  createReturn(@Body() dto: CreateSaleReturnDto & { tenantId: string }) {
    return this.saleService.createReturn(dto.tenantId, dto);
  }
  @Get('returns')
  findReturns(@Query() query: SaleReturnQueryDto, @Body('tenantId') tenantId: string) {
    return this.saleService.findReturns(tenantId, query);
  }
  @Patch('returns/:id/audit')
  auditReturn(@Param('id') id: string, @Body() body: { tenantId: string; userId: string }) {
    return this.saleService.auditReturn(body.tenantId, id, body.userId);
  }
}
```

src/modules/sale/sale.module.ts

```
import { Module } from '@nestjs/common';
import { SaleController } from './sale.controller';
import { SaleService } from './sale.service';

@Module({
  controllers: [SaleController],
  providers: [SaleService],
  exports: [SaleService],
})
export class SaleModule {}
```

src/modules/sale/sale.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateSaleOrderDto, UpdateSaleOrderDto, SaleOrderQueryDto,
  CreateSaleStockOutDto, SaleStockOutQueryDto,
  CreateSaleReturnDto, SaleReturnQueryDto } from './dto/sale.dto';

@Injectable()
export class SaleService {
  constructor(private prisma: PrismaService) {}

  // ========== 销售订单 ==========
  async createOrder(tenantId: string, dto: CreateSaleOrderDto) {
    const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);

    return this.prisma.saleOrder.create({
      data: {
        tenantId,
        orderDate: new Date(dto.orderDate),
        customerId: dto.customerId,
        warehouseId: dto.warehouseId,
        contact: dto.contact,
        phone: dto.phone,
        deliveryDate: dto.deliveryDate ? new Date(dto.deliveryDate) : null,
        remark: dto.remark,
        totalAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            warehouseId: item.warehouseId || dto.warehouseId,
            quantity: item.quantity,
            price: item.price,
            amount: item.amount,
            taxRate: item.taxRate || 0,
            taxAmount: item.taxAmount || 0,
          })),
        },
      },
      include: { items: true, customer: true, warehouse: true },
    });
  }

  async findOrders(tenantId: string, query: SaleOrderQueryDto) {
    const { keyword, customerId, warehouseId, status, startDate, endDate, page = 1, pageSize = 20 }
= query;

    const where: any = { tenantId };
    if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
    if (customerId) where.customerId = customerId;
```

```
  if (warehouseId) where.warehouseId = warehouseId;
  if (status) where.status = status;
  if (startDate || endDate) {
    where.orderDate = {};
    if (startDate) where.orderDate.gte = new Date(startDate);
    if (endDate) where.orderDate.lte = new Date(endDate);
  }

  const [list, total] = await Promise.all([
    this.prisma.saleOrder.findMany({
      where,
      include: { customer: true, warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.saleOrder.count({ where }),
  ]);

  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}

async findOrderById(tenantId: string, id: string) {
  return this.prisma.saleOrder.findFirst({
    where: { id, tenantId },
    include: { items: { include: { product: true } }, customer: true, warehouse: true },
  });
}

async updateOrder(tenantId: string, id: string, dto: UpdateSaleOrderDto) {
  const order = await this.prisma.saleOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending') throw new BadRequestException('只能编辑待审核订单');

  const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);

  await this.prisma.saleOrderItem.deleteMany({ where: { orderId: id } });

  return this.prisma.saleOrder.update({
    where: { id },
    data: {
      orderDate: new Date(dto.orderDate),
      customerId: dto.customerId,
      warehouseId: dto.warehouseId,
      contact: dto.contact,
      phone: dto.phone,
      deliveryDate: dto.deliveryDate ? new Date(dto.deliveryDate) : null,
      remark: dto.remark,
      totalAmount,
      items: {
        create: dto.items.map(item => ({
          tenantId,
          productId: item.productId,
          warehouseId: item.warehouseId || dto.warehouseId,
          quantity: item.quantity,
          price: item.price,
          amount: item.amount,
          taxRate: item.taxRate || 0,
          taxAmount: item.taxAmount || 0,
        })),
      },
    },
    include: { items: true },
  });
}

async auditOrder(tenantId: string, id: string, userId: string) {
  const order = await this.prisma.saleOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending') throw new BadRequestException('只能审核待审核订单');

  return this.prisma.saleOrder.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}

async cancelOrder(tenantId: string, id: string) {
  const order = await this.prisma.saleOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status === 'completed') throw new BadRequestException('已完成订单无法取消');

  return this.prisma.saleOrder.update({
    where: { id },
    data: { status: 'cancelled' },
  });
}

async deleteOrder(tenantId: string, id: string) {
```

```
  const order = await this.prisma.saleOrder.findFirst({ where: { id, tenantId } });
  if (!order) throw new BadRequestException('订单不存在');
  if (order.status !== 'pending' && order.status !== 'cancelled') {
    throw new BadRequestException('只能删除待审核或已取消订单');
  }
  await this.prisma.saleOrderItem.deleteMany({ where: { orderId: id } });
  await this.prisma.saleOrder.delete({ where: { id } });
}
// ========== 销售出库 ==========
async createStockOut(tenantId: string, dto: CreateSaleStockOutDto) {
  const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);
  const result = await this.prisma.$transaction(async (tx) => {
    const stockOut = await tx.saleStockOut.create({
      data: {
        tenantId,
        stockOutDate: new Date(dto.stockOutDate),
        customerId: dto.customerId,
        warehouseId: dto.warehouseId,
        remark: dto.remark,
        totalAmount,
        items: {
          create: dto.items.map(item => ({
            tenantId,
            productId: item.productId,
            saleOrderId: item.saleOrderId,
            saleOrderItemId: item.saleOrderItemId,
            quantity: item.quantity,
            price: item.price,
            amount: item.amount,
          })),
        },
      },
      include: { items: true },
    });
    // 更新库存（减少）
    for (const item of dto.items) {
      const inv = await tx.inventory.findFirst({
        where: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.warehouseId,
        },
      });
      if (!inv || inv.quantity < item.quantity) {
        throw new BadRequestException(`商品库存不足`);
      }
      await tx.inventory.update({
        where: {
          tenantId_productId_warehouseId: {
            tenantId,
            productId: item.productId,
            warehouseId: dto.warehouseId,
          },
        },
        data: { quantity: { decrement: item.quantity } },
      });
      await tx.inventoryLog.create({
        data: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.warehouseId,
          businessType: 'sale_out',
          businessId: stockOut.id,
          businessNo: stockOut.orderNo,
          quantity: -item.quantity,
          beforeQuantity: inv.quantity,
          afterQuantity: inv.quantity - item.quantity,
          price: item.price,
          amount: item.amount,
        },
      });
    }
    return stockOut;
  });
  return result;
}
async findStockOuts(tenantId: string, query: SaleStockOutQueryDto) {
```

```
    const { keyword, customerId, warehouseId, status, startDate, endDate, page = 1, pageSize = 20 }
= query;
    const where: any = { tenantId };
    if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
    if (customerId) where.customerId = customerId;
    if (warehouseId) where.warehouseId = warehouseId;
    if (status) where.status = status;
    if (startDate || endDate) {
      where.stockOutDate = {};
      if (startDate) where.stockOutDate.gte = new Date(startDate);
      if (endDate) where.stockOutDate.lte = new Date(endDate);
    }
    const [list, total] = await Promise.all([
      this.prisma.saleStockOut.findMany({
        where,
        include: { customer: true, warehouse: true },
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.saleStockOut.count({ where }),
    ]);
    return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
  }
  async findStockOutById(tenantId: string, id: string) {
    return this.prisma.saleStockOut.findFirst({
      where: { id, tenantId },
      include: { items: { include: { product: true } }, customer: true, warehouse: true },
    });
  }
  async auditStockOut(tenantId: string, id: string, userId: string) {
    const stockOut = await this.prisma.saleStockOut.findFirst({ where: { id, tenantId } });
    if (!stockOut) throw new BadRequestException('出库单不存在');
    if (stockOut.status !== 'pending') throw new BadRequestException('只能审核待审核出库单');
    return this.prisma.saleStockOut.update({
      where: { id },
      data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
    });
  }
  // ========== 销售退货 ==========
  async createReturn(tenantId: string, dto: CreateSaleReturnDto) {
    const totalAmount = dto.items.reduce((sum, item) => sum + item.amount, 0);
    const result = await this.prisma.$transaction(async (tx) => {
      const returnOrder = await tx.saleStockOut.create({
        data: {
          tenantId,
          orderNo: `SR${Date.now()}`,
          stockOutDate: new Date(dto.returnDate),
          customerId: dto.customerId,
          warehouseId: dto.warehouseId,
          remark: dto.remark,
          totalAmount: -totalAmount,
          status: 'pending',
          items: {
            create: dto.items.map(item => ({
              tenantId,
              productId: item.productId,
              saleOrderId: item.saleOrderId,
              quantity: -item.quantity,
              price: item.price,
              amount: -item.amount,
            })),
          },
        },
        include: { items: true },
      });
      // 增加库存
      for (const item of dto.items) {
        await tx.inventory.upsert({
          where: {
            tenantId_productId_warehouseId: {
              tenantId,
              productId: item.productId,
              warehouseId: dto.warehouseId,
            },
          },
          create: {
            tenantId,
```

```
          productId: item.productId,
          warehouseId: dto.warehouseId,
          quantity: item.quantity,
          lastOutPrice: item.price,
        },
        update: {
          quantity: { increment: item.quantity },
          lastOutPrice: item.price,
        },
      });
      await tx.inventoryLog.create({
        data: {
          tenantId,
          productId: item.productId,
          warehouseId: dto.warehouseId,
          businessType: 'sale_return',
          businessId: returnOrder.id,
          businessNo: returnOrder.orderNo,
          quantity: item.quantity,
          price: item.price,
          amount: item.amount,
        },
      });
    }
    return returnOrder;
  });
  return result;
}
async findReturns(tenantId: string, query: SaleReturnQueryDto) {
  const { keyword, customerId, status, page = 1, pageSize = 20 } = query;
  const where: any = {
    tenantId,
    totalAmount: { lt: 0 },
  };
  if (keyword) where.OR = [{ orderNo: { contains: keyword } }];
  if (customerId) where.customerId = customerId;
  if (status) where.status = status;
  const [list, total] = await Promise.all([
    this.prisma.saleStockOut.findMany({
      where,
      include: { customer: true, warehouse: true },
      skip: (page - 1) * pageSize,
      take: pageSize,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.saleStockOut.count({ where }),
  ]);
  return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
}
async auditReturn(tenantId: string, id: string, userId: string) {
  const returnOrder = await this.prisma.saleStockOut.findFirst({ where: { id, tenantId } });
  if (!returnOrder) throw new BadRequestException('退货单不存在');
  if (returnOrder.status !== 'pending') throw new BadRequestException('只能审核待审核退货单');
  return this.prisma.saleStockOut.update({
    where: { id },
    data: { status: 'approved', auditedById: userId, auditedAt: new Date() },
  });
}
}
```

src/modules/supplier/dto/supplier.dto.ts

```
import { IsString, IsOptional, IsNumber, MinLength, MaxLength } from 'class-validator';
import { Type } from 'class-transformer';

export class CreateSupplierDto {
  @IsString()
  @MinLength(1)
  @MaxLength(50)
  code: string;

  @IsString()
  @MinLength(1)
  @MaxLength(200)
  name: string;

  @IsString()
  @IsOptional()
  contact?: string;
```

```
  @IsString()
  @IsOptional()
  phone?: string;

  @IsString()
  @IsOptional()
  mobile?: string;

  @IsString()
  @IsOptional()
  email?: string;

  @IsString()
  @IsOptional()
  address?: string;

  @IsString()
  @IsOptional()
  bankName?: string;

  @IsString()
  @IsOptional()
  bankAccount?: string;

  @IsString()
  @IsOptional()
  taxNo?: string;
}

export class UpdateSupplierDto extends CreateSupplierDto {
  @IsString()
  @IsOptional()
  status?: string;
}

export class SupplierQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @Type(() => Number)
  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}
```

src/modules/supplier/supplier.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { SupplierService } from './supplier.service';
import { CreateSupplierDto, UpdateSupplierDto, SupplierQueryDto } from './dto/supplier.dto';

@Controller('suppliers')
@UseGuards(AuthGuard('jwt'))
export class SupplierController {
  constructor(private supplierService: SupplierService) {}

  @Post()
  create(@Body() dto: CreateSupplierDto & { tenantId: string }) {
    return this.supplierService.create(dto.tenantId, dto);
  }

  @Get()
  findAll(@Query() query: SupplierQueryDto, @Body('tenantId') tenantId: string) {
    return this.supplierService.findAll(tenantId, query);
  }

  @Get(':id')
  findOne(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.supplierService.findOne(tenantId, id);
  }

  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateSupplierDto & { tenantId: string }) {
    return this.supplierService.update(dto.tenantId, id, dto);
  }

  @Delete(':id')
  remove(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.supplierService.remove(tenantId, id);
  }
```

```
  @Patch(':id/status')
  updateStatus(@Param('id') id: string, @Body() body: { tenantId: string; status: string }) {
    return this.supplierService.updateStatus(body.tenantId, id, body.status);
  }
}
```

src/modules/supplier/supplier.module.ts

```
import { Module } from '@nestjs/common';
import { SupplierController } from './supplier.controller';
import { SupplierService } from './supplier.service';

@Module({
  controllers: [SupplierController],
  providers: [SupplierService],
  exports: [SupplierService],
})
export class SupplierModule {}
```

src/modules/supplier/supplier.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateSupplierDto, UpdateSupplierDto, SupplierQueryDto } from './dto/supplier.dto';

@Injectable()
export class SupplierService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateSupplierDto) {
    const exists = await this.prisma.supplier.findFirst({
      where: { tenantId, code: dto.code },
    });
    if (exists) throw new BadRequestException('供应商编码已存在');

    return this.prisma.supplier.create({ data: { tenantId, ...dto } });
  }

  async findAll(tenantId: string, query: SupplierQueryDto) {
    const { keyword, status, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [
        { code: { contains: keyword } },
        { name: { contains: keyword } },
        { contact: { contains: keyword } },
      ];
    }
    if (status) where.status = status;

    const [list, total] = await Promise.all([
      this.prisma.supplier.findMany({
        where,
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.supplier.count({ where }),
    ]);

    return { list, pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) } };
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.supplier.findFirst({ where: { id, tenantId } });
  }

  async update(tenantId: string, id: string, dto: UpdateSupplierDto) {
    const exists = await this.prisma.supplier.findFirst({ where: { id, tenantId } });
    if (!exists) throw new BadRequestException('供应商不存在');

    if (dto.code && dto.code !== exists.code) {
      const codeExists = await this.prisma.supplier.findFirst({
        where: { tenantId, code: dto.code, id: { not: id } },
      });
      if (codeExists) throw new BadRequestException('供应商编码已存在');
    }

    return this.prisma.supplier.update({ where: { id }, data: dto });
  }

  async remove(tenantId: string, id: string) {
    await this.prisma.supplier.delete({ where: { id, tenantId } });
  }

  async updateStatus(tenantId: string, id: string, status: string) {
    return this.prisma.supplier.update({ where: { id, tenantId }, data: { status } });
  }
}
```

src/modules/warehouse/dto/warehouse.dto.ts

```
import { IsString, IsOptional, IsNumber, MinLength, MaxLength } from 'class-validator';
import { Prisma } from '@prisma/client';

export class CreateWarehouseDto {
  @IsString()
  @MinLength(1)
  @MaxLength(50)
  code: string;

  @IsString()
  @MinLength(1)
  @MaxLength(100)
  name: string;

  @IsString()
  @IsOptional()
  address?: string;

  @IsString()
  @IsOptional()
  principal?: string;

  @IsNumber()
  @IsOptional()
  sortOrder?: number;
}

export class UpdateWarehouseDto extends CreateWarehouseDto {
  @IsString()
  @IsOptional()
  status?: string;
}

export class WarehouseQueryDto {
  @IsString()
  @IsOptional()
  keyword?: string;

  @IsString()
  @IsOptional()
  status?: string;

  @IsNumber()
  @IsOptional()
  page?: number = 1;

  @IsNumber()
  @IsOptional()
  pageSize?: number = 20;
}

export const warehouseSelect = {
  id: true,
  code: true,
  name: true,
  address: true,
  principal: true,
  sortOrder: true,
  status: true,
  createdAt: true,
  updatedAt: true,
} as const;
```

src/modules/warehouse/warehouse.controller.ts

```
import { Controller, Get, Post, Put, Delete, Patch, Body, Param, Query, UseGuards } from '@nestjs/co
mmon';
import { AuthGuard } from '@nestjs/passport';
import { WarehouseService } from './warehouse.service';
import { CreateWarehouseDto, UpdateWarehouseDto, WarehouseQueryDto } from './dto/warehouse.dto';

@Controller('warehouses')
@UseGuards(AuthGuard('jwt'))
export class WarehouseController {
  constructor(private warehouseService: WarehouseService) {}

  @Post()
  create(@Body() dto: CreateWarehouseDto) {
    return this.warehouseService.create(dto.tenantId, dto);
  }

  @Get()
  findAll(@Query() query: WarehouseQueryDto, @Body('tenantId') tenantId: string) {
    return this.warehouseService.findAll(tenantId, query);
  }

  @Get(':id')
  findOne(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.warehouseService.findOne(tenantId, id);
```

```
  }
  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateWarehouseDto & { tenantId: string }) {
    return this.warehouseService.update(dto.tenantId, id, dto);
  }
  @Delete(':id')
  remove(@Param('id') id: string, @Body('tenantId') tenantId: string) {
    return this.warehouseService.remove(tenantId, id);
  }
  @Patch(':id/status')
  updateStatus(
    @Param('id') id: string,
    @Body() body: { tenantId: string; status: string },
  ) {
    return this.warehouseService.updateStatus(body.tenantId, id, body.status);
  }
}
```

src/modules/warehouse/warehouse.module.ts

```
import { Module } from '@nestjs/common';
import { WarehouseController } from './warehouse.controller';
import { WarehouseService } from './warehouse.service';

@Module({
  controllers: [WarehouseController],
  providers: [WarehouseService],
  exports: [WarehouseService],
})
export class WarehouseModule {}
```

src/modules/warehouse/warehouse.service.ts

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateWarehouseDto, UpdateWarehouseDto, WarehouseQueryDto } from './dto/warehouse.dto';

@Injectable()
export class WarehouseService {
  constructor(private prisma: PrismaService) {}

  async create(tenantId: string, dto: CreateWarehouseDto) {
    // 检查编码唯一性
    const exists = await this.prisma.warehouse.findFirst({
      where: { tenantId, code: dto.code },
    });
    if (exists) {
      throw new BadRequestException('仓库编码已存在');
    }

    return this.prisma.warehouse.create({
      data: {
        tenantId,
        ...dto,
      },
    });
  }

  async findAll(tenantId: string, query: WarehouseQueryDto) {
    const { keyword, status, page = 1, pageSize = 20 } = query;

    const where: any = { tenantId };
    if (keyword) {
      where.OR = [
        { code: { contains: keyword } },
        { name: { contains: keyword } },
      ];
    }
    if (status) {
      where.status = status;
    }

    const [list, total] = await Promise.all([
      this.prisma.warehouse.findMany({
        where,
        skip: (page - 1) * pageSize,
        take: pageSize,
        orderBy: { sortOrder: 'asc' },
      }),
      this.prisma.warehouse.count({ where }),
    ]);

    return {
      list,
      pagination: { page, pageSize, total, totalPages: Math.ceil(total / pageSize) },
```

```
    };
  }

  async findOne(tenantId: string, id: string) {
    return this.prisma.warehouse.findFirst({
      where: { id, tenantId },
    });
  }

  async update(tenantId: string, id: string, dto: UpdateWarehouseDto) {
    const exists = await this.prisma.warehouse.findFirst({
      where: { id, tenantId },
    });
    if (!exists) {
      throw new BadRequestException('仓库不存在');
    }

    // 检查编码唯一性（排除自己）
    if (dto.code && dto.code !== exists.code) {
      const codeExists = await this.prisma.warehouse.findFirst({
        where: { tenantId, code: dto.code, id: { not: id } },
      });
      if (codeExists) {
        throw new BadRequestException('仓库编码已存在');
      }
    }

    return this.prisma.warehouse.update({
      where: { id },
      data: dto,
    });
  }

  async remove(tenantId: string, id: string) {
    await this.prisma.warehouse.delete({
      where: { id, tenantId },
    });
  }

  async updateStatus(tenantId: string, id: string, status: string) {
    return this.prisma.warehouse.update({
      where: { id, tenantId },
      data: { status },
    });
  }
}
```

src/prisma/prisma.module.ts

```
import { Global, Module } from '@nestjs/common';
import { PrismaService } from './prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

src/prisma/prisma.service.ts

```
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}

// 后端代码合计：约 4190 行
```

源代码总计：约 8706 行
前端：约 4516 行，后端：约 4190 行